



Universidade do Minho

Licenciatura em Engenharia Informática

Unidade Curricular de Bases de Dados

Ano Letivo de 2025/2026



NOMÁLIA

VIAJA, REGISTA, VIVE

**Ana Patrícia da Silva Machado(a111661),
Marta Alexandra Rodrigues Araújo(a110317),
Sara Vieira Marques(a111182),
Teresa Carolina Oliveira Teixeira(a111415).**

Dezembro, 2025

BD

Data de Recepção		
Responsável		
Avaliação		
Observações		

NOMÁLIA

**Ana Patrícia da Silva Machado(a111661),
Marta Alexandra Rodrigues Araújo(a110317),
Sara Vieira Marques(a111182),
Teresa Carolina Oliveira Teixeira(a111415).**

Grupo 47

Dezembro, 2025

Resumo

Doutora Filipa Martins, fundadora e diretora da empresa NOMÁLIA, sugeriu a um grupo de alunas de Engenharia Informática da Universidade do Minho, o desenvolvimento de um Sistema de Base de Dados Relacional, destinado à gestão de utilizadores, viagens, memórias e interações, na sua plataforma digital, dedicada à partilha de experiências de viagens pelo mundo. Este documento descreve todo o processo de desenvolvimento conduzido pela equipa de estudantes, desde a análise até à implementação final do sistema.

Área de Aplicação: Desenho e Arquitetura de Sistemas de Bases de Dados.

Palavras-Chave: Bases de Dados, Diagramas ER, Modelo Relacional, Modelação Lógica, Normalização de Bases de Dados Relacionais, Álgebra Relacional, SQL, MySQL, InnoDB

Índice

Resumo	1
Índice	2
Índice de Tabelas	4
Índice de Figuras	5
1. Definição do Sistema	6
1.1. Contexto de aplicação	6
Apresentação do Caso de Estudo	6
1.2. Motivação e Objetivos do Trabalho	7
1.3. Análise da Viabilidade do processo	8
1.4. Recursos e Equipa de Trabalho	9
1.5. Plano de Execução do Projeto	10
2. Levantamento e Análise de Requisitos	11
2.1. Método de Levantamento e de Análise de Requisitos Adotado	11
2.2. Organização dos Requisitos Levantados	12
2.3. Análise e Validação Geral dos Requisitos	18
3. Modelação Conceptual	19
3.1. Apresentação da Abordagem de Modelação Realizada	19
3.2. Identificação e Caracterização das Entidades	19
3.3. Identificação e Caracterização dos Relacionamentos	22
3.4. Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos	24
3.5. Apresentação e Explicação do Diagrama ER Produzido	26
4. Modelação Lógica	27
4.1. Construção e Validação do Modelo de Dados Lógico	27
4.2. Apresentação e Explicação do Modelo Lógico Produzido	28
4.3. Normalização de Dados	36
4.4. Validação do Modelo com Interrogações do Utilizador	39
5. Implementação física	47
5.1. Apresentação e explicação da base de dados implementada	47
5.2. Criação de utilizadores da base de dados	54
5.3. Povoamento da base de dados	58
5.4. Cálculo do espaço da base de dados (inicial e taxa de crescimento anual)	63
5.5. Definição e caracterização de vistas de utilização em SQL	67
5.6. Tradução das interrogações do utilizador para SQL	68
5.7. Indexação do Sistema de Dados	74
5.8. Implementação de procedimentos, funções e gatilhos	75
5.9. Backup da Base de Dados	83
6. Conclusões e Trabalho Futuro	84
7. Bibliografia	86
Lista de Siglas e Acrónimos	87
Anexos	88
I. Diagrama de Gantt	89
II. Ata de Reunião	90
III. Requisitos levantados	91
IV. Diagrama ER	93

V. Modelo Lógico	94
VI. Expressões de Álgebra Relacional	95
VII. Árvores de Interrogação	96
VIII. Grafo do diagrama de dependências	100
IX. Modelo Físico	101
X. Script de criação de utilizadores	105
XI. Povoamento da BD	106
XII. Tamanho ocupado por cada relação na BD	110
XIII. Tradução das Interrogações	111
XIV. Procedimentos, Funções e Gatilhos	114

Índice de Tabelas

Tabela 1 - Requisitos de Descrição identificados pela equipa de desenvolvimento	14
Tabela 2 - Requisitos de Manipulação identificados pela equipa de desenvolvimento	16
Tabela 3 - Requisitos de Controlo identificados pela equipa de desenvolvimento.	17
Tabela 4 - Entidades identificadas pela equipa de desenvolvimento	21
Tabela 5 - Relacionamentos identificados pela equipa de desenvolvimento	23
Tabela 6 - Atributos identificados pela equipa de desenvolvimento	25
Tabela 7 - Dicionário de dados de “utilizador”	29
Tabela 8 - Dicionário de dados de “viagem”	30
Tabela 9 - Dicionário de dados de “publicacao”	30
Tabela 10 - Dicionário de dados de “vale_de_desconto”	31
Tabela 11 - Dicionário de dados de “tipo_do_vale”	31
Tabela 12 - Dicionário de dados de “tipo_viagem”	32
Tabela 13 - Dicionário de dados de “pais”	32
Tabela 14 - Dicionário de dados de “cidade”	33
Tabela 15 - Dicionário de dados de “comentario”	33
Tabela 16 - Dicionário de dados de “ficheiro_audio_visual”	34
Tabela 17 - Tabela de Relacionamento “segue”	35
Tabela 18 - Tabela de Relacionamento “viagem_cidade”	35
Tabela 19 - Tipos de dados e o espaço consumido pelo InnoDB	63
Tabela 20 - Especificação dos domínios e ocupação dos atributos da entidade “utilizador”.	63
Tabela 21 - Especificação dos domínios e ocupação dos atributos da entidade “viagem”	64
Tabela 22 - Especificação dos domínios e ocupação dos atributos da entidade “viagem”	64
Tabela 23 - Especificação dos domínios e ocupação dos atributos da entidade “publicacao”	64
Tabela 24 - Especificação dos domínios e ocupação dos atributos da entidade “vale_de_desconto”	64
Tabela 25 - Especificação dos domínios e ocupação dos atributos da entidade “tipo_vale”	64
Tabela 26 - Especificação dos domínios e ocupação dos atributos da entidade “comentario”	65
Tabela 27 - Especificação dos domínios e ocupação dos atributos da entidade “ficheiros_audio_visual”	65
Tabela 28 - Especificação dos domínios e ocupação dos atributos da entidade “cidade”	65
Tabela 29 - Especificação dos domínios e ocupação dos atributos da entidade “pais”	65
Tabela 30 - Especificação dos domínios e ocupação dos atributos da relação “segue”	65
Tabela 31 - Especificação dos domínios e ocupação dos atributos da relação “viagem_cidade”	65
Tabela 32 - Tamanho ocupado por cada relação na BD NOMÁLIA (usando o tamanho teórico dos registos)	66
Tabela 33 - Evolução do tamanho da BD NOMÁLIA (tamanho teórico dos registos)	66

Índice de Figuras

Figura 1 - Tabela de Gantt	10
Figura 2 - Ata de Reunião	11
Figura 2 - Ata da reunião com a CEO Filipa Martins	11
Figura 3 - Diagrama ER produzido	26
Figura 4 - Modelo lógico produzido	28
Figura 5 - Árvore de interrogação do requisito 13 aplicada aos dados de teste, para "id_utilizador" de "seguidor" igual a 5, para perfis que sigo	40
Figura 6 - Árvore de interrogação do requisito 13 aplicada aos dados de teste, para "id_utilizador" de "seguidor" igual a 5, para perfis que me seguem	40
Figura 7 - Árvore de interrogação do requisito 14 aplicada aos dados de teste	41
Figura 8 - Árvore de interrogação do requisito 15 aplicada aos dados de teste, para "id_utilizador" igual a 5	42
Figura 9 - Árvore de interrogação do requisito 16 aplicada aos dados de teste	42
Figura 10 - Árvore de interrogação do requisito 17 aplicada aos dados de teste, com "id_utilizador" igual a 5	43
Figura 11 - Árvore de interrogação do requisito 18 aplicada aos dados de teste, com "id_utilizador" igual a 5	43
Figura 12 - Árvore de interrogação do requisito 19 aplicada aos dados de teste	44
Figura 13 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte das viagens	45
Figura 14 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte dos ficheiros audiovisuais	45
Figura 15 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte das publicações	46
Figura 16 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte dos comentários	46
Figura 17 - Grafo do diagrama de dependências de criação da Base de Dados	48

1. Definição do Sistema

1.1. Contexto de aplicação

Ao longo do tempo, o registo de memórias e experiências pessoais foi um processo limitado e muito dependente de suportes físicos, como: fotografias, postais e diários. No entanto, estes meios, apesar do seu valor sentimental, são suscetíveis ao desgaste provocado pelo passar dos anos, o que dificulta a sua preservação a longo prazo.

Com o avanço das tecnologias, tornou-se possível armazenar, organizar e partilhar experiências de forma estruturada, segura e acessível, superando as limitações impostas pelos meios de registo não digitais, o que facilita a transmissão de conhecimentos e vivências às gerações futuras.

As Bases de Dados Relacionais surgem como uma solução para gerir grandes volumes de informação, permitindo a ligação entre diferentes tipos de dados (fotografias, contactos, vídeos, entre outros), o que facilita a análise e o cruzamento de informação de forma organizada e estruturada.

O presente trabalho insere-se neste contexto, tendo como objetivo o desenvolvimento de um Sistema de Base de Dados voltado para a gestão e preservação de memórias de viagens, de forma moderna, funcional e interativa, garantindo que memórias valiosas não se percam com o tempo.

Apresentação do Caso de Estudo

Durante toda a sua infância, Filipa cresceu a ouvir as fascinantes histórias da sua avó paterna, a Dona Amália, uma mulher aventureira que percorreu o mundo numa época em que viajar era um privilégio e comunicar à distância era um verdadeiro desafio. As fotografias, postais, cartas e lembranças que a avó colecionava tornaram-se o testemunho das suas aventuras (memórias guardadas em caixas de sapatos e álbuns já desgastados pelo tempo).

Com o passar dos anos, estas recordações foram-se perdendo e degradando, deixando lacunas nas memórias e nas histórias que antes encantavam os serões familiares. Movida pela vontade de preservar estas experiências e garantir que outras pessoas pudessem também eternizar as suas viagens, a Doutora Filipa fundou a empresa NOMÁLIA, uma plataforma digital totalmente em português, dedicada à partilha e preservação de memórias de viagens.

NOMÁLIA nasceu com o propósito de criar um espaço onde os utilizadores possam armazenar fotografias, vídeos, contactos e descrições de locais visitados, mantendo vivas as suas recordações e fomentando a interação entre turistas de diferentes idades e culturas.

De forma a garantir a estabilidade, segurança e eficiência deste sistema, a Doutora Filipa procurou a ajuda de um grupo de estudantes de Engenharia Informática da Universidade do Minho, confiando-lhes a missão de desenvolver um Sistema de Base de Dados que sirva de alicerce ao seu projeto e que permita uma gestão eficaz das informações relativas a viagens, utilizadores e conteúdos multimédia.

1.2. Motivação e Objetivos do Trabalho

O crescimento da empresa NOMÁLIA trouxe consigo a necessidade de uma estrutura para organizar e gerir o volume crescente de dados gerados pelos utilizadores. A abordagem inicial, que se baseava em ficheiros e registos dispersos, mostrou-se rapidamente insuficiente e ineficaz face à dimensão do projeto e à diversidade de informação a armazenar.

Além disso, a CEO Filipa reconheceu que, sem a nossa ajuda, a plataforma enfrentaria desafios significativos na organização, acessibilidade e segurança das memórias partilhadas pelos utilizadores.

Com base nessa necessidade, o projeto de desenvolvimento do Sistema de Base de Dados tem como principais objetivos:

- Estruturar a informação relacionada com utilizadores, viagens, contactos e conteúdos multimédia;
- Facilitar o armazenamento e recuperação de memórias, garantindo que os dados se mantêm organizados, seguros e facilmente acessíveis;
- Promover a interação entre utilizadores, permitindo a partilha de experiências e a criação de laços entre viajantes de diferentes partes do mundo;
- Apoiar a gestão da empresa NOMÁLIA, fornecendo uma ferramenta fiável para o controlo e análise da atividade na plataforma;
- Assegurar e permitir o crescimento contínuo da aplicação sem comprometer o seu desempenho.

A implementação deste sistema permitirá preservar, de forma digital e duradoura, as histórias dos seus utilizadores, cumprindo o sonho da doutora Filipa de dar uma nova vida às memórias da sua avó Amália e de todos aqueles que desejem eternizar as suas viagens.

1.3. Análise da Viabilidade do processo

A empresa NOMÁLIA, atualmente em fase de crescimento, enfrenta limitações tecnológicas derivadas da utilização de ferramentas convencionais e pouco eficientes (como folhas de cálculo e armazenamento local de ficheiros). Com a criação de um Sistema de Base de Dados, a empresa prevê ganhos significativos em eficiência e segurança de dados.

Após a análise, concluiu-se que o projeto é uma técnica economicamente viável, apresentando os seguintes benefícios estimados:

- Aumento de 40% na produtividade dos colaboradores de NOMÁLIA, graças à automatização da gestão de dados e à redução de tarefas manuais;
- Redução de 15% dos erros associados à perda ou duplicação de informação, assegurando maior consistência nos registos;
- Melhoria de 35% na velocidade de acesso e consulta de registos, permitindo respostas mais rápidas e uma experiência de utilizador otimizada;
- Reforço da segurança da informação, uma vez que o sistema será controlado internamente e protegido por mecanismos de autenticação e *backups* automáticos.

Deste modo, a implementação do Sistema de Base de Dados de NOMÁLIA é considerado um investimento estratégico e sustentável, com potencial para suportar o crescimento futuro da empresa e consolidar a sua posição no mercado digital de experiências de viagens.

1.4. Recursos e Equipa de Trabalho

Os recursos necessários para o desenvolvimento do projeto dividem-se em duas categorias principais: recursos humanos e recursos materiais. Estes recursos garantirão que o desenvolvimento decorra de forma alinhada com os objetivos de NOMÁLIA e da empresária Filipa Martins.

Recursos Humanos

Pessoal Interno (NOMÁLIA):

- **Doutora Filipa Martins** – Fundadora e diretora de NOMÁLIA. Responsável pela supervisão do projeto, definição de requisitos e validação dos resultados;
- **Beatriz Silva** – Coordenadora de Comunicação e responsável pela ligação entre os utilizadores da plataforma e a equipa de desenvolvimento;
- **Ricardo Matos** – Gestor técnico de NOMÁLIA, encarregado de garantir a integração do Sistema de Base de Dados com o site e restantes sistemas da empresa.

Pessoal Externo (Equipa de Desenvolvimento):

- **Equipa de Engenharia Informática** – Grupo de estudantes contratadas pela empreendedora Filipa para desenvolver o Sistema de Base de Dados, responsáveis pela análise de requisitos, modelação conceptual, lógica e física. A equipa possui competências em programação SQL, modelação de dados e gestão de projetos de *software*.

Recursos Materiais e Tecnológicos

- *Software* necessário: Ferramentas de modelação (MySQL *Workbench*); Sistema de Gestão de Base de Dados Relacional MySQL; Ferramentas de colaboração e gestão de projeto (*Notion*).

1.5. Plano de Execução do Projeto

O grupo de alunas reuniu-se com Filipa, fundadora e diretora de NOMÁLIA, para definir o plano de desenvolvimento do Sistema de Base de Dados da empresa.

Durante esta reunião, foram estabelecidos os objetivos de cada fase do projeto, o período total de execução e a distribuição das tarefas pelos diferentes membros da equipa, de acordo com as competências individuais de cada elemento e as necessidades técnicas do sistema.

Como resultado desta planificação, foi elaborado o Diagrama de *Gantt* (apresentado no **Anexo I**), onde se encontram representadas as etapas principais do projeto.

Tabela de Gantt



Figura 1 - Tabela de Gantt

2. Levantamento e Análise de Requisitos

2.1. Método de Levantamento e de Análise de Requisitos Adotado

Para o levantamento de requisitos do Sistema de Base de Dados de NOMÁLIA, a equipa de desenvolvimento recorreu a uma combinação de reuniões presenciais e remotas, troca de correspondência digital com colaboradores da empresa e análise de necessidades funcionais fornecidas pela fundadora, Doutora Filipa Martins. Alguns métodos tradicionais, como a observação direta do funcionamento da empresa, seriam inviáveis, dado que a plataforma ainda se encontra em fase de planeamento.

Reuniões com a Empreendedora Filipa e colaboradores NOMÁLIA



NOMÁLIA
VIAJA, REGISTA, VIVE

ATA DA REUNIÃO

DATA: 20 DE OUTUBRO DE 2025
HORA: 15H

1. Objetivo da Reunião	
Discutir os objetivos principais da plataforma e as funções essenciais da Base de Dados. Explicar a sua visão para NOMÁLIA.	
2. Participantes presentes	
Nome	Posição
Doutora Filipa Martins	Fundadora e Diretora de Nomália
Beatriz Silva	Coordenadora de Comunicação
Ricardo Matos	Gestor técnico de NOMÁLIA

Figura 2 - Ata da reunião com a CEO Filipa Martins

Nestas reuniões foram discutidos tópicos como:

- Que tipos de informação cada utilizador deveria ter no seu perfil e como as proteger;
- Que dados seriam registados em cada viagem ou publicação;
- Que tipo de sistema de recompensas seria adequado para motivar a participação dos utilizadores.

2.2. Organização dos Requisitos Levantados

Durante as reuniões, a Doutora Filipa detalhou a missão de NOMÁLIA, os seus objetivos e a experiência pretendida para os utilizadores da plataforma. A partir desta partilha, a equipa conseguiu identificar e formalizar requisitos essenciais ao registo, gestão e interação de utilizadores, viagens e publicações. Cada requisito foi discutido e validado com os representantes da empresa, assegurando que todos respondiam às necessidades do sistema.

Após a recolha da informação, a equipa procedeu à organização sistemática dos requisitos, categorizando-os segundo três dimensões fundamentais da modelação de sistemas:

- **Requisitos de Descrição:** relacionados com a estrutura e os atributos das entidades principais da Base de Dados (ex.: utilizadores, viagens, publicações);
- **Requisitos de Manipulação:** que definem operações que o sistema deve permitir (ex.: pesquisar viagens, seguir utilizadores, comentar publicações);
- **Requisitos de Controlo:** referentes à gestão de acessos, níveis de permissão e segurança da informação (ex.: quem pode editar perfis ou eliminar publicações).

Esta classificação permitiu estruturar o domínio de forma clara, facilitar a derivação das entidades e dos relacionamentos no modelo conceptual, e garantir que todos os aspetos relevantes para a NOMÁLIA estavam devidamente representados. Cada requisito foi acompanhado de informações contextuais essenciais, como data da recolha, origem do pedido, área a que diz respeito e analista que o registou, assegurando transparência em todo o processo.

As tabelas seguintes sintetizam estes requisitos, tal como foram confirmados com a Doutora Filipa e a sua equipa. A tabela completa de requisitos pode ser consultada no **Anexo II**.

2.2.1. Requisitos de Descrição

Para que não houvesse qualquer divergência entre aquilo que os responsáveis da NOMÁLIA necessitavam, foram realizadas várias sessões de análise e discussão. O objetivo destas reuniões era assegurar que todos os requisitos essenciais eram identificados com clareza, permitindo desenvolver um sistema totalmente alinhado com a visão da fundadora.

Numa das primeiras reuniões com a Doutora Filipa e com os seus colaboradores, os mesmos destacaram a importância de cada utilizador possuir um perfil completo, uma vez que NOMÁLIA é uma plataforma fortemente baseada na interação social e na partilha de experiências de viagem. Segundo a CEO, “O utilizador tem de sentir que tem um espaço que o representa”. Ficou assim estabelecido que cada perfil deverá conter as informações pessoais mais relevantes que permitam uma maior facilidade de interação entre os vários utilizadores. Com base nestas preferências decidiu-se que cada perfil deveria ter o respetivo identificador, nome de utilizador, *e-mail*, biografia e o número de pontos a ele associado.

Verificou-se que, numa outra sessão, os colaboradores apresentaram a necessidade de definir claramente a estrutura de uma viagem. De acordo com as orientações da equipa, uma viagem deveria incluir o seu identificador único, o tipo, o destino e as respetivas datas de partida e chegada. Isto porque, nas palavras da Doutora Filipa, “A plataforma só funcionará bem se as viagens estiverem muito bem organizadas. Se um utilizador visitar várias cidades de um mesmo país, então ele poderá colocar todas as fotos dessas cidades numa publicação”.

Seguidamente, abordou-se a questão das publicações. Beatriz Silva, a coordenadora de Comunicação da empresa, referiu que cada publicação deveria ter uma breve descrição, a respetiva data e o seu identificador único.

Numa outra reunião, entre a equipa de NOMÁLIA e a equipa de alunas, onde o assunto principal foi a componente interativa e social da plataforma, chegou-se à conclusão, por opiniões de Ricardo Matos, que um comentário deveria ser composto pelo respetivo texto onde estaria expressa a opinião do utilizador e o identificador único do mesmo. Aí, a equipa interrogou os integrantes de NOMÁLIA, acerca da publicação. Queriam saber se numa publicação seria possível incluir mais do que uma viagem, aos quais Ricardo Matos respondeu que não, pois assim teriam uma organização máxima na plataforma, ficando decidido, então, que uma publicação estaria associada a uma única viagem.

Para incentivar as várias pessoas a participarem ativamente na plataforma, a Doutora Filipa teve a ideia de estabelecer um sistema simples de recompensas. Foi então realizada uma videoconferência, para ser comunicada o mais rápido possível. Nesta, a equipa esclareceu que não iria ser necessário uma estrutura muito complexa, instituindo um único tipo de benefício: os vales de desconto. Pelas palavras de um dos membros da equipa: “Assim teríamos uma recompensa única e universal que seria, então, alcançada de acordo com a atividade do utilizador na plataforma, que, posteriormente, seria transformada em pontos. No futuro, estes pontos poderão ser trocados por vales”. Para manter uma organização na secção dos vales, Ricardo Matos, deu a ideia de separar os vales pelos seus tipos, ou seja, se fosse de desconto em companhias aéreas seria um, em hotéis outro e em atividades lúdicas outro. Assim, a plataforma estaria bem estruturada em todas as suas secções.

Nessa reunião surgiu a proposta de anexar um ou mais ficheiros audiovisuais a cada publicação. Beatriz Silva afirmou: “Para que os utilizadores possam melhorar as suas publicações visualmente, ou seja, tornarem-nas mais vistosas e agradáveis, acho por bem poderem incluir fotos e vídeos, nas várias publicações”. Foi, então, colocada em prática, permitindo a correta associação e reprodução destes conteúdos.

A doutora Filipa Martins lembrou-se da possibilidade de referir explicitamente a cidade de cada viagem para disponibilizar uma melhor organização, facilitar a pesquisa e a avaliação das publicações. Todos concordaram com a ideia de Filipa, tendo sido posta em prática. Assim, definiu-se que cada viagem publicada estaria associada a uma ou mais cidades, identificadas de forma única no sistema, e caracterizada pelos seus identificadores e respetivos nomes. Pela

mesma lógica de Filipa, Ricardo sugeriu que, a cada cidade deve ser associado o país correspondente, evitando, assim, problemas em casos em que, por exemplo, existam cidades com o mesmo nome em países diferentes. Como resultado, conclui-se que a ideia de Ricardo era boa e que permitiria que a plataforma estivesse bem organizada, facilitando a pesquisa.

Com a intenção de tornar a plataforma bem estruturada, Beatriz lembrou-se de que as viagens também poderiam ser classificadas de acordo com o seu tipo e apresentou as categorias que idealizou, numa reunião posterior. Aqui, todos concordaram, pois assim o sistema fornecerá seleções de viagens pelo seu tipo e recomendações personalizadas, isto é, se, por exemplo, uma viagem estiver identificada por lazer, irão aparecer publicações em que o utilizador estaria a explorar mais a respetiva cidade, enquanto se for gastronómica irão aparecer fotos e vídeos das diferentes refeições feitas.

Abaixo, encontra-se uma tabela onde estão organizadas todas estas ideias, transformadas em requisitos de descrição:

Requisitos	Data/Hora	Descrição	Área	Fonte	Analista
1	18/10 15h30	Cada perfil de utilizador deve incluir o identificador único, nome do utilizador, <i>email</i> , biografia e o número de pontos conquistados.	Utilizador	Filipa Martins	Sara Marques
2	19/10 10h30	A viagem é caracterizada pelo seu tipo, identificador único, destino, data de partida e chegada.	Viagem	Beatriz Silva	Sara Marques
3	19/10 10h40	Uma viagem pode incluir várias cidades pertencentes ao mesmo país.	Viagem	Filipa Martins	Marta Araújo
4	19/10 11h00	A publicação é caracterizada por um identificador único, a respetiva descrição e data de publicação.	Publicação	Beatriz Silva	Teresa Teixeira
5	21/10 19h30	Cada publicação pode conter comentários, e cada comentário deve incluir o respetivo texto e identificador único.	Comentário	Ricardo Matos	Teresa Teixeira
6	21/10 19h50	Cada publicação é associada a apenas uma viagem.	Publicação	Ricardo Matos	Ana Patrícia Machado
7	23/10 15h00	Existe a possibilidade de trocar os pontos acumulados por vales de desconto.	Vale de desconto	Filipa Martins	Teresa Teixeira
8	23/10 19h00	Existem vários tipos de vales possíveis para resgatar: vale de desconto em companhias aéreas, em hotéis e em atividades lúdicas.	Tipo Vale	Ricardo Matos	Marta Araújo
9	23/10 19h00	Existe a possibilidade de anexar a cada publicação, ficheiros áudio-visuais para enriquecer o seu conteúdo.	Ficheiro Áudio-visual	Beatriz Silva	Marta Araújo
10	23/10 19h20	É necessário identificar a cidade em cada viagem publicada, sendo esta caracterizada pelo seu identificador único e nome.	Cidade	Ricardo Matos	Sara Marques
11	23/10 19h40	É necessário associar à cidade que foi conectada a uma viagem, o país correspondente, que é caracterizado pelo seu identificador único e nome.	País	Filipa Martins	Sara Marques
12	25/10 16h00	Existem vários tipos de viagem possíveis: viagem profissional, de lazer, cultural e gastronómica.	Tipo Viagem	Beatriz Silva	Ana Patrícia Machado

Tabela 1 - Requisitos de Descrição identificados pela equipa de desenvolvimento

2.2.2. Requisitos de Manipulação

Beatriz Silva recordou-se da possibilidade de um utilizador poder ou parar de seguir outros perfis, interagir com as várias publicações da plataforma e visualizar a lista dos seus seguidores. Foi então organizada uma rápida videoconferência onde ela nos explicou que isto permitiria criar uma rede de ligações dentro da plataforma, o que faria com que os diferentes viajantes mantivessem contacto com quem admiram ou com quem partilham interesses semelhantes.

Nessa mesma videoconferência falou-se também sobre as várias funcionalidades disponíveis para cada utilizador. “Como é óbvio, um utilizador tem de poder criar e visualizar publicações associadas às suas múltiplas viagens, assim como as pode editar”, afirmou a Doutora Filipa, cuja afirmação foi de acordo com todos os intervenientes da reunião virtual. Também se achou por bem colocar a ideia de Ricardo Matos em prática, que seria que o utilizador pode registar as suas novas viagens na plataforma.

Falou-se também da importância da existência de uma secção de comentários. Ricardo salientou que queria que outros utilizadores pudessem interagir com as publicações, dizendo “Quero um local onde as pessoas possam partilhar as suas opiniões, dúvidas e as diferentes experiências”. A equipa concordou com o integrante de NOMÁLIA, uma vez que permitiria tornar a plataforma mais dinâmica e abrangente. Aqui, Beatriz Silva, sugeriu que fosse possível a um utilizador poder criar e editar os seus próprios comentários, alegando que: “Várias vezes acontece de nos enganarmos a escrever, por exemplo, então, acho por bem inserir a funcionalidade de o utilizador poder editar os seus próprios comentários”. Todos os intervenientes concordaram, tornando-se assim um requisito fundamental.

Foi referida a importância dos vários utilizadores poderem editar os seus dados. De acordo com Filipa Martins, “Há várias pessoas que gostam de ir modificando o que escrevem sobre si, acho por bem que os diferentes utilizadores possam ser capazes de alterar a respetiva biografia”. Todos os integrantes da reunião concordaram, pelo que se tornou um requisito essencial para a diversidade da plataforma.

Na videoconferência referida na secção anterior, onde se falou sobre os vales de desconto, Beatriz Silva reforçou a importância de permitir que os utilizadores troquem os pontos acumulados por vales disponíveis na plataforma, garantindo a existência de um sistema de recompensa simples e motivador.

Com base nos acontecimentos do mundo atual, Ricardo Matos lembrou-se que seria uma boa ideia haver algo que controlasse os conteúdos colocados na plataforma, as viagens associadas às mesmas e os comentários que serão feitos. Assim, a plataforma seria, garantidamente, segura, evitando qualquer tipo de problemas entre utilizadores.

Abaixo, encontra-se uma tabela onde estão organizadas todas estas ideias transformadas em requisitos de manipulação:

Requisitos	Data/Hora	Descrição	Área	Fonte	Analista
13	20/10 18h10	O utilizador pode seguir outros perfis, interagir com as respetivas publicações, deixar de seguir outros utilizadores e visualizar a lista dos seus seguidores.	Utilizador	Beatriz Silva	Ana Patrícia Machado
14	20/10 18h30	O utilizador deve poder criar, editar e visualizar publicações associadas às suas viagens.	Publicação	Filipa Martins	Marta Araújo
15	20/10 19h10	O utilizador deve poder criar, visualizar e editar as suas viagens na plataforma.	Viagem	Ricardo Matos	Ana Patrícia Machado
16	21/10 19h00	Cada publicação tem uma secção de comentários, onde os utilizadores podem partilhar opiniões sobre a viagem em questão.	Comentário	Ricardo Matos	Marta Araújo
17	21/10 19h20	O utilizador deve poder criar, editar e visualizar os seus comentários.	Comentário	Beatriz Silva	Ana Patrícia Machado
18	22/10 10h40	O utilizador deve poder inserir, visualizar ou editar dados do seu perfil.	Utilizador	Filipa Martins	Marta Araújo
19	23/10 15h00	A plataforma deve permitir que os pontos sejam trocados por vales de desconto.	Vale de desconto	Beatriz Silva	Ana Patrícia Machado
20	23/10 15h30	A plataforma deve permitir ao administrador visualizar e remover viagens, as publicações a elas associadas e os comentários dos vários utilizadores que forem escritos nas mesmas.	Administração	Ricardo Matos	Teresa Teixeira

Tabela 2 - Requisitos de Manipulação identificados pela equipa de desenvolvimento

2.2.3. Requisitos de Controlo

“Para que cada utilizador obtenha os pontos que vão ser trocados pelos respetivos vales, a plataforma terá de os atribuir de alguma forma. Acho que o melhor seria de acordo com a sua atividade na plataforma.”, afirmou Filipa Martins na videoconferência onde se falou acerca do sistema de premiação. Toda a equipa concordou, pois, assim, estaria assegurado o registo transparente e consistente da respetiva premiação.

Para a segurança e a privacidade dos dados pessoais, Ricardo Matos sugeriu que cada utilizador deveria possuir credenciais de acesso únicas, afirmando “Os diferentes utilizadores devem sentir que as suas informações pessoais estão asseguradas para que apenas o utilizador possa aceder, editar ou eliminar os respetivos dados”. Todos concordaram com a sugestão, porque queriam criar uma plataforma resguardada para toda a gente onde os respetivos dados tivessem bem salvaguardados.

Para elevar ainda mais a segurança dos dados pessoais dos utilizadores, Ricardo Matos sugeriu que a palavra-passe deveria ter um número mínimo de caracteres. Com isto, Filipa Martins propôs que a palavra-passe fosse guardada de forma ilegível, garantindo, assim, a proteção dos dados sensíveis. Estas opiniões foram tomadas como relevantes pois, desta forma, a palavra-passe tornar-se-ia mais difícil de ser descoberta.

Abaixo, encontra-se uma tabela onde estão organizadas todas estas ideias transformadas em requisitos de controlo:

Requisitos	Data/Hora	Descrição	Area	Fonte	Analista
21	23/10 15h00	A plataforma deve atribuir pontos ao utilizador de acordo com a sua atividade no sistema.	Vale de desconto	Filipa Martins	Marta Araújo
22	24/10 11h00	O acesso ao perfil do utilizador deve ser protegido através de um sistema de autenticação com palavra-passe.	Utilizador	Ricardo Matos	Sara Marques
23	24/10 11h20	O e-mail e nome de utilizador devem ser únicos na plataforma.	Utilizador	Beatriz Silva	Ana Patrícia Machado
24	24/10 11h40	A palavra-passe deve cumprir requisitos mínimos de segurança (ex.: número mínimo de 15 caracteres).	Segurança	Ricardo Matos	Marta Araújo
25	24/10 12h00	A palavra-passe não deve ser armazenada em formato legível.	Segurança	Filipa Martins	Sara Marques
26	24/10 12h20	O administrador deve ter permissões específicas que lhe permitam gerir conteúdos e contas, sem acesso aos dados privados dos utilizadores.	Administração	Ricardo Matos	Teresa Teixeira

Tabela 3 - Requisitos de Controlo identificados pela equipa de desenvolvimento.

2.3. Análise e Validação Geral dos Requisitos

Concluída a elaboração do documento de requisitos, procedeu-se a uma análise rigorosa por parte do grupo de estudantes. Esta revisão final teve como finalidade assegurar a precisão e coerência das informações registadas, bem como identificar eventuais faltas de informação ou ambiguidades decorrentes das fases anteriores do levantamento. A avaliação interna não revelou inconformidades significativas, permitindo avançar para a etapa seguinte do processo de validação.

Posteriormente, foi convocada uma reunião formal com os intervenientes associados ao projeto, com o objetivo de proceder à validação conjunta de todos os requisitos definidos. Durante esta sessão, cada elemento descrito no documento foi analisado individualmente, possibilitando o esclarecimento de dúvidas e a confirmação da importância de cada ponto relativamente às necessidades identificadas.

A reunião constituiu um momento essencial de resolução de possíveis discrepâncias. Algumas observações consideradas relevantes foram incorporadas no documento atualizado, garantindo que este reflète de forma realista as expectativas e orientações definidas pelos participantes.

No final do processo, todos os intervenientes concordaram quanto à última versão do documento, validando formalmente o conjunto de requisitos estabelecidos.

3. Modelação Conceptual

3.1. Apresentação da Abordagem de Modelação Realizada

Após a recolha e análise dos requisitos definidos pela Doutora Filipa Martins, a equipa iniciou o processo de planeamento da estrutura de Base de Dados. O principal objetivo foi construir um sistema capaz de armazenar e organizar informações sobre utilizadores, viagens, publicações, países, cidades e sobre os vales.

Para tal, foi elaborado um modelo conceptual capaz de representar todas as operações essenciais da plataforma, nomeadamente a gestão de utilizadores, viagens, publicações e os vales de desconto. O processo iniciou-se com a identificação das entidades principais, seguida pela determinação dos relacionamentos e definição dos atributos de cada entidade.

3.2. Identificação e Caracterização das Entidades

A identificação das entidades foi realizada com base na leitura dos requisitos levantados junto da Doutora Filipa. Cada entidade foi definida e devidamente justificada:

“utilizador”: Entidade que representa cada pessoa registada na plataforma NOMÁLIA, sendo a base para toda a interação dentro do sistema. O “utilizador” contém dados pessoais, como: nome de utilizador, pontos, palavra-passe, *email*, biografia e o seu identificador próprio (*id_utilizador*). Termos como “perfil de utilizador”, “conta” ou “página pessoal” foram utilizados nos requisitos com o mesmo significado.

“viagem”: Entidade essencial ao propósito de NOMÁLIA que descreve um evento específico associado a um utilizador, caracterizado pelo seu identificador único, datas de partida e chegada. O termo “experiência” foi utilizado nos requisitos como sinónimo de “viagem”.

“publicacao”: Entidade que representa o conteúdo compartilhado pelos utilizadores acerca das suas viagens. Cada “publicacao” inclui o seu identificador próprio, uma descrição e data da experiência, permitindo a outros utilizadores comentar e interagir. Nos requisitos, surgem expressões como “conteúdo” ou “anúncio”, utilizadas com o mesmo significado.

“comentario”: Entidade que representa as interações dos utilizadores com as publicações existentes na plataforma, permitindo a partilha de opiniões, experiências ou *feedback* relativos a uma viagem publicada. Cada “comentario” é caracterizado pelo respetivo texto e um identificador único. Esta entidade garante que um utilizador pode escrever vários comentários e que cada publicação pode reunir múltiplas contribuições, assegurando que todas estão associadas a um utilizador e a uma publicação.

“tipo_viagem”: Entidade que representa a categoria associada a uma viagem realizada por um utilizador. Cada tipo de viagem possui um identificador único e uma designação que permite distinguir diferentes naturezas de experiências, como: viagens profissionais, de lazer, culturais ou gastronómicas. A criação desta entidade permite normalizar a informação relativa ao tipo de experiência, evitando redundâncias e garantindo consistência na classificação das viagens.

“vale_de_desconto”: A entidade representa as recompensas disponíveis na plataforma, que os utilizadores podem obter através da acumulação de pontos gerados pela publicação de viagens. Cada vale possui uma designação específica, que identifica a sua finalidade (por exemplo, “10% de desconto na companhia aérea X”), e um valor em pontos necessário para a sua obtenção, incentivando os utilizadores a participar ativamente na plataforma.

“tipo_vale”: Entidade que representa a tipologia dos vales de desconto existentes na plataforma. Cada tipo de vale possui um identificador único e uma designação, permitindo distinguir vales de desconto aplicáveis a companhias aéreas, hotéis e atividades lúdicas. Esta entidade foi criada para estruturar e organizar os diferentes tipos de recompensa definidos pela própria plataforma, facilitando a gestão, manutenção e eventual expansão do sistema.

No âmbito do processo de modelação conceptual, foram tomadas decisões com o objetivo de garantir a consistência, a integridade e a normalização dos dados. Em particular, optou-se por modelar “pais”, “cidade” e “ficheiro_audio_visual” como entidades autónomas, em vez de as representar como atributos de outras entidades, atendendo à natureza e utilização da informação que representam.

As entidades “pais” e “cidade” correspondem à informação introduzida pelos utilizadores no contexto das viagens. A atribuição de um identificador único a cada país e cidade assegura que diferentes viagens possam referenciar o mesmo local de forma consistente, promovendo a integridade dos dados e facilitando operações futuras de pesquisa, análise e manutenção da base de dados.

Por sua vez, a entidade “ficheiro_audio_visual” foi criada pelo facto de uma publicação poder conter um ou vários ficheiros multimédia, caracterizando um atributo multivalorado. Adicionalmente, a criação de uma entidade própria permite distinguir ficheiros que possam possuir o mesmo nome (por exemplo, “foto1.png”), garantindo que cada ficheiro seja identificado de forma inequívoca através de um identificador único. Esta abordagem assegura a correta gestão dos conteúdos multimédia e evita ambiguidades na identificação dos ficheiros.

“pais”: Entidade que representa um país identificado na plataforma. Cada país possui um identificador único e um nome, sendo utilizado para contextualizar geograficamente as cidades associadas às viagens.

“cidade”: Entidade que representa uma cidade associada às viagens realizadas pelos utilizadores. Cada cidade é identificada de forma única e possui um nome, estando associada a um país específico.

“ficheiro_audio_visual”: Entidade que representa os ficheiros multimédia associados às publicações efetuadas pelos utilizadores. Cada ficheiro áudio-visual possui um identificador único e corresponde a um conteúdo áudio ou visual anexado a uma publicação.

Estas entidades podem ser vistas na tabela abaixo:

Designação	Descrição	Sinónimos	Ocorrência
utilizador	Identificação de uma pessoa no <i>site</i> de publicação de viagens (NOMÁLIA).	Perfil de utilizador, conta, página pessoal	N

	Contém informações pessoais, dados de interação social e o número de pontos associados.		
viagem	Representa a experiência de um utilizador, com informações sobre o datas e o tipo de viagem e local de estadia.	Experiência	N
tipo_viagem	Classificação associada a uma viagem, permitindo distinguir diferentes categorias de experiências.	Categoria de viagem	N
pais	Representa um país identificado na plataforma, utilizado para contextualizar geograficamente as cidades associadas às viagens.	País de destino	N
cidade	Representa uma cidade associada às viagens realizadas pelos utilizadores, pertencente a um país específico.	Local	N
publicacao	Conteúdo partilhado no <i>site</i> NOMÁLIA acerca das viagens de cada utilizador.	Conteúdo	N
comentario	Representa um comentário efetuado por um utilizador numa publicação.	<i>Feedback</i>	N
ficheiro_audio_visual	Representa um ficheiro multimédia associado a uma publicação.	Ficheiro multimédia	N
vale_de_desconto	Recompensa que os utilizadores podem obter na plataforma através da acumulação de pontos.	Prémio, benefício, recompensa	N
tipo_de_vale	Classificação dos vales de desconto disponíveis na plataforma.	Categoria de vale	N

Tabela 4 - Entidades identificadas pela equipa de desenvolvimento

3.3. Identificação e Caracterização dos Relacionamentos

Após a definição das entidades, foram identificados os principais relacionamentos entre elas. Cada relacionamento estabelece a interação das entidades, a sua cardinalidade e participação, bem como a sua respetiva descrição, tendo sido devidamente justificados:

“cria”: Relacionamento entre a entidade “utilizador” e a entidade “publicacao”, que representa a ação de publicar conteúdos sobre viagens. Um utilizador pode criar várias publicações, mas cada publicação pertence exclusivamente a um utilizador. A cardinalidade 1:N e a participação P:T refletem essa dependência direta, em que cada publicação tem de estar associada a um criador.

“contem”: Relacionamento entre as entidades “publicacao” e “comentario”, que representa os comentários efetuados nas publicações. Uma publicação pode conter vários comentários, enquanto cada comentário está associado a uma única publicação. A cardinalidade é 1:N, com participação P:T, pois uma publicação pode não ter comentários, mas cada comentário necessita obrigatoriamente de uma publicação associada.

“faz”: Relacionamento entre a entidade “utilizador” e a entidade “viagem”, que corresponde à função essencial do sistema, o registo de viagens realizadas. Cada utilizador pode ter várias viagens associadas, mas cada viagem pertence a um único utilizador. Mantém-se a mesma cardinalidade 1:N, com participação P:T, já que um utilizador pode não ter viagens registadas, mas cada viagem necessita obrigatoriamente de estar associada a um utilizador.

“decorre”: Relacionamento entre as entidades “viagem” e “cidade”, indicando o local onde a viagem ocorre. Cada viagem decorre numa ou mais cidades e uma cidade pode estar associada a várias viagens. A cardinalidade é N:M, com participação T:P, pois toda a viagem tem de ocorrer numa cidade, mas uma cidade pode não ter viagens associadas.

“segue”: Relacionamento entre a entidade “utilizador” com ela própria (relação recursiva). Traduz a possibilidade de um utilizador seguir o perfil de outro, criando a possibilidade de interações dentro do sistema. Trata-se de um relacionamento N:M com participação P:P, uma vez que qualquer utilizador pode seguir vários outros e, simultaneamente, ser seguido por diversos membros.

“troca”: Relacionamento entre a entidade “utilizador” e “vale_de_desconto”, que permite a conversão de pontos acumulados em vales de desconto. Um utilizador pode efetuar múltiplas trocas, e cada recompensa pode ser obtida por diferentes utilizadores, ao longo do tempo. Por esse motivo, este relacionamento apresenta cardinalidade N:M e participação P:P, refletindo a multiplicidade de interações entre utilizadores e vales.

“refere-se a”: Relacionamento entre a entidade “publicacao” e “viagem”, que demonstra que cada publicação é baseada numa viagem específica. Uma viagem pode dar origem a uma publicação e cada publicação apenas se refere a uma viagem. A cardinalidade 1:1 e participação T:P expressam o facto de uma publicação não poder existir sem uma viagem associada, enquanto uma viagem pode existir mesmo sem publicações.

“é do tipo”: Relacionamento entre a entidade “viagem” e a entidade “tipo_viagem”, que classifica as viagens segundo o seu tipo. Cada viagem pertence a um único tipo, enquanto um tipo de viagem pode estar associado a várias viagens. A cardinalidade é N:1, com participação T:P.

“situada”: Relacionamento entre as entidades “cidade” e “pais”, que representa a localização geográfica das cidades. Um país pode possuir várias cidades, mas cada cidade pertence apenas a um país. Este relacionamento apresenta cardinalidade N:1, com participação T:T, uma vez que ambas as entidades existem de forma independente.

“escreve”: Relacionamento entre as entidades “utilizador” e “comentario”, que representa a ação de um utilizador escrever comentários na plataforma. Um utilizador pode escrever vários comentários, enquanto cada comentário é escrito por um único utilizador. Este relacionamento apresenta cardinalidade 1:N, com participação P:T, uma vez que um utilizador pode nunca escrever comentários, mas cada comentário tem obrigatoriamente de estar associado a um utilizador.

“classifica”: Relacionamento entre as entidades “vale_de_desconto” e “tipo_vale”, que indica que cada vale pertence a um determinado tipo. Um tipo de vale pode originar vários vales de desconto, mas cada vale está associado a um único tipo. A cardinalidade é N:1, com participação T:T.

“possui”: Relacionamento entre as entidades “publicacao” e “ficheiro_audio_visual”, que representa a associação de conteúdos multimédia às publicações. Uma publicação pode possuir um ou vários ficheiros audiovisuais, enquanto cada ficheiro audiovisual está associado exclusivamente a uma única publicação. Este relacionamento apresenta cardinalidade 1:N, com participação P:T, uma vez que uma publicação pode existir sem ficheiros associados, mas cada ficheiro audiovisual necessita obrigatoriamente de estar associado a uma publicação.

Os **relacionamentos** podem ser vistos na tabela abaixo:

Entidade	Relacionamentos	Cardinalidade	Participação	Entidade	Descrição
utilizador	cria	1:N	P:T	publicacao	O utilizador publica conteúdos sobre as suas viagens.
utilizador	faz	1:N	P:T	viagem	O utilizador realiza várias viagens registadas no sistema.
utilizador	segue	N:M	P:P	utilizador	O utilizador acompanha perfis de outros utilizadores.
utilizador	troca	1:N	P:P	vale_de_desconto	O utilizador acumula pontos e troca-os por recompensas.
publicacao	refere-se a	1:1	T:P	viagem	A publicação está associada a uma viagem realizada.
utilizador	escreve	1:N	P:T	comentario	O utilizador partilha opiniões nas publicações de viagens.
vale_de_desconto	classifica	N:1	T:T	tipo_vale	O vale de desconto é classificado segundo um tipo de vale definido pela plataforma.
cidade	situada	N:1	T:P	pais	A cidade está situada num país identificado na plataforma.
viagem	decorre	N:M	T:P	cidade	A viagem decorre numa ou em várias cidades registadas no sistema.
viagem	é do tipo	N:1	T:P	tipo_viagem	A viagem é associada a um tipo de viagem previamente definido.
publicacao	possui	1:N	P:T	ficheiro_audio_visu al	A publicação possui ficheiros audiovisuais associados.
publicacao	contém	1:N	P:T	comentario	A publicação contém comentários efetuados pelos utilizadores.

Tabela 5 - Relacionamentos identificados pela equipa de desenvolvimento

3.4. Identificação e Caracterização dos Atributos das Entidades e dos Relacionamentos

A seguir apresentam-se os atributos principais de cada entidade, definidos a partir dos requisitos levantados:

A entidade **“utilizador”** revelou-se a mais complexa, uma vez que os atributos definidos incluem desde informações de identificação, como “nome_utilizador” e “id_utilizador”, até outros elementos, como: “palavra_passe”, “biografia”, “email” e “pontos”. A maior parte destes atributos apresenta não-nulidade, dada a sua importância no contexto da plataforma.

Para a entidade **“viagem”**, foram incluídos atributos que caracterizam a experiência do utilizador, nomeadamente o “id_viagem”, “data_partida” e “data_chegada”. Estes permitem descrever cada deslocação, servindo de base para a criação de publicações.

A entidade **“publicacao”** inclui os atributos necessários para representar o conteúdo compartilhado pelos utilizadores: a “descricao”, a “data_publicacao” e o “id_publicacao”, que refletem a interação social na plataforma.

A entidade **“ficheiro_audio_visual”** foi definida para representar os conteúdos multimédia associados às publicações. Os atributos identificados incluem o “id_ficheiro” e o “ficheiro_audiovisual”. Permitindo a identificação única de cada ficheiro e o caminho ou referência do ficheiro, que indica a localização do conteúdo audiovisual armazenado. Esta entidade garante a flexibilidade do sistema, permitindo que uma publicação possa conter um ou vários ficheiros multimédia, ou nenhum.

Na entidade **“vale_de_desconto”**, os atributos identificados foram: o “id_vale_desconto” e se este já foi ou não usado (atributo “usado”). Este está diretamente associado à interação do utilizador na plataforma.

A entidade **“pais”** foi definida para representar os países existentes no sistema, sendo caracterizada pelos atributos “id_pais” e “nome_pais”. Esta entidade permite organizar geograficamente as cidades, funcionando como base para a estruturação dos destinos das viagens.

A entidade **“cidade”** representa as cidades onde as viagens podem ocorrer e inclui os atributos “id_cidade” e “nome_cidade”. Cada cidade encontra-se associada a um único país, permitindo uma organização geográfica coerente dos destinos disponíveis na plataforma.

A entidade **“tipo_vale”** foi criada para classificar os diferentes tipos de vales de desconto existentes no sistema. Os atributos definidos incluem o “id_vale_desconto”, o “nome_tipo_vale”, a “descricao_tipo” e o “valor_pontos_tipo”, que determina o custo do vale no sistema de recompensas.

A entidade **“tipo_viagem”** caracteriza as viagens de acordo com a sua finalidade. Os atributos identificados incluem o “id_tipo_viagem” e o “nome_tipo”, permitindo a classificação das viagens realizadas pelos utilizadores.

Finalmente, a entidade **“comentario”** foi definida para representar as interações dos utilizadores com as publicações. Os atributos associados incluem o “id_comentario” e o “comentario”, permitindo registar as opiniões ou observações efetuadas pelos utilizadores.

Entidade	Atributos	Descrição	Domínio/ Tamanho	Nulo	MV	Exemplo
utilizador	nome_utilizador	Identificador único escolhido pelo utilizador para exibição no sistema.	VARCHAR[45]	N	N	"diogopereira10"
	pontos	Número de pontos acumulados.	INT	N	N	100
	palavra_passe	Pequeno código único que pode conter letras maiúsculas e minúsculas, caracteres especiais e números.	VARCHAR[64]	N	N	"diogopsantos23\$"
	email	E-mail pessoal do utilizador.	VARCHAR[45]	N	N	diogo06pereira@gmail.com
	biografia	Breve descrição pessoal.	VARCHAR[400]	S	N	"Explorar o mundo é a minha paixão!"
	id_utilizador	Identificador único e inalterável de um utilizador.	INT	N	N	56
viagem	data_chegada	Data de chegada ao destino.	DATE	N	N	2024-11-15
	data_partida	Data de partida da origem.	DATE	N	N	2024-11-11
	id_viagem	Identificador único de cada viagem.	INT	N	N	130
publicacao	descricao	Breve descrição sobre a viagem.	VARCHAR[500]	S	N	"Gibraltar surpreendeu-me bastante."
	data	Dia, mês e ano de publicação da viagem.	DATE	N	N	2024-11-20
	id_publicacao	Identificador único de cada publicação.	INT	N	N	4
vale_de_desc onto	usado	Número de pontos usados para obter o vale de desconto. Estes pontos, obtidos através do número de publicações/viagens efetuadas.	TINYINT	N	N	1
	id_vale_desconto	Identificador único do vale de desconto.	INT	N	N	2
ficheiro_audio _visual	id_ficheiro	Identificador único do ficheiro multimédia.	INT	N	N	45
	ficheiro_audiovisual	Nome do ficheiro áudio-visual.	VARCHAR(400)	N	N	"foto1.png"
tipo_vale	id_vale_desconto	Identificador único do tipo de vale.	INT	N	N	7
	descricao_tipo	Designação do tipo de vale.	VARCHAR(45)	N	N	"10% de desconto no "Hotel Axis Vemar Conference & Beach Hotel""
	nome_tipo_vale	Nome do tipo do vale.	VARCHAR(30)	N	N	"Hotel"
	valor_pontos_tipo	Valor do tipo do vale em pontos.	INT	N	N	20
comentario	id_comentario	Identificador único do comentário.	INT	N	N	203
	comentario	Texto do comentário associado a uma publicação.	VARCHAR[500]	N	S	"Que paraíso!!"
cidade	id_cidade	Identificador único da cidade.	INT	N	N	15
	nome_cidade	Nome da cidade.	VARCHAR(45)	N	N	"Londres"
pais	id_pais	Identificador único do país.	INT	N	N	12
	nome_pais	Nome do país.	VARCHAR(45)	N	N	"Portugal"
tipo_viagem	id_tipo_viagem	Identificador único do tipo de viagem.	INT	N	N	3
	nome_tipo	Nome do tipo da viagem	VARCHAR(45)	N	N	"Lazer"

Tabela 6 - Atributos identificados pela equipa de desenvolvimento

3.5. Apresentação e Explicação do Diagrama ER Produzido

Após a identificação e caracterização de entidades, relacionamentos e os seus atributos, procedeu-se à esquematização de um diagrama Entidade-Relacionamento.

Este processo foi realizado com recurso ao *software* brModelo.

O diagrama desenvolvido pela equipa de trabalho representa todos os elementos fundamentais da plataforma NOMÁLIA e as interações entre eles. O objetivo principal é fornecer uma visão clara do funcionamento do sistema, garantindo que todas as necessidades identificadas durante o levantamento de requisitos se encontram refletidas numa estrutura coerente.

O diagrama Entidade-Relacionamento apresenta as 10 entidades identificadas, descrevendo os relacionamentos essenciais que suportam a dinâmica da plataforma. A seguir, apresenta-se uma explicação detalhada de cada componente do modelo:

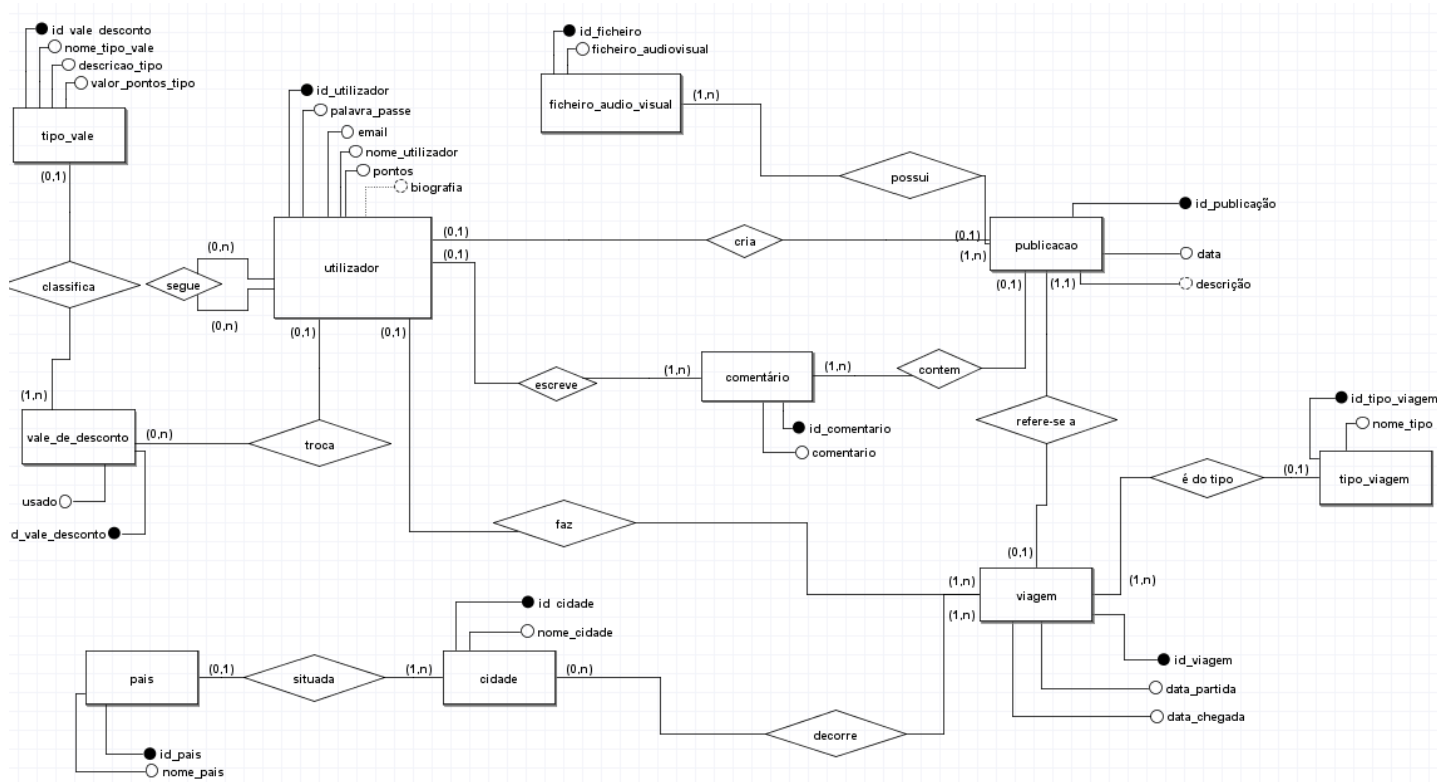


Figura 3 - Diagrama ER produzido

Nota: Durante a construção do modelo conceptual, para efeitos de apresentação no relatório, foi necessário reduzir a escala do diagrama para garantir a sua legibilidade numa única página. Essa redução fez com que alguns caracteres “_” presentes na designação dos atributos deixassem de ser visíveis na imagem exportada, sem qualquer impacto semântico ou estrutural. A nomenclatura correta dos atributos mantém-se intacta nas definições lógica e física da Base de Dados e é respeitada em todas as restantes secções do relatório.

4. Modelação Lógica

Após a definição do modelo conceptual através do diagrama Entidade-Relacionamento, procedeu-se à sua conversão para o modelo lógico relacional. Este processo teve como objetivo traduzir entidades, atributos e relacionamentos do modelo conceptual para tabelas, chaves primárias, chaves estrangeiras e restrições. Para a construção deste modelo lógico, foi utilizado o *MySQL Workbench*.

4.1. Construção e Validação do Modelo de Dados Lógico

A construção e validação do modelo de dados lógico constituíram uma etapa fundamental no desenvolvimento da base de dados da plataforma NOMÁLIA. Após a definição do modelo conceptual, foi necessário converter todas as entidades, atributos e relacionamentos para um formato relacional, garantindo que a estrutura obtida fosse coerente, funcional e compatível com o sistema de gestão escolhido.

Este processo permitiu transformar conceitos abstratos em tabelas concretas, definindo chaves primárias, chaves estrangeiras e restrições capazes de assegurar a integridade e fiabilidade dos dados.

Através do uso do *MySQL Workbench*, foi possível visualizar, analisar e confirmar que o modelo lógico representava de forma fiel o modelo conceptual, garantindo que o sistema estaria preparado para suportar as funcionalidades previstas na plataforma.

Esta etapa assegura, assim, uma base sólida para a posterior implementação física da base de dados.

4.2. Apresentação e Explicação do Modelo Lógico Produzido

O modelo lógico produzido resulta da transformação do diagrama Entidade-Relacionamento num conjunto de tabelas relacionais. Abaixo apresentamos uma explicação sobre as decisões tomadas na construção do modelo lógico, bem como a função de cada tabela e a forma como os relacionamentos foram convertidos.

O modelo lógico é composto por várias tabelas principais, correspondentes às entidades centrais identificadas no modelo conceptual, nomeadamente “utilizador”, “viagem”, “publicacao”, “comentario”, “ficheiro_audio_visual” e “vale_de_desconto”, bem como por tabelas de apoio responsáveis pela classificação e organização dos dados, como “pais”, “cidade”, “tipo_viagem” e “tipo_vale”. Para além destas, foram ainda incluídas duas tabelas associativas, necessárias para representar o relacionamento do tipo N:M, a tabela “segue” e “viagem_cidade” assegurando a correta implementação das interações definidas no modelo conceptual.

Cada tabela contém a sua chave primária, atributos específicos e, quando aplicável, uma ou mais chaves estrangeiras que asseguram a ligação entre tabelas, garantindo assim a integridade da base de dados.

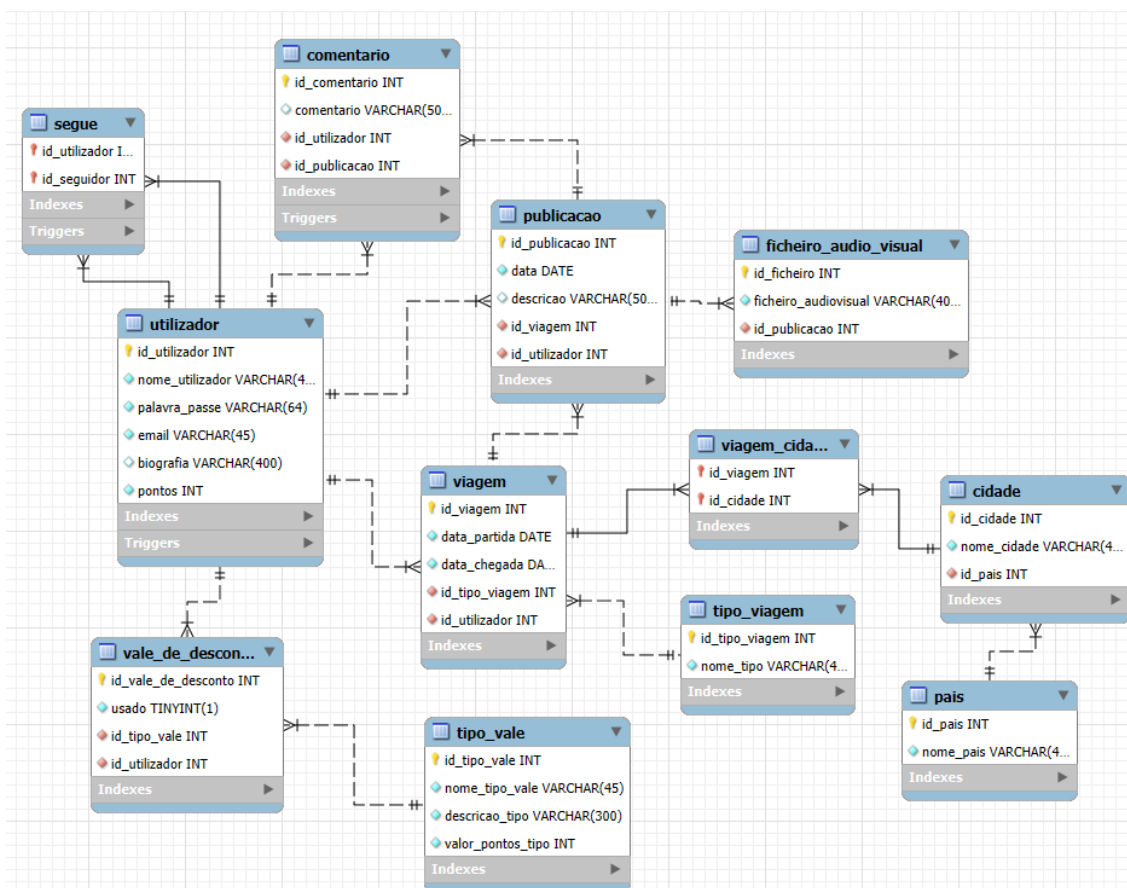


Figura 4 - Modelo lógico produzido

4.2.1. Explicação das tabelas

A estrutura do modelo lógico da plataforma NOMÁLIA assenta num conjunto de tabelas principais que representam as entidades do sistema e suportam as funcionalidades essenciais da aplicação. Estas tabelas constituem a base da organização dos dados, assegurando uma divisão clara entre utilizadores, viagens, publicações e vales de desconto, bem como a integridade das relações estabelecidas entre elas.

Cada uma destas tabelas foi concebida para refletir, de forma fiel e normalizada, as características fundamentais do modelo conceptual, garantindo consistência na gestão da informação e permitindo que as operações da plataforma decorram de forma eficiente e estruturada.

- **Tabela “utilizador”**

A tabela “**utilizador**” armazena toda a informação essencial sobre os membros registados na plataforma NOMÁLIA. Cada registo corresponde a um único utilizador e é identificado de forma inequívoca através do atributo **id_utilizador**, que funciona como chave primária da tabela.

Esta tabela inclui atributos de identificação e caracterização do utilizador, nomeadamente o **nome_utilizador**, o endereço de correio eletrónico (**email**) e uma **biografia** opcional, bem como atributos relacionados com a segurança e funcionamento do sistema, como a **palavra_passe**, utilizada no processo de autenticação, e o número de **pontos** acumulados pelo utilizador ao longo da sua utilização da plataforma.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_utilizador	INT	N	--	Identificador único do utilizador, funcionando como chave primária da tabela.
nome_utilizador	VARCHAR(45)	N	--	Nome escolhido pelo utilizador para se identificar na plataforma.
pontos	INT	N	--	Número de pontos associados a um utilizador.
palavra_passe	VARCHAR (64)	N	--	Código secreto definido pelo utilizador para garantir a segurança do acesso ao sistema.
email	VARCHAR (45)	N	--	Endereço eletrónico associado ao utilizador, utilizado para autenticação e recuperação de conta.
biografia	VARCHAR (400)	S	--	Texto facultativo que permite ao utilizador apresentar uma breve descrição sobre si próprio.

Tabela 7 - Dicionário de dados de “utilizador”

- **Tabela “viagem”**

A tabela “**viagem**” representa todas as viagens realizadas pelos utilizadores. Cada registo corresponde a uma viagem específica e é identificado de forma única através do atributo **id_viagem**, que funciona como chave primária da tabela. Esta tabela armazena informação temporal relativa à viagem, nomeadamente as **data_partida** e **data_chegada**, permitindo organizar cronologicamente as experiências registadas pelos utilizadores. A classificação da viagem é assegurada através da ligação com a tabela **tipo_viagem**, com a associação a um **id_tipo_viagem**, possibilitando a categorização das experiências realizadas. A tabela **viagem** encontra-se ainda ligada à tabela **utilizador**, garantindo que cada viagem está associada a um utilizador concreto.

Desta forma, a tabela **viagem** assume um papel central na organização das experiências dos utilizadores, servindo de base para a associação de conteúdos partilhados na plataforma, como publicações, e para a contextualização espacial e temporal das atividades desenvolvidas.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_viagem	INT	N	--	Identificador único de cada viagem, funcionando como chave primária da tabela.
id_tipo_viagem	VARCHAR(45)	N	FK → tipo_viagem (id_tipo_viagem)	Chave estrangeira que identifica a categoria da viagem realizada, permitindo classificar e organizar as experiências dos utilizadores.
data_partida	DATE	N	--	Registo da data de início, permitindo organizar temporalmente as experiências do utilizador.
data_chegada	DATE	N	--	Registo da data de chegada, permitindo organizar temporalmente as experiências do utilizador.
id_utilizador	INT	N	FK → utilizador (id_utilizador)	Chave estrangeira que identifica o utilizador que realizou a viagem, garantindo a ligação entre a experiência e o membro responsável por ela.

Tabela 8 - Dicionário de dados de “viagem”

- Tabela “publicacao”

A tabela “**publicacao**” armazena todas as publicações criadas pelos utilizadores no contexto das suas viagens. Cada registo corresponde a uma publicação específica e é identificado de forma única através do atributo **id_publicacao**, que funciona como chave primária da tabela. Esta tabela inclui a **descrição** da publicação e a respetiva **data** de criação, permitindo registar e organizar temporalmente os conteúdos partilhados pelos utilizadores. Cada publicação encontra-se obrigatoriamente associada a uma única viagem, através do atributo **id_viagem**, garantindo que todo o conteúdo publicado está contextualizado numa experiência concreta. Assim como, se encontra associada a um único utilizador também, através do atributo **id_utilizador**, permitindo que a mesma esteja enquadrada corretamente com uma pessoa. A tabela “**publicacao**” assume um papel central no modelo lógico, uma vez que constitui o ponto de ligação entre as viagens e os restantes elementos de interação da plataforma, nomeadamente os ficheiros áudio-visuais associados e os comentários efetuados por outros utilizadores.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_publicacao	INT	N	--	Identificador único da publicação, que funciona como chave primária da tabela.
descricao	VARCHAR (500)	N	--	Texto introduzido pelo utilizador para descrever a publicação.
data	DATE	N	--	Data em que a publicação foi criada.
id_viagem	INT	N	FK → viagem (id_viagem)	Chave estrangeira que identifica a viagem associada à publicação, garantindo a ligação entre o conteúdo publicado e a viagem que lhe deu origem.
id_utilizador	INT	N	FK → utilizador (id_utilizador)	

Tabela 9 - Dicionário de dados de “publicacao”

- **Tabela “vale_de_desconto”**

A tabela “**vale_de_desconto**” armazena os vales de desconto disponibilizados na plataforma NOMÁLIA no âmbito do sistema de recompensas. Cada registo corresponde a um vale específico e é identificado de forma única através do atributo **id_vale_de_desconto**, que funciona como chave primária da tabela. Esta tabela inclui o atributo **usado**, que indica se o vale de desconto já foi utilizado ou não, permitindo controlar o estado de cada recompensa atribuída. A associação ao **tipo_vale**, através do atributo **id_tipo_vale**, possibilita a classificação dos vales de acordo com a sua tipologia, definida pela própria plataforma. Cada vale de desconto encontra-se associado a um **utilizador**, através do atributo **id_utilizador**, garantindo que a recompensa é atribuída a um membro concreto da plataforma. Desta forma, a tabela **vale_de_desconto** desempenha um papel fundamental na gestão do sistema de pontos e recompensas, assegurando o correto registo, controlo e utilização dos incentivos disponibilizados aos utilizadores.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_vale_de_desconto	INT	N	--	Identificador único do vale de desconto, que funciona como chave primária da tabela.
usado	TINYINT	N	--	Valor 0 a 1 que permite saber se o vale de desconto foi ou não usado.
id_tipo_vale	INT	N	FK → tipo_de_vale (id_tipo_vale)	Chave estrangeira que identifica o tipo de vale associado a um determinado vale, permitindo a ligação entre o vale atribuído e a respetiva categoria definida no sistema.
id_utilizador	INT	N	FK → utilizador (id_utilizador)	Chave estrangeira que identifica o utilizador a quem o vale está associado, garantindo a ligação entre o vale e a pessoa que o obteve e utiliza.

Tabela 10 - Dicionário de dados de “vale_de_desconto”

- **Tabela “tipo_vale”**

A tabela “**tipo_vale**” armazena a informação relativa às diferentes tipologias de vales de desconto disponibilizadas pela plataforma NOMÁLIA. Cada registo corresponde a um tipo de vale distinto e é identificado de forma única através do atributo **id_tipo_vale**, que funciona como chave primária da tabela. Esta tabela inclui atributos que permitem caracterizar cada tipo de vale, nomeadamente o **nome_tipo_vale**, a **descricao_tipo** e o **valor_pontos_tipo**, que indica o número de pontos necessários para que um utilizador possa obter um vale dessa tipologia. Estes valores são definidos e geridos pela própria plataforma, garantindo uniformidade e controlo sobre o sistema de recompensas. A tabela “**tipo_vale**” desempenha um papel fundamental na organização e gestão dos vales de desconto, permitindo classificar e estruturar os diferentes tipos de recompensas disponíveis, de forma a dar suporte ao mecanismo de troca de pontos por benefícios na plataforma NOMÁLIA.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_tipo_vale	INT	N	--	Identificador único do tipo do vale de desconto, que funciona como chave primária da tabela.
nome_tipo_vale	VARCHAR(45)	N	--	Informação que define o tipo do vale de desconto.
descricao_tipo	VARCHAR (300)	N	--	Texto que descreve de forma detalhada o tipo do vale.
valor_pontos_tipo	INT	N	--	Número de pontos que um utilizador precisa de ter para esse tipo de vale.

Tabela 11 - Dicionário de dados de “tipo_do_vale”

- **Tabela “tipo_viagem”**

A tabela “**tipo_viagem**” armazena as diferentes categorias de viagem existentes na plataforma NOMÁLIA. Cada registo corresponde a um tipo de viagem distinto e é identificado de forma única através do atributo **id_tipo_viagem**, que funciona como chave primária da tabela. Esta tabela inclui o atributo **nome_tipo**, que permite designar e identificar a categoria da viagem, como por exemplo viagens de lazer, profissionais, culturais ou gastronómicas. Os valores desta tabela são definidos e geridos pela própria plataforma, garantindo consistência na classificação das viagens realizadas pelos utilizadores. A tabela **tipo_viagem** tem como principal função suportar a categorização das viagens, facilitando a organização, análise e gestão das experiências registadas na plataforma NOMÁLIA.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_tipo_viagem	INT	N	--	Identificador único do tipo da viagem, que funciona como chave primária da tabela.
nome_tipo	VARCHAR(45)	N	--	Informação legível que define o nome do tipo da viagem.

Tabela 12 - Dicionário de dados de “tipo_viagem”

- **Tabela “pais”**

A tabela “**pais**” armazena a informação relativa aos países identificados na plataforma NOMÁLIA. Cada registo corresponde a um país distinto e é identificado de forma única através do atributo **id_pais**, que funciona como chave primária da tabela. Esta tabela inclui o atributo **nome_pais**, que permite identificar o nome do país associado às cidades e viagens registadas no sistema. A existência desta tabela permite estruturar a informação geográfica de forma normalizada, assegurando consistência na identificação dos países e evitando redundância de dados.

A tabela “**pais**” é utilizada como elemento de apoio à localização geográfica das viagens, servindo de referência para a associação das cidades e contribuindo para a correta contextualização das experiências dos utilizadores na plataforma NOMÁLIA.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_pais	INT	N	--	Identificador único do país associado, que funciona como chave primária da tabela.
nome_pais	VARCHAR(45)	N	--	Informação legível que define o nome do país associado às cidades que foram conectadas a uma viagem.

Tabela 13 - Dicionário de dados de “pais”

- **Tabela “cidade”**

A tabela “**cidade**” armazena a informação relativa às cidades associadas às viagens registadas na plataforma NOMÁLIA. Cada registo corresponde a uma cidade distinta e é identificado de forma única através do atributo **id_cidade**, que funciona como chave primária da tabela. Esta tabela inclui o atributo **nome_cidade**, que permite identificar a cidade associada às experiências dos utilizadores. A associação da cidade a um país é assegurada através do atributo **id_pais**, permitindo estabelecer a ligação entre a cidade e o respetivo país onde esta se localiza. A existência da tabela “**cidade**” permite organizar a informação geográfica de forma normalizada, garantindo consistência na identificação das localizações e possibilitando que várias viagens sejam associadas à mesma cidade, evitando redundância e assegurando a integridade dos dados geográficos na plataforma NOMÁLIA.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_cidade	INT	N	--	Identificador único de cada cidade associada a uma viagem, que funciona como chave primária da tabela.
nome_cidade	VARCHAR (45)	N	--	Informação legível que define o nome da cidade associada à viagem.
id_pais	INT	N	FK → pais (id_pais)	Chave estrangeira que identifica o país associado às respetivas cidades, garantindo a ligação entre a cidade e o país em que a mesma está localizada.

Tabela 14 - Dicionário de dados de “cidade”

- **Tabela “comentario”**

A tabela “**comentario**” armazena os comentários efetuados pelos utilizadores nas publicações da plataforma NOMÁLIA. Cada registo corresponde a um comentário específico e é identificado de forma única através do atributo **id_comentario**, que funciona como chave primária da tabela.

Esta tabela inclui o atributo **comentario**, que contém o texto introduzido pelo utilizador, permitindo a partilha de opiniões e feedback relativamente às publicações existentes na plataforma. A associação do comentário a um utilizador é garantida através do atributo **id_utilizador**, permitindo identificar o autor do comentário.

Adicionalmente, cada comentário encontra-se associado a uma publicação específica por meio do atributo **id_publicacao**, assegurando que todos os comentários estão contextualizados num conteúdo concreto. Desta forma, a tabela “**comentario**” desempenha um papel fundamental no suporte às interações entre os utilizadores da plataforma NOMÁLIA.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_comentario	INT	N	--	Identificador único de um comentário, que funciona como chave primária da tabela.
comentario	VARCHAR (500)	N	--	Texto escrito pelo utilizador, onde o mesmo pode partilhar a sua opinião.
id_utilizador	INT	N	FK → utilizador (id_utilizador)	Chave estrangeira que identifica o utilizador associado ao comentário, que garante a ligação entre o comentário e o respetivo utilizador.
id_publicacao	INT	N	FK → publicacao (id_publicacao)	Chave estrangeira que identifica a publicação à qual o comentário está associado, que permite a identificação da publicação onde o comentário foi escrito.

Tabela 15 - Dicionário de dados de “comentario”

- **Tabela “ficheiro_audio_visual”**

A tabela “ficheiro_audio_visual” armazena os ficheiros multimédia associados às publicações da plataforma NOMÁLIA. Cada registo corresponde a um ficheiro áudio-visual específico e é identificado de forma única através do atributo **id_ficheiro**, que funciona como chave primária da tabela.

Esta tabela inclui o atributo “ficheiro_audiovisual”, que armazena a referência ao ficheiro multimédia associado à publicação, como por exemplo o nome ou caminho do ficheiro. A associação de cada ficheiro a uma publicação é assegurada através do atributo **id_publicacao**, garantindo que todos os conteúdos multimédia estão devidamente contextualizados num conteúdo publicado.

A existência da tabela “ficheiro_audio_visual” permite que cada publicação possua um ou vários ficheiros associados, assegurando uma gestão adequada dos conteúdos multimédia, evitando redundância e garantindo a identificação inequívoca de cada ficheiro na plataforma NOMÁLIA.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_ficheiro	INT	N	--	Identificador único do ficheiro audiovisual, que funciona como chave primária da tabela.
ficheiro_audiovisual	VARCHAR (400)	N	--	Informação que identifica o nome do ficheiro audiovisual.
id_publicacao	INT	N	FK → publicacao (id_publicacao)	Chave estrangeira que identifica a publicação à qual o ficheiro audiovisual está associado, que garante que o mesmo está ligado à publicação correta.

Tabela 16 - Dicionário de dados de “ficheiro_audio_visual”

- **Tabela “segue”**

A tabela “segue” implementa o relacionamento recursivo do tipo N:M entre utilizadores da plataforma NOMÁLIA, permitindo que um utilizador siga outros utilizadores. Cada registo estabelece uma ligação entre um utilizador seguidor e um utilizador seguido, sendo ambos identificados através de chaves estrangeiras que referenciam a tabela **utilizador**.

O atributo **id_seguidor** identifica o utilizador que realiza a ação de seguir, enquanto o atributo **id_utilizador** identifica o utilizador que é seguido. Em conjunto, estes atributos constituem a chave primária composta da tabela, garantindo que cada relação de seguimento é registada de forma única.

A tabela “segue” assegura a correta implementação do único relacionamento N:M presente no modelo conceptual e garante que as interações de seguimento entre utilizadores são registadas de forma consistente, normalizada e alinhada com os princípios fundamentais da modelação de dados.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_seguidor	INT	N	FK -> utilizador (id_utilizador)	Identifica o utilizador que segue outro utilizador.
id_utilizador	INT	N	FK -> utilizador (id_utilizador)	Identifica o utilizador que é seguido.

Tabela 17 - Tabela de Relacionamento “segue”

Esta tabela assegura a correta implementação do único relacionamento N:M presente no modelo conceptual e garante que as interações de seguimento entre utilizadores são registadas de forma consistente, normalizada e alinhada com os princípios fundamentais da modelação de dados.

- Tabela “**viagem_cidade**”

A tabela “**viagem_cidade**” é uma tabela associativa que materializa o relacionamento N:M entre as entidades **viagem** e **cidade**. Esta tabela existe para permitir que uma mesma viagem possa estar associada a várias cidades, bem como que uma cidade possa integrar várias viagens, refletindo corretamente o percurso de uma viagem ao longo de diferentes localizações.

Cada registo da tabela representa a associação entre uma viagem específica e uma cidade específica. A identificação de cada registo é garantida através de uma chave primária composta, formada pelos atributos **id_viagem** e **id_cidade**, que assegura a unicidade de cada associação viagem e cidade e evita duplicações.

Ambos os atributos funcionam simultaneamente como chaves estrangeiras, garantindo a integridade referencial relativamente às tabelas **viagem** e **cidade**.

Atributo	Domínio	Nulo	Chave Estrangeira	Descrição
id_viagem	INT	N	FK -> viagem (id_viagem)	Identificador da viagem associada que funciona como parte da chave primária composta e estabelece a ligação à tabela viagem.
id_cidade	INT	N	FK -> cidade (id_utilizador)	Identifica a cidade associada, funcionando como parte da chave primária composta e estabelecendo a ligação à tabela cidade.

Tabela 18 - Tabela de Relacionamento “viagem_cidade”

4.3. Normalização de Dados

A Normalização de Dados corresponde a um conjunto de princípios que asseguram a organização eficiente das estruturas de informação, como: tabelas, atributos e relacionamentos. O seu objetivo é eliminar redundâncias, evitar problemas de inserção, atualização ou eliminação, e garantir a integridade dos dados. A aplicação das formas normais permite estruturar cada tabela de forma lógica e consistente, assegurando que cada atributo desempenha uma função específica e devidamente justificada no modelo de dados.

A normalização foi realizada tendo como referência o modelo lógico desenvolvido para a plataforma NOMÁLIA, analisando-se a validação das entidades segundo as três primeiras formas normais (1FN, 2FN e 3FN), consideradas fundamentais para garantir a robustez de um Sistema de Base de Dados.

4.3.1. Primeira Forma Normal (1FN)

No modelo lógico da plataforma NOMÁLIA, todas as entidades foram estruturadas de forma a evitar atributos que pudessem armazenar múltiplos valores. Sempre que surgiram elementos de informação suscetíveis de ocorrer repetidamente para a mesma entidade, optou-se pela sua representação em entidades separadas, assegurando a correta estruturação dos dados.

Um exemplo claro desta abordagem é o caso dos ficheiros multimédia associados às publicações. Para evitar que uma publicação tivesse de armazenar múltiplos ficheiros num único atributo, foi criada a entidade “ficheiro_audio_visual”, permitindo a representação independente de cada ficheiro. Esta separação possibilita o registo individual de cada ficheiro, eliminando valores multivalorados e evitando dados repetitivos.

Assim, o modelo lógico assegura que cada atributo contém apenas um valor indivisível e que todas as ocorrências múltiplas são representadas através de relações entre entidades, demonstrando que a estrutura se encontra conforme os requisitos da Primeira Forma Normal.

4.3.2. Segunda Forma Normal (2FN)

No modelo lógico da plataforma NOMÁLIA, as tabelas principais apresentam chaves primárias simples, constituídas por um único atributo identificador. Nesses casos, não existe a possibilidade de ocorrência de dependências parciais, uma vez que todos os atributos dependem necessariamente da totalidade da chave primária.

A análise da conformidade com esta forma normal torna-se, assim, relevante apenas na tabela associativa “**segue**”, que implementa um relacionamento do tipo N:M e possui uma chave primária composta pelos atributos “id_seguidor” e “id_seguido”. Esta tabela não contém quaisquer atributos adicionais para além dos que constituem a própria chave primária, pelo que não existem atributos suscetíveis de depender apenas de parte dessa chave. A tabela associativa “**viagem_cidade**”, que implementa o relacionamento N:M entre as entidades viagem e cidade, possui uma chave primária composta pelos atributos “id_viagem” e “id_cidade”. Esta tabela não contém atributos adicionais para além dos que constituem a própria chave primária, pelo que não existem dependências parciais.

Deste modo, verifica-se que não ocorrem dependências parciais em nenhuma das tabelas do modelo, concluindo-se que a totalidade da base de dados se encontra devidamente estruturada ao nível da Segunda Forma Normal.

4.3.3. Terceira Forma Normal (3FN)

A análise das estruturas do esquema evidencia que, em todas as tabelas, os atributos apresentam uma dependência direta e inequívoca relativamente à respetiva chave primária, não se verificando situações em que um atributo dependa de outro atributo não identificador.

De seguida, analisaremos cada tabela individualmente:

- Tabela “**utilizador**”: A chave primária “id_utilizador” identifica unicamente cada utilizador. Os restantes atributos: “nome_utilizador”, “palavra_passe”, “email”, “biografia” e “pontos” dependem exclusivamente da chave primária, não existindo quaisquer dependências parciais nem transitivas. A estrutura da tabela garante, assim, a ausência de dependências transitivas.
- Tabela “**publicacao**”: A chave primária “id_publicacao” identifica, de forma única, cada publicação. Os atributos “descricao”, “data”, “id_viagem” e “id_utilizador” dependem diretamente dessa chave primária, não sendo possível inferir qualquer atributo a partir de outro que não seja a própria chave primária.
- Tabela “**viagem**”: A chave primária “id_viagem” identifica cada viagem de forma independente. Os atributos “id_tipo_viagem”, “data_partida”, “data_chegada”, “id_utilizador” encontram-se diretamente associados a essa identificação. A associação às cidades não é realizada diretamente nesta tabela, sendo implementada através da tabela associativa “viagem_cidade”, o que elimina redundâncias e dependências transitivas, assegurando a conformidade com a Terceira Forma Normal.
- Tabela “**tipo_viagem**”: Possui uma chave primária simples “id_tipo_viagem”, da qual depende exclusivamente o atributo “nome_tipo”. Não existem outros atributos que possam introduzir dependências adicionais.
- Tabela “**viagem_cidade**”: A tabela possui uma chave primária composta pelos atributos “id_viagem” e “id_cidade”, ambos chaves estrangeiras. Não existem atributos não-chave, pelo que não se verificam dependências funcionais nem transitivas. Assim, a tabela encontra-se em conformidade com a Terceira Forma Normal.

- Tabela “**vale_de_desconto**”: A chave primária “id_vale_desconto” identifica cada vale de desconto. Os atributos “usado”, “id_tipo_vale” e “id_utilizador” estão diretamente relacionados com a chave primária, não dependendo entre si, o que assegura uma estrutura livre de dependências transitivas.
- Tabela “**tipo_vale**”: Possui uma chave primária simples “id_tipo_vale”, à qual estão associados os atributos “nome_tipo_vale”, “descricao_tipo” e “valor_pontos_tipo” que descrevem exclusivamente o tipo de vale identificado pela chave, não existindo dependências funcionais entre eles.
- Tabela “**segue**”: responsável pela implementação do relacionamento N:M entre utilizadores e constituída apenas pelos atributos “id_utilizador” e “id_seguidor”, que compõem a sua chave primária composta. Não existem atributos não-chave, logo não há dependências funcionais nem transitivas.
- Tabela “**ficheiro_audio_visual**”: A chave primária “id_ficheiro” identifica cada ficheiro multimédia. O atributo “ficheiro_audio_visual” e a chave estrangeira “id_publicacao” dependem exclusivamente desta chave primária, não existindo dependências indiretas.
- Tabela “**comentario**”: A chave primária “id_comentario” identifica cada comentário de forma única. Os atributos “comentario”, “id_utilizador” e “id_publicacao” dependem diretamente dessa chave, sem que qualquer atributo não-chave determine outro.
- Tabela “**pais**”: Possui uma chave primária simples “id_pais” e um atributo descritivo “nome_pais” que depende exclusivamente desta chave, não existindo dependências transitivas.
- Tabela “**cidade**”: A chave primária “id_cidade” identifica cada cidade. Os atributos “nome_cidade” e “id_pais” estão diretamente associados à chave primária, sendo a hierarquia geográfica corretamente representada através de chaves estrangeiras, sem redundância de informação.

4.4. Validação do Modelo com Interrogações do Utilizador

A validação do modelo lógico foi realizada através da formulação e execução de um conjunto de interrogações de utilizador, definidas a partir dos requisitos de manipulação identificados na fase inicial do projeto. O objetivo desta validação consiste em comprovar que o esquema relacional construído é capaz de suportar todas as operações funcionais previstas, garantindo que as tabelas, chaves primárias, chaves estrangeiras e restrições de integridade dão resposta completa às necessidades operacionais da plataforma NOMÁLIA.

Deste modo, procedeu-se à transformação de cada requisito de manipulação numa interrogação formal, aplicando princípios da álgebra relacional. Esta abordagem permitiu verificar se o modelo é apto para representar corretamente as interações entre entidades e se permite executar ações, como: seguir utilizadores, criar e gerir publicações, adicionar viagens, realizar comentários, trocar vales de desconto ou aplicar permissões administrativas. A reavaliação destes requisitos, anteriormente descritos na Secção 2.2.2, encontra-se sintetizada novamente na tabela seguinte, de forma a facilitar a compreensão do processo de validação:

Requisitos	Data/Hora	Descrição	Área	Fonte	Analista
13	20/10 18h10	O utilizador pode seguir outros perfis, interagir com as respetivas publicações, deixar de seguir outros utilizadores e visualizar a lista dos seus seguidores.	Utilizador	Beatriz Silva	Ana Patricia Machado
14	20/10 18h30	O utilizador deve poder criar, editar e visualizar publicações associadas às suas viagens.	Publicação	Filipa Martins	Marta Araújo
15	20/10 19h10	O utilizador deve poder criar, visualizar e editar as suas viagens na plataforma.	Viagem	Ricardo Matos	Ana Patrícia Machado
16	21/10 19h00	Cada publicação tem uma secção de comentários, onde os utilizadores podem partilhar opiniões sobre a viagem em questão.	Comentário	Ricardo Matos	Marta Araújo
17	21/10 19h20	O utilizador deve poder criar, editar e visualizar os seus comentários.	Comentário	Beatriz Silva	Ana Patrícia Machado
18	22/10 10h40	O utilizador deve poder inserir, visualizar ou editar dados do seu perfil.	Utilizador	Filipa Martins	Marta Araújo
19	23/10 15h00	A plataforma deve permitir que os pontos sejam trocados por vales de desconto.	Vale de desconto	Beatriz Silva	Ana Patrícia Machado
20	23/10 15h30	A plataforma deve permitir ao administrador visualizar e remover viagens, as publicações a elas associadas e os comentários dos vários utilizadores que forem escritos nas mesmas.	Administração	Ricardo Matos	Teresa Teixeira

Tabela 2 - Requisitos de Manipulação identificados pela equipa de desenvolvimento

A equipa começou, então, a implementar as interrogações pela ordem em que os requisitos de manipulação se encontram na tabela. Note-se que a Álgebra Relacional não suporta operações de atualização. Assim, as expressões apresentadas identificam apenas o registo e os atributos envolvidos, sendo a modificação efetiva dos dados realizada na tradução para SQL.

Interrogação 1: Visualizar seguidores

Começamos então pelo requisito 13 onde se pretende visualizar os perfis que um utilizador segue. A interrogação parte da tabela “segue”, filtrando pelo “id_seguidor”, o utilizador X que irá seguir alguém. Esta seleção é, posteriormente, associada à tabela “utilizador”,

obtendo assim os nomes desses perfis, através de uma junção ($\bowtie_{segue.id_utilizador = seguido.id_utilizador}$ $\rho_{seguido}(utilizador)$). Para identificar os seguidores do utilizador, invertem-se as junções e o filtro de seleção.

Perfis que sigo: $\pi_{seguido.nome_utilizador \rightarrow NomeSeguido}(\sigma_{id_seguidor = x}(segue) \bowtie_{segue.id_utilizador = seguido.id_utilizador} \rho_{seguido}(utilizador))$

Perfis que me seguem: $\pi_{u_seguidor.nome_utilizador \rightarrow NomeSeguidor}(\sigma_{id_utilizador = x}(segue) \bowtie_{segue.id_seguidor = u_seguidor.id_utilizador} \rho_{u_seguidor}(utilizador))$

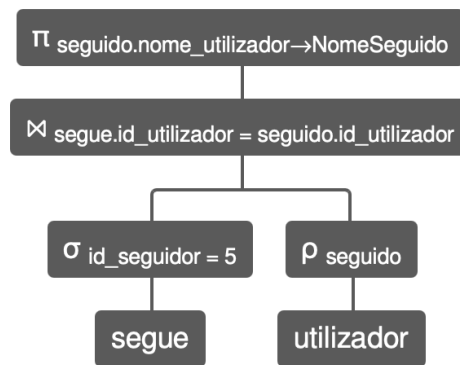


Figura 5 - Árvore de interrogação do requisito 13 aplicada aos dados de teste, para "id_utilizador" de "seguidor" igual a 5, para perfis que sigo

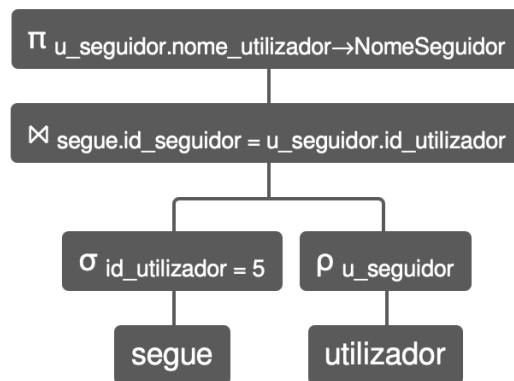


Figura 6 - Árvore de interrogação do requisito 13 aplicada aos dados de teste, para "id_utilizador" de "seguidor" igual a 5, para perfis que me seguem

Interrogação 2: Criar publicações associadas a viagens

Continuando com o requisito 14, pretende-se mostrar cada utilizador juntamente com as suas viagens e publicações associadas. Para isso, começamos por juntar a tabela "utilizador" com a tabela "viagem" ($utilizador \bowtie_{utilizador.id_utilizador=viagem.id_utilizador} viagem$). De seguida, juntamos a tabela "viagem_cidade" para se saber todos os locais visitados ($\bowtie_{viagem.id_viagem = vc.id_viagem} \rho_{vc}(viagem_cidade)$). Posteriormente, não só fizemos uma junção com a tabela

“cidade” para obtermos a cidade correspondente, mas também realizamos uma outra junção com a tabela “publicacao” para ligarmos a respetiva publicação à viagem a que corresponde ($\bowtie_{viagem.id_viagem=publicacao.id_viagem}$ publicacao). Por fim, aplicou-se uma projeção para apresentar apenas os atributos “nome_utilizador”, “nome_cidade” e “descricao” ($\pi_{nome_utilizador, destino, descricao}$).

$\pi_{nome_utilizador, nome_cidade, descricao} (((((utilizador \bowtie_{utilizador.id_utilizador=viagem.id_utilizador} viagem) \bowtie_{viagem.id_viagem=vc.id_viagem} \rho_{vc}(viagem_cidade)) \bowtie_{vc.id_cidade=cidade.id_cidade} cidade) \bowtie_{viagem.id_viagem=publicacao.id_viagem} publicacao))$

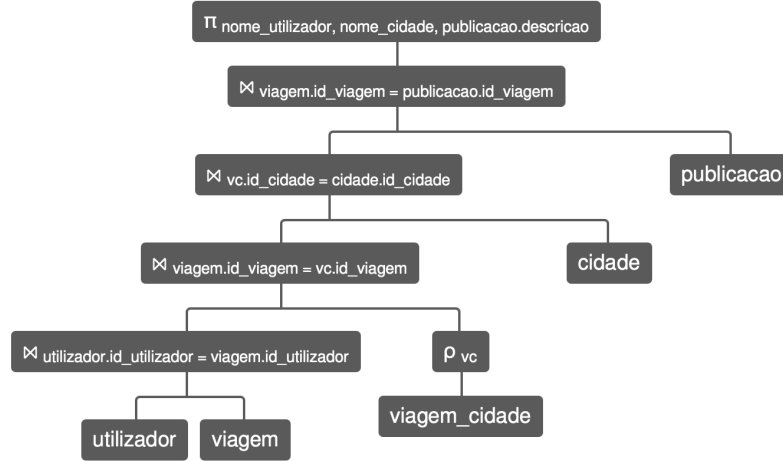


Figura 7 - Árvore de interrogação do requisito 14 aplicada aos dados de teste

Interrogação 3: Visualizar viagens de um utilizador

Com o requisito 15, pretende-se que um utilizador possa visualizar as suas próprias viagens. Para responder a esta interrogação utilizou-se uma operação de seleção sobre a tabela “viagem” ($\sigma_{id_utilizador=X}$ (viagem)), filtrando os registos onde o atributo “id_utilizador” correspondesse ao ID do utilizador em questão (X). Posteriormente, efetuou-se uma junção entre a tabela “viagem” e “tipo_viagem”, para se obter o tipo da viagem ($\bowtie_{tipo_viagem}$). Seguidamente, aplicou-se uma outra junção com a tabela “viagem_cidade”, para que se obtivessem as cidades onde a viagem foi realizada ($\bowtie_{\rho_{vc}(viagem_cidade)}$). Para além disso, efetuamos mais duas junções, uma com a tabela “cidade”, para obtermos os nomes das cidades ($\bowtie_{vc.id_cidade=cidade.id_cidade}$ cidade) e outra com a tabela “pais”, para obtermos o país correspondente às respectivas cidades ($\bowtie_{cidade.id_pais=pais.id_pais}$ pais). Por fim, aplica-se a operação de projeção para listar todos os atributos relevantes da viagem ($\pi_{id_viagem, nome_tipo, nome_cidade, nome_pais, data_partida, data_chegada}$).

$\pi_{id_viagem, nome_tipo, nome_cidade, nome_pais, data_partida, data_chegada} (((((\sigma_{id_utilizador=X} (viagem) \bowtie_{tipo_viagem} \rho_{tp}(tipo_viagem)) \bowtie_{vc.id_cidade=cidade.id_cidade} cidade) \bowtie_{cidade.id_pais=pais.id_pais} pais))$

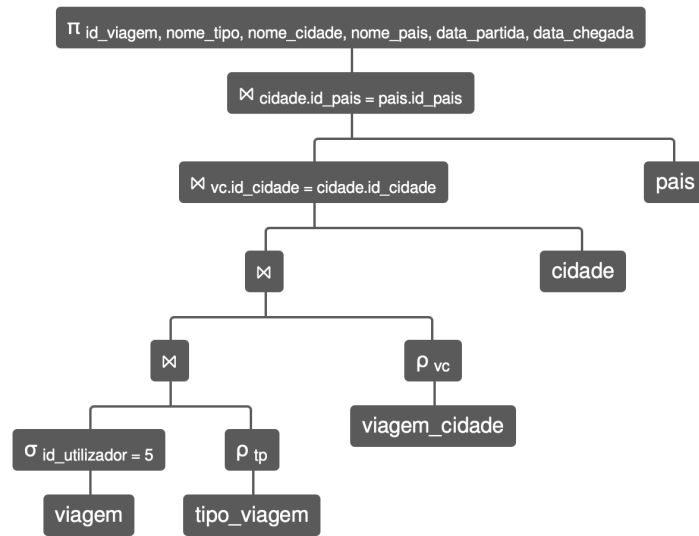


Figura 8 - Árvore de interrogação do requisito 15 aplicada aos dados de teste, para “id_utilizador” igual a 5

Interrogação 4: Consultar comentários das publicações

Continuamos com o requisito 16, onde se pretende listar os comentários de cada publicação juntamente com o utilizador que os escreveu. Para responder a esta interrogação utilizou-se uma junção entre a tabela “utilizador” e a tabela associativa “comentario” (utilizador $\bowtie_{\text{utilizador.id_utilizador=comentario.id_utilizador}}$ comentario), permitindo relacionar cada comentário com seu autor. Posteriormente, efetuou-se uma segunda junção com a tabela “publicacao” ($\bowtie_{\text{comentario.id_publicacao=publicacao.id_publicacao}}$ publicacao) para ligar o comentário à publicação correspondente. Por fim, aplicou-se uma projeção para extrair apenas “nome_utilizador”, “descricao” (da publicação) e “comentario”.

$\pi_{\text{utilizador.nome_utilizador, publicacao.descricao} \rightarrow \text{DescricaoPublicacao, comentario} \rightarrow \text{TextoComentario}} ((\text{utilizador} \bowtie_{\text{utilizador.id_utilizador=comentario.id_utilizador}} \text{comentario}) \bowtie_{\text{comentario.id_publicacao=publicacao.id_publicacao}} \text{publicacao})$

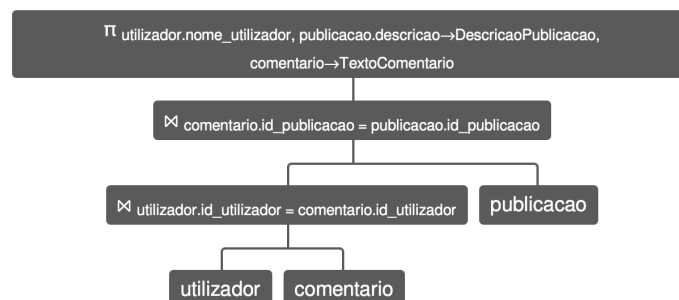


Figura 9 - Árvore de interrogação do requisito 16 aplicada aos dados de teste

Interrogação 5: Visualizar comentários feitos pelo utilizador

No requisito 17, pretende-se que um utilizador possa identificar e visualizar os seus comentários para editar. Para responder a esta interrogação utilizou-se uma operação de seleção sobre a tabela “comentario” ($\sigma_{id_utilizador=X}(\text{comentario})$), filtrando os registos pelo “id_utilizador” correspondente. Por fim, aplica-se a operação de projeção para listar os campos relevantes do comentário ($\pi_{id_comentario, comentario, id_publicacao}$). Desta forma, garante-se que apenas os comentários produzidos pelo utilizador sejam considerados para edição.

$\pi_{id_comentario, comentario, id_publicacao}(\sigma_{id_utilizador=X}(\text{comentario}))$

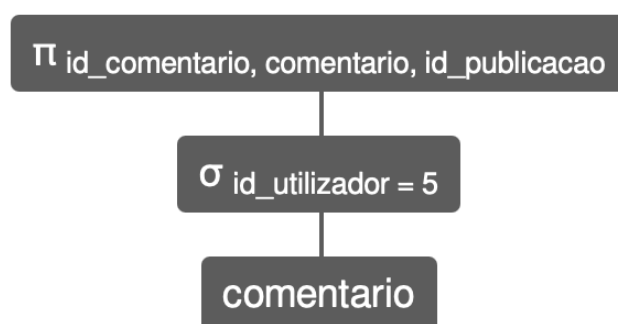


Figura 10 - Árvore de interrogação do requisito 17 aplicada aos dados de teste, com “id_utilizador” igual a 5

Interrogação 6: Editar dados do perfil

Seguimos com o requisito 18, onde se pretende que um utilizador possa visualizar e atualizar os seus dados do perfil. Para responder a esta interrogação utilizou-se uma operação de seleção sobre a tabela “utilizador”, filtrando o registo pelo “id_utilizador” do utilizador em questão ($\sigma_{id_utilizador=X}(\text{utilizador})$). Por fim, aplica-se uma projeção para listar os atributos que podem ser editados ($\pi_{nome_utilizador, email, biografia, pontos}$). Esta operação permite identificar exatamente os atributos que podem ser modificados, garantindo a integridade do perfil.

$\pi_{nome_utilizador, email, biografia, pontos}(\sigma_{id_utilizador=X}(\text{utilizador}))$

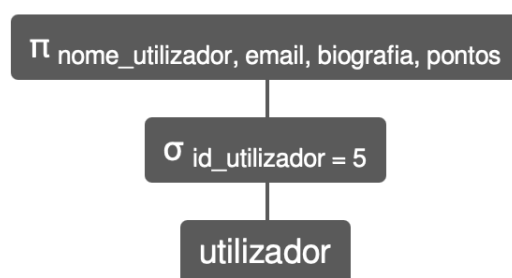
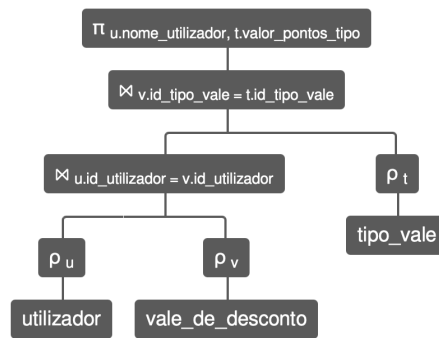


Figura 11 - Árvore de interrogação do requisito 18 aplicada aos dados de teste, com “id_utilizador” igual a 5

Interrogação 7: Listar Vales de Desconto Trocados pelo Utilizador

De seguida, apresenta-se o requisito 19, onde se pretende listar os vales de desconto que foram trocados por cada utilizador. Para responder a esta interrogação efetuamos uma primeira junção entre a tabela “utilizador” e a tabela “vale_de_desconto” ($\rho_u(\text{utilizador}) \bowtie_{u.id_utilizador=v.id_utilizador} \rho_v(\text{vale_de_desconto})$), permitindo identificar qual o utilizador que realizou a troca. Posteriormente, aplica-se uma segunda junção com a tabela “tipo_vale” onde se obtém o valor do vale ($\bowtie_{v.id_tipo_vale = t.id_tipo_vale} \rho_t(\text{tipo_vale})$). Por fim, efetuamos uma projeção que extrai os atributos “nome_utilizador” e “valor_pontos_tipo” ($\pi_{u.nome_utilizador, t.valor_pontos_tipo}$).

$\pi_{u.nome_utilizador, t.valor_pontos_tipo}((\rho_u(\text{utilizador}) \bowtie_{u.id_utilizador = v.id_utilizador} \rho_v(\text{vale_de_desconto})) \bowtie_{v.id_tipo_vale = t.id_tipo_vale} \rho_t(\text{tipo_vale}))$



$\rho_t(\text{tipo_vale}))$

Figura 12 - Árvore de interrogação do requisito 19 aplicada aos dados de teste

Interrogação 8: Remoção de publicações ou comentários pela administração

Concluindo, com o requisito 20, pretende-se identificar todos os conteúdos (viagens, publicações a elas associadas e comentários escritos nas mesmas) que devem ser removidos pelo administrador, se o mesmo o achar por bem. Para responder a esta interrogação iremos dividir o processo em quatro partes, uma para a viagem, outra para a publicação, outra para os ficheiros audiovisuais e outra para os comentários. Para as viagens, primeiramente efetuaremos uma junção entre as tabelas “viagem” e “utilizador” de forma a obter o utilizador que publicou a mesma ($\rho_v(\text{viagem}) \bowtie_{v.id_utilizador=u.id_utilizador} \rho_u(\text{utilizador})$). Seguidamente, faz-se uma outra junção com a tabela “tipo_viagem” para se saber qual o tipo da viagem que se irá eliminar ($\bowtie_{v.id_tipo_viagem=t.id_tipo_viagem} \rho_t(\text{tipo_viagem})$). Posteriormente, faz-se uma outra junção com a tabela “viagem_cidade”, para obtermos todas as cidades onde a viagem foi realizada ($\bowtie_{v.id_cidade=vc.id_cidade} \rho_c(\text{viagem_cidade})$), e, a posteriori, uma outra junção mas com a tabela “cidade” para obtermos o nome das cidades correspondentes. Por fim, aplicamos uma projeção para obtermos os atributos “id_viagem”, “nome_utilizador”, “nome_tipo”, “nome_cidade”, “data_partida” e “data_chegada”. Para garantir que removemos todos os conteúdos da publicação, eliminamos os ficheiros audiovisuais referentes à mesma. Primeiramente, fazemos uma junção entre as tabelas “publicacao” e “ficheiro_audio_visual” para identificarmos os registos que dependem da mesma ($\rho_f(\text{ficheiro_audio_visual}) \bowtie_{f.id_publicacao=p.id_publicacao} \rho_p(\text{publicacao})$). Por fim, fazemos uma projeção para obtermos o identificador do ficheiro e o respetivo conteúdo ($\pi_{f.id_ficheiro, f.ficheiro_audiovisual}$). Para as publicações, começamos por efetuar uma

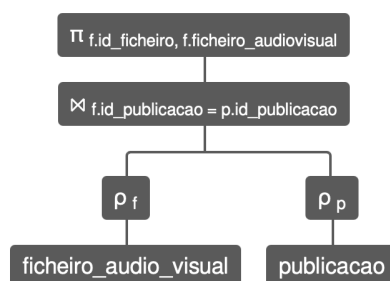


Figura 14 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte dos ficheiros audiovisuais

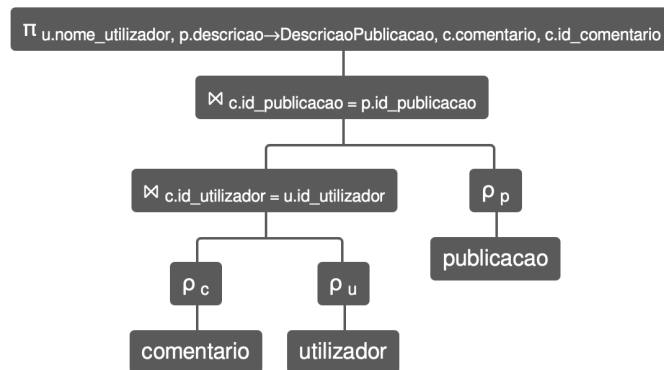


Figura 15 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte das publicações

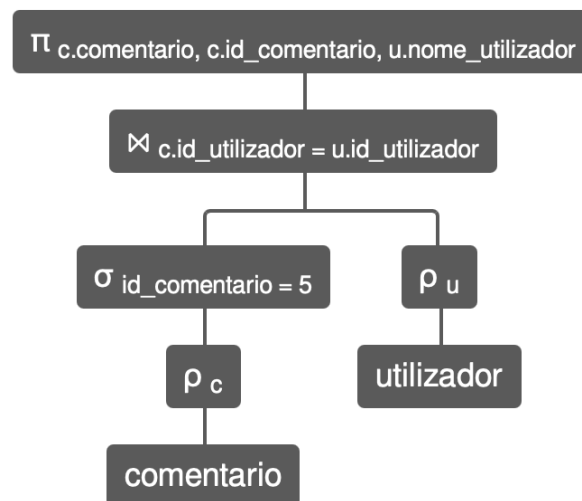


Figura 16 - Árvore de interrogação do requisito 20 aplicada aos dados de teste, por parte dos comentários

Concluindo, como se verificou, é possível a implementação de todas as interrogações e, por isso, o modelo lógico foi considerado válido e pronto a ser implementado fisicamente.

5. Implementação física

5.1. Apresentação e explicação da base de dados implementada

A Implementação Física da Base de Dados da plataforma NOMÁLIA corresponde à fase final do processo de modelação, em que o Modelo Lógico é previamente validado, normalizado e alinhado com os requisitos levantados.

O modelo físico da base de dados da plataforma NOMÁLIA é materializado através de um *script* SQL desenvolvido pela equipa de alunos. Este *script* é interpretado pelo SGBD MySQL, que cria automaticamente o esquema da base de dados e todas as tabelas necessárias, de acordo com as instruções fornecidas.

O primeiro passo da implementação consiste na criação do *schema* que irá conter todos os objetos da base de dados. No excerto abaixo observa-se o conjunto inicial de instruções:

```
DROP DATABASE IF EXISTS Nomalia;  
CREATE DATABASE Nomalia  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;  
USE Nomalia;
```

Estas instruções garantem que o ambiente de desenvolvimento se mantenha consistente a cada execução. Em pormenor:

“DROP DATABASE IF EXISTS Nomalia” elimina a base de dados caso ela já exista. Esta abordagem permite que o script seja executado repetidamente sem gerar erros, assegurando que não permanecem estruturas antigas que possam comprometer a integridade da implementação atual.

“CREATE DATABASE Nomalia” cria uma nova instância da base de dados com esse nome. À instrução é adicionada a configuração “CHARACTER SET utf8mb4”, que assegura a utilização de *Unicode* completo, permitindo armazenar corretamente a acentuação portuguesa. Por sua vez, “COLLATE utf8mb4_unicode_ci” estabelece que comparação e ordenação de texto devem ser insensíveis a maiúsculas, minúsculas e acentos, garantindo um comportamento mais intuitivo nas pesquisas realizadas pelos utilizadores.

“USE Nomalia” define que todas as instruções seguintes serão aplicadas a esta base de dados.

Segue-se a implementação das tabelas, que deve respeitar uma ordem lógica determinada pelas dependências existentes entre chaves estrangeiras. Neste caso, entidades como “utilizador”, “pais”, “tipo_viagem” e “tipo_vale” não dependem de quaisquer outras tabelas, logo devem ser criadas inicialmente.

Entidades dependentes como “vale_de_desconto”, que referencia “tipo_vale”, “viagem”, que referencia “utilizador” e “tipo_viagem”, e “cidade”, que referencia “pais”, só podem ser criadas depois das que referimos anteriormente, uma vez que referenciam chaves estrangeiras pertencentes às mesmas, seguidas das tabelas cujas dependências incluem diversas entidades, como a tabela associativa “viagem_cidade”.

Por fim, são criadas as tabelas dependentes de entidades previamente definidas, como “publicacao”, “ficheiro_audio_visual”, “comentario” e a tabela associativa “segue”.

Foi definida, portanto, uma sequência que garante que nenhuma chave estrangeira referência uma tabela ainda inexistente, onde não são necessárias instruções `ALTER TABLE` para adicionar restrições posteriormente, e que a construção do modelo físico se mantenha clara, sequencial e organizada. Para se descobrir a ordem da criação das tabelas, criou-se, então, um grafo de dependências, onde cada nodo equivale a uma tabela do modelo lógico. Seguidamente, definiu-se que o grafo tem aresta $A \rightarrow B$, o que significa que A é uma tabela que contém uma chave estrangeira que referencia a tabela B. Assim, definiu-se o seguinte grafo na linguagem DOT (Graphviz, 2022).

```
digraph ModeloLogico {
    utilizador;
    pais;
    tipo_viagem;
    tipo_vale;
    cidade -> pais;
    viagem -> utilizador;
    viagem -> tipo_viagem;
    viagem_cidade -> viagem;
    viagem_cidade -> cidade;
    publicacao -> viagem;
    publicacao -> utilizador;
    ficheiro_audio_visual -> publicacao;
    comentario -> utilizador;
    comentario -> publicacao;
    segue -> utilizador;
    vale_de_desconto -> utilizador;
    vale_de_desconto -> tipo_vale;
}
```

Quando renderizado graficamente, obtemos o seguinte resultado:

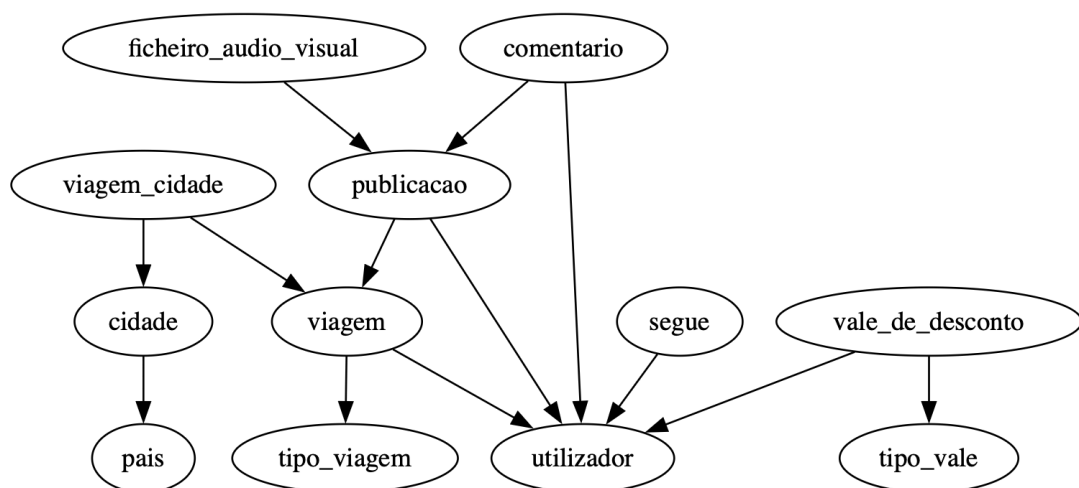


Figura 17 - Grafo do diagrama de dependências de criação da Base de Dados

Tal como no modelo lógico, cada atributo foi diretamente transposto para o modelo físico, mantendo nome e domínio. Assim como indicado na bibliografia de referência do MySQL, a instrução “CREATE TABLE” permite especificar cada tabela, os seus atributos, tipos de dados, restrições de integridade e chaves primárias ou estrangeiras.

Cada atributo é declarado indicando a sua designação e o respetivo domínio, que pode assumir diferentes tipos de dados, como: INT, VARCHAR ou DATE, bem como a possibilidade de conter valores nulos, expressa através das palavras-chave NULL e NOT NULL (Oracle, 2024a, p. 2709).

Seguindo a ordem definida anteriormente, começamos pela criação da tabela “**utilizador**”, por constituir a entidade base de múltiplas relações.

```
DROP TABLE IF EXISTS utilizador ;
CREATE TABLE IF NOT EXISTS utilizador (
    id_utilizador INT AUTO_INCREMENT,
    nome_utilizador VARCHAR(45) NOT NULL,
    palavra_passe VARCHAR(64) NOT NULL,
    email VARCHAR(45) NOT NULL,
    biografia VARCHAR(400),
    pontos INT NOT NULL,
    PRIMARY KEY (id_utilizador),
    UNIQUE (nome_utilizador),
    UNIQUE (email)
) ENGINE=InnoDB;
```

O atributo “id_utilizador” é definido como chave primária através de “PRIMARY KEY”, garantindo unicidade para integridade referencial, essencial no cumprimento das dependências entre entidades. A propriedade “AUTO_INCREMENT” está presente em “id” para respeitar a natureza de número sequencial deste atributo. Os atributos “nome_utilizador” e “email” são definidos com a restrição UNIQUE, assegurando que não existem duplicações destes valores no sistema.

Nesta e para todas as próximas tabelas, como não existe nenhum requisito que justifique o uso de um motor de armazenamento específico em prol de outro, será utilizada a definição por omissão, InnoDB.

Todas as tabelas que serão apresentadas seguem o mesmo padrão de criação explicado anteriormente, pelo que não serão apresentadas em detalhe.

A próxima tabela criada foi a tabela “**país**” que armazena a informação relativa aos países considerados no sistema. Cada país é identificado de forma única através do atributo “id_pais”, definido como chave primária. O atributo “nome_pais” encontra-se sujeito à restrição UNIQUE, assegurando que não existem registos duplicados de países.

```
CREATE TABLE pais (
    id_pais INT AUTO_INCREMENT,
    nome_pais VARCHAR(45) NOT NULL,
    PRIMARY KEY (id_pais),
    UNIQUE (nome_pais)
) ENGINE=InnoDB;
```

De seguida, foi criada a tabela “**tipo_viagem**”, responsável por armazenar os diferentes tipos de viagem existentes no sistema. Cada tipo de viagem é identificado de forma única através do atributo “id_tipo_viagem”, definido como chave primária. O atributo “nome_tipo” encontra-se sujeito à restrição UNIQUE, garantindo que não existem tipos de viagem duplicados no modelo.

```
DROP TABLE IF EXISTS tipo_viagem;
CREATE TABLE tipo_viagem (
    id_tipo_viagem INT AUTO_INCREMENT,
    nome_tipo VARCHAR(45) NOT NULL,
    PRIMARY KEY (id_tipo_viagem),
    UNIQUE (nome_tipo)
) ENGINE=InnoDB;
```

A próxima tabela , “**tipo_vale**”, foi criada para armazenar os diferentes tipos de vales de descontos disponíveis no sistema. Cada tipo de vale é identificado unicamente, através de “id_tipo_vale”, definido como chave primária. Os atributos “nome_tipo_vale”, “descricao_tipo” e “valor_pontos_tipo” permitem caracterizar cada tipo de vale, sendo o atributo “nome_tipo_vale” sujeito à restrição UNIQUE, garantindo que não existem tipos de vale duplicados no modelo. Adicionalmente, este atributo possui uma restrição de integridade (CHECK) que limita a entrada de dados apenas aos tipos: 'hotel', 'atividade' ou 'companhia aerea'."

```
CREATE TABLE tipo_vale (
    id_tipo_vale INT AUTO_INCREMENT,
    nome_tipo_vale VARCHAR(45) NOT NULL CHECK (nome_tipo_vale IN
    ('hotel', 'atividade', 'companhia aerea')),
    descricao_tipo VARCHAR(300) NOT NULL,
    valor_pontos_tipo INT NOT NULL,
    PRIMARY KEY (id_tipo_vale),
    UNIQUE (nome_tipo_vale)
) ENGINE=InnoDB;
```

A tabela “**cidade**” permite o registo das cidades associadas aos países existentes no sistema. Cada cidade é identificada de forma única através do atributo “id_cidade”, definido como chave primária. O atributo “id_pais” funciona como chave estrangeira, assegurando que cada cidade se encontra associada a um país previamente registado, garantindo a integridade referencial da hierarquia geográfica.

```
CREATE TABLE cidade (
    id_cidade INT AUTO_INCREMENT,
    nome_cidade VARCHAR(45) NOT NULL,
    id_pais INT NOT NULL,
    PRIMARY KEY (id_cidade),
    FOREIGN KEY (id_pais)
    REFERENCES pais(id_pais)
) ENGINE=InnoDB;
```

A próxima relação criada foi “**viagem**”, através da seguinte instrução SQL:

```
DROP TABLE IF EXISTS viagem ;
CREATE TABLE viagem (
    id_viagem INT AUTO_INCREMENT,
    data_partida DATE NOT NULL,
```

```

data_chegada DATE NOT NULL,
id_tipo_viagem INT NOT NULL,
id_utilizador INT NOT NULL,
PRIMARY KEY (id_viagem),
FOREIGN KEY (id_tipo_viagem)
    REFERENCES tipo_viagem(id_tipo_viagem),
FOREIGN KEY (id_utilizador)
    REFERENCES utilizador(id_utilizador),
) ENGINE=InnoDB;

```

O atributo “id_viagem” é definido como chave primária através da restrição PRIMARY KEY, garantindo a identificação única de cada registo na tabela. A propriedade AUTO_INCREMENT é aplicada a este atributo e permite a geração automática de valores sequenciais, adequados à sua função de identificador. Os atributos “id_tipo_viagem” e “id_utilizador” são definidos como chaves estrangeiras que referenciam respetivamente as tabelas “**tipo_viagem**” e “**utilizador**”. Estas restrições asseguram que cada viagem está associada a um tipo de viagem e a um utilizador previamente existente no sistema.

Durante operações de inserção ou atualização, o SGBD valida automaticamente a existência dos valores referenciados, rejeitando qualquer operação que viole a integridade referencial entre entidades.

A tabela “**viagem_cidade**” foi criada para implementar o relacionamento do tipo **N:M** entre as entidades “**viagem**” e “**cidade**”. A chave primária composta pelos atributos “id_viagem” e “id_cidade” garante a unicidade de cada associação. Ambos os atributos funcionam simultaneamente como chaves estrangeiras, assegurando que apenas associações entre viagens e cidades existentes podem ser registadas no sistema.

```

CREATE TABLE viagem_cidade (
    id_viagem INT NOT NULL,
    id_cidade INT NOT NULL,
    PRIMARY KEY (id_viagem, id_cidade),
    FOREIGN KEY (id_viagem) REFERENCES viagem(id_viagem),
    FOREIGN KEY (id_cidade) REFERENCES cidade(id_cidade)
);

```

A criação de “**publicacao**” é semelhante às anteriores em diversos aspetos, como pode ser visto pela sua instrução de criação abaixo. Destacamos as múltiplas dependências de “publicação” referenciando simultaneamente “utilizador” e “viagem”.

```

DROP TABLE IF EXISTS publicacao ;
CREATE TABLE publicacao (
    id_publicacao INT AUTO_INCREMENT,
    data DATE NOT NULL,
    descricao VARCHAR(500),
    id_viagem INT NOT NULL,
    id_utilizador INT NOT NULL,
    PRIMARY KEY (id_publicacao),
    UNIQUE (id_viagem),
    FOREIGN KEY (id_viagem) REFERENCES viagem(id_viagem),
    FOREIGN KEY (id_utilizador) REFERENCES utilizador(id_utilizador)
)

```

);

Aqui, surgem duas chaves estrangeiras dentro da mesma relação, reforçando o facto de que cada publicação é criada por um utilizador e cada publicação está associada a uma viagem específica. O SGBD assegura que nenhum registo pode ser inserido com valores inexistentes nestas duas colunas, garantindo a coerência entre entidades.

A tabela “**ficheiro_audio_visual**” permite o armazenamento dos ficheiros multimédia associados às publicações da plataforma:

```
DROP TABLE IF EXISTS ficheiro_audio_visual ;
CREATE TABLE ficheiro_audio_visual (
    id_ficheiro INT AUTO_INCREMENT,
    ficheiro_audiovisual VARCHAR(400) NOT NULL,
    id_publicacao INT NOT NULL,
    PRIMARY KEY (id_ficheiro),
    FOREIGN KEY (id_publicacao)
        REFERENCES publicacao(id_publicacao)
) ENGINE=InnoDB;
```

Aqui cada ficheiro é identificado de forma única através do atributo “id_ficheiro”, definido como chave primária e com a propriedade AUTO_INCREMENT, que garante a geração automática de identificadores sequenciais. O atributo “id_publicacao” funciona como chave estrangeira e referencia a tabela “**publicacao**”, assegurando que cada ficheiro áudio-visual encontra-se obrigatoriamente associado a uma publicação existente. Esta estrutura permite que uma mesma publicação possa ter vários ficheiros associados, eliminando a necessidade de atributos multivalorados.

A tabela “**comentario**” permite o registo dos comentários efetuados pelos utilizadores nas publicações. Cada comentário é identificado de forma única através do atributo “id_comentario”, definido como chave primária. Os atributos “id_utilizador” e “id_publicacao” funcionam como chaves estrangeiras, garantindo que cada comentário está associado a um utilizador e a uma publicação existentes, assegurando a integridade referencial do modelo.

```
CREATE TABLE comentario (
    id_comentario INT AUTO_INCREMENT,
    comentario VARCHAR(500),
    id_utilizador INT NOT NULL,
    id_publicacao INT NOT NULL,
    PRIMARY KEY (id_comentario),
    FOREIGN KEY (id_utilizador)
        REFERENCES utilizador(id_utilizador),
    FOREIGN KEY (id_publicacao)
        REFERENCES publicacao(id_publicacao)
) ENGINE=InnoDB;
```

A relação criada “**segue**” representa o relacionamento recursivo entre utilizadores, configurando o mecanismo de seguidores:

```
DROP TABLE IF EXISTS segue ;
CREATE TABLE segue (
    id_utilizador INT NOT NULL,
```

```

        id_seguidor INT NOT NULL,
        PRIMARY KEY (id_utilizador, id_seguidor),
        FOREIGN KEY (id_utilizador)
            REFERENCES utilizador(id_utilizador),
        FOREIGN KEY (id_seguidor)
            REFERENCES utilizador(id_utilizador)
    ) ENGINE=InnoDB;

```

Esta relação apresenta uma chave primária composta e trata-se de um caso em que ambos os atributos são simultaneamente chaves estrangeiras que referenciam a mesma tabela, implementando um relacionamento recursivo. O SGBD valida que tanto o seguidor como o seguido existem previamente na tabela **utilizador**, impedindo inconsistências como seguidores inexistentes ou ligações inválidas.

De seguida, foi criada a tabela “**vale_de_desconto**”, responsável pelo registo dos vales atribuídos aos utilizadores. Esta tabela apresenta uma estrutura simples, que contém chaves estrangeiras que asseguram a associação de um vale a um utilizador e a um tipo de vale previamente definidos, garantindo a integridade referencial do modelo..

```

DROP TABLE IF EXISTS vale_de_desconto ;
CREATE TABLE vale_de_desconto (
    id_vale_de_desconto INT AUTO_INCREMENT,
    usado BOOLEAN NOT NULL,
    id_tipo_vale INT NOT NULL,
    id_utilizador INT NOT NULL,
    PRIMARY KEY (id_vale_de_desconto),
    FOREIGN KEY (id_tipo_vale)
        REFERENCES tipo_vale(id_tipo_vale),
    FOREIGN KEY (id_utilizador)
        REFERENCES utilizador(id_utilizador)
) ENGINE=InnoDB;

```

5.2. Criação de utilizadores da base de dados

Após a implementação física do esquema da base de dados da plataforma NOMÁLIA, tornou-se necessário definir os utilizadores do SGBD, uma etapa fundamental para garantir a segurança, o controlo de acessos e garantir que apenas entidades autorizadas possam executar operações críticas no ambiente da base de dados.

Tal como indicado nos requisitos recolhidos, diferentes intervenientes da plataforma possuem necessidades distintas de acesso e de manipulação de dados. Logo, em vez de se atribuírem privilégios individualmente a cada utilizador, criaram-se *roles* (papéis), conjuntos de privilégios, que podem ser associados a perfis funcionais específicos. Para o presente projeto, foram definidos dois papéis principais:

- **Administrador:** responsável pela gestão de conteúdos sensíveis da plataforma, como a remoção de viagens, publicações e comentários, bem como pela intervenção em situações excepcionais;
- **Utilizador:** responsável pela execução das operações essenciais ao funcionamento da plataforma, como a criação de publicações, o registo de viagens, a interação social e a gestão do próprio perfil, sem permissões para alterar a estrutura da base de dados.

Inicialmente, procedeu-se à remoção dos *roles*, caso já existissem, garantindo que a configuração final corresponde exatamente ao comportamento especificado:

```
DROP ROLE IF EXISTS 'administrador', 'utilizador';
CREATE ROLE 'administrador', 'utilizador';
```

O uso de “DROP ROLE IF EXISTS” garante que, caso estes perfis já tenham sido previamente criados, podem ser removidos sem causar erros. Esta prática assegura que a configuração final coincide exatamente com o comportamento especificado, evitando acumulação de versões anteriores. “CREATE ROLE” define os *roles* que serão posteriormente associados aos utilizadores do SGBD.

Após a definição dos papéis, foram criadas as contas de utilizador no SGBD, às quais seriam posteriormente atribuídos os respetivos *roles*:

```
DROP USER IF EXISTS 'admin1'@'localhost', 'user1'@'localhost';

CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'Admin123#';

CREATE USER 'user1'@'localhost' IDENTIFIED BY 'User123#';
```

De seguida, procedeu-se à associação dos papéis aos utilizadores e à definição do papel por defeito de cada conta:

```
GRANT 'administrador' TO 'admin1'@'localhost';

SET DEFAULT ROLE 'administrador' TO 'admin1'@'localhost';

GRANT 'utilizador' TO 'user1'@'localhost';

SET DEFAULT ROLE 'utilizador' TO 'user1'@'localhost';
```


Desta forma, a conta **admin1** passa a assumir automaticamente todos os privilégios associados ao papel administrador, enquanto a conta **user1** herda apenas os privilégios definidos para o papel utilizador.

Após a criação dos perfis, passaram-se a atribuir as permissões específicas, de acordo com os Requisitos de Manipulação e de Controlo identificados pela equipa:

Requisito de Manipulação 13 – Seguir utilizadores, deixar de seguir e visualizar seguidores

Isto exige que o papel “utilizador” tenha permissões de inserção, visualização e eliminação na tabela “segue”.

```
GRANT INSERT, SELECT, DELETE ON segue TO 'utilizador';
```

Requisito de Manipulação 14 e 15 – Criar, editar e visualizar publicações e viagens.

Todos os utilizadores podem criar (INSERT), atualizar (UPDATE) e visualizar publicações (SELECT):

```
GRANT SELECT, UPDATE ON viagem TO 'utilizador';
```

```
GRANT SELECT, UPDATE ON publicacao TO 'utilizador';
```

```
GRANT EXECUTE ON PROCEDURE RegistrarViagemPublicacao TO 'utilizador';
```

Requisitos de Manipulação 16 e 17 – Inserção, edição e visualização de comentários

Os utilizadores podem interagir com publicações através da criação (INSERT), edição (UPDATE) e visualização (SELECT) de comentários:

```
GRANT SELECT, UPDATE ON comentario TO 'utilizador';
```

```
GRANT EXECUTE ON PROCEDURE CriarComentario TO 'utilizador';
```

Requisito de manipulação 18 – Inserir, editar e visualizar dados do perfil

O utilizador pode atualizar (UPDATE) e visualizar (SELECT) dados do perfil e atributos do próprio registo na tabela “utilizador”. O controlo de acesso ao nível dos registos não é suportado nativamente pelo MySQL através da atribuição de privilégios (GRANT). Assim, embora o MySQL permita a atribuição de permissões de atualização ao nível da tabela, optou-se por não conceder diretamente privilégios UPDATE sobre a tabela “utilizador”, assegurando que a alteração dos dados do perfil é realizada exclusivamente através de um procedimento armazenado controlado. A garantia de que cada utilizador apenas pode alterar os seus próprios dados é assegurada ao nível da lógica do sistema, através da utilização de

procedimentos armazenados invocados pela aplicação, que recebem explicitamente o identificador do utilizador autenticado e restringem a atualização ao respetivo registo.

```
GRANT SELECT (id_utilizador, nome_utilizador, email,
biografia,pontos) ON utilizador TO 'utilizador';
```

```
GRANT EXECUTE ON PROCEDURE AtualizarPerfil TO 'utilizador';
```

Requisito de manipulação 19 – Trocar pontos por vales de desconto

O utilizador pode criar registos de troca e consultar os vales disponíveis, não podendo alterá-los ou removê-los.

```
GRANT SELECT ON tipo_vale TO 'utilizador';
```

```
GRANT SELECT ON vale_de_desconto TO 'utilizador';
```

```
GRANT EXECUTE ON PROCEDURE TrocarValeDesconto TO 'utilizador';
```

```
GRANT EXECUTE ON FUNCTION PontosDisponiveis TO 'utilizador';
```

Requisito de manipulação 20 e Requisito de Controlo 26 – O administrador pode visualizar e remover viagens, as publicações a elas associadas e os comentários

Este requisito aplica-se exclusivamente ao papel “administrador”, que necessita de permissões de leitura e remoção sobre estas entidades:

```
GRANT SELECT, DELETE ON viagem TO 'administrador';
```

```
GRANT SELECT, DELETE ON publicacao TO 'administrador';
```

```
GRANT SELECT, DELETE ON comentario TO 'administrador';
```

```
GRANT SELECT, DELETE ON viagem_cidade TO 'administrador';
```

```
GRANT SELECT, DELETE ON ficheiro_audio_visual TO 'administrador';
```

```
GRANT      SELECT      (id_utilizador,      nome_utilizador,      email,
biografia,pontos) ON nomalia.utilizador
```

```
TO administrador;
```

Relativamente aos Requisitos de Controlo identificados, importa salientar que nem todos são implementados exclusivamente através de permissões no SGBD. A autenticação dos utilizadores é assegurada através da criação de contas no sistema de base de dados, protegidas por palavra-passe, garantindo que apenas utilizadores autenticados podem aceder à plataforma (Requisito 22). A unicidade do nome de utilizador e do endereço de correio eletrónico é garantida através de restrições de integridade (UNIQUE) definidas ao nível do esquema da base de dados (Requisito 23). Os requisitos relacionados com a robustez e

confidencialidade das palavras-passe (Requisitos 24 e 25) são assegurados ao nível do SGBD através da utilização de um gatilho que valida os critérios mínimos de segurança da palavra-passe e procede ao seu armazenamento de forma ilegível, recorrendo à função de hashing SHA2 que garante que nenhuma palavra-passe é armazenada em formato legível. A atribuição de pontos aos utilizadores de acordo com a sua atividade na plataforma (Requisito 21) corresponde à lógica de negócio específica do sistema, sendo implementada através de mecanismos próprios, como funções ou procedimentos armazenados, de forma a assegurar o cálculo consistente e centralizado do saldo dos utilizadores. Por fim, o Requisito de Controlo 26 é concretizado através da definição de um papel específico de administrador, ao qual são atribuídas permissões de visualização e remoção de conteúdos. O acesso à tabela “utilizador” é restringido ao nível dos atributos, não sendo concedidas permissões de leitura sobre o atributo “palavra_passe”, para que dessa forma o administrador possa gerir conteúdos e contas sem acesso a dados privados dos utilizadores, assegurando a separação de responsabilidades e a proteção da informação sensível.

De forma geral, a criação dos utilizadores no SGBD materializa o controlo rigoroso de acessos, reforçando medidas de segurança e contribuindo para a fiabilidade da plataforma. A implementação destas restrições permite isolar responsabilidades, evitar alterações indevidas e garantir que a manipulação dos dados segue as regras definidas no modelo lógico, assegurando coerência e consistência em todas as operações realizadas sobre a base de dados.

5.3. Povoamento da base de dados

Após a definição da estrutura física da Base de Dados e da criação dos respetivos mecanismos de controlo de acesso, tornou-se necessário proceder ao povoamento inicial da plataforma NOMÁLIA, permitindo que, aquando da execução de testes funcionais e de integração, a Base de Dados contenha dados para validar o comportamento das funcionalidades previstas nos requisitos de manipulação e de controlo.

De entre as várias formas de povoar uma Base de Dados, a equipa de desenvolvimento implementou o povoamento manual através de instruções SQL. Consiste em escrever diretamente num *script* SQL todas as instruções INSERT, criando manualmente cada registo das tabelas. Esta abordagem permite que exista um controlo rigoroso sobre os dados introduzidos, a fim de assegurar a coerência com o modelo lógico e validação das relações entre entidades.

A equipa de desenvolvimento seguiu uma ordem alinhada com o grafo de dependências apresentado anteriormente, iniciando o povoamento pelas tabelas que não dependem de chaves estrangeiras, garantindo que todas as relações necessárias já existiam aquando da inserção de registos em tabelas que dependem destas referências.

O povoamento iniciou-se com tabelas de domínio que não dependem de outras entidades: `tipo_viagem`, `pais` e `tipo_vale`. Estas tabelas suportam valores reutilizados por outras relações.

```
INSERT INTO tipo_viagem (id_tipo_viagem, nome_tipo) VALUES
    (1, 'Lazer'),
    (2, 'Profissional'),
    (3, 'Cultural'),
    (4, 'Gastronómica');
```

```
INSERT INTO pais (id_pais, nome_pais) VALUES
    (1, 'Portugal'),
    (2, 'Espanha'),
    (3, 'Islândia'),
    (4, 'Itália'),
    (5, 'França'),
    (6, 'Estados Unidos da América'),
    (7, 'Japão'),
    (8, 'Brasil');
```

```
INSERT INTO tipo_vale (id_tipo_vale, nome_tipo_vale, descricao_tipo,
valor_pontos_tipo) VALUES
    (1, 'hotel', 'Desconto de 10% em parceiros.', 100),
    (2, 'atividade', 'Desconto de 15% em parceiros.', 150);
```

De seguida, foram inseridas cidades, dependentes de pais (através de “id_pais”).

```
INSERT INTO cidade (id_cidade, nome_cidade, id_pais) VALUES
```

```
-- Portugal
(1, 'Lisboa', 1),
(3, 'Porto', 1),
(5, 'Braga', 1),

-- Espanha
(6, 'Barcelona', 2),
(7, 'Madrid', 2),
(8, 'Sevilha', 2),

-- Islândia
(10, 'Reiquiavique', 3),
(11, 'Vik', 3),

-- Itália
(13, 'Roma', 4),
(15, 'Florença', 4),
(16, 'Veneza', 4),

-- França
(17, 'Paris', 5),
(18, 'Lyon', 5),

-- EUA
(19, 'Nova Iorque', 6),
(20, 'Los Angeles', 6),
(21, 'São Francisco', 6),

-- Japão
(22, 'Tóquio', 7),
(23, 'Quioto', 7),
(24, 'Osaca', 7),

-- Brasil
(25, 'Rio de Janeiro', 8),
(26, 'São Paulo', 8),
(27, 'Salvador', 8);
```

Após a criação dos valores de referência, procedeu-se ao povoamento da tabela “utilizador”, que não possui chaves estrangeiras. Foram criados utilizadores com perfis distintos, adequados aos cenários de teste: utilizadores comuns e um utilizador com permissões de administração da plataforma. Note-se que, para efeitos de testes, são utilizados valores exemplificativos no atributo “palavra_passe”; num contexto real, a palavra-passe não deve ser armazenada em formato legível.

```

INSERT INTO utilizador (id_utilizador, nome_utilizador, palavra_passe,
email, biografia, pontos) VALUES
(1, 'soraia_faria', 'PasswordMuitoSegura#123', 'sfaria@gmail.com',
'Apaixonada por viagens.', 120),
(2, 'matheus_azevedo', 'PasswordMuitoSegura#456',
'matheus@gmail.com', 'Explorador urbano.', 80),
(3, 'miguel_santos', 'PasswordMuitoSegura#789',
'miguel@gmail.com', 'Viajar é viver.', 60),
(4, 'joana_mendes', 'PasswordUltraSegura#ABC', 'joana@gmail.com',
'Cultura e gastronomia.', 95),
(5, 'carolina_ribeiro', 'PasswordMuitoSegura#477',
'carolina.ribeiro@gmail.com', 'Amante de viagens culturais e
gastronómicas', 70),
(6, 'admin_nomalia', 'AdminPasswordSuperSafe#1',
'admin@nomalia.com', 'Administrador.', 0);

```

Após os utilizadores terem sido registados, procedeu-se ao povoamento da tabela “viagem”. Esta relação apresenta uma chave estrangeira que referencia “utilizador”, pelo que apenas foi possível inserir dados após a conclusão do passo anterior.

```

INSERT INTO viagem (id_viagem, data_partida, data_chegada,
id_tipo_viagem, id_utilizador) VALUES
(1, '2024-02-10', '2024-02-18', 3, 1),
(2, '2023-09-10', '2023-09-15', 1, 2),
(3, '2024-03-01', '2024-03-10', 3, 4),
(4, '2024-04-05', '2024-04-20', 2, 1),
(5, '2024-05-01', '2024-05-12', 1, 3),
(6, '2024-06-10', '2024-06-15', 4, 5),
(7, '2024-07-01', '2024-07-07', 1, 2),
(8, '2024-08-10', '2024-08-25', 1, 3);

```

Uma vez que uma viagem pode estar associada a múltiplas cidades, e vice-versa (relação N:M), procedeu-se ao povoamento da tabela de associação “viagem_cidade”.

```

INSERT INTO viagem_cidade (id_viagem, id_cidade) VALUES
(1, 10),
(1, 11),

(2, 6),
(2, 8),

(3, 12),
(3, 15),
(3, 16),

(4, 19),
(4, 20),
(4, 21),

(5, 22),
(5, 23),

```

```

(5, 24),
(6, 17),
(6, 18),

(7, 1),
(7, 5),

(8, 25),
(8, 27);

```

De seguida, inseriram-se registos na tabela “publicacao”, a qual associa cada publicação a uma viagem e ao utilizador respetivo. Adicionalmente, existe a restrição UNIQUE (“id_viagem”), garantindo que cada viagem tem no máximo uma publicação associada. Assim, foi inserida uma publicação por viagem.

```

INSERT INTO publicacao (id_publicacao, data, descricao, id_viagem,
id_utilizador) VALUES
(1, '2024-02-15', 'As paisagens da Islândia são irreais.', 1, 1),
(2, '2023-09-14', 'Barcelona é cheia de energia.', 2, 2),
(3, '2024-03-06', 'Roma é história viva.', 3, 4),
(4, '2024-04-12', 'Viagem profissional intensa.', 4, 1),
(5, '2024-05-07', 'Japão superou expectativas.', 5, 3),
(6, '2024-06-13', 'Gastronomia francesa incrível.', 6, 5),
(7, '2024-07-04', 'Portugal é sempre especial.', 7, 2),
(8, '2024-08-18', 'Brasil é pura alegria.', 8, 3);

```

Como cada publicação pode possuir múltiplos ficheiros, foram associados ficheiros audiovisuais a publicações previamente criadas, respeitando a chave estrangeira “id_publicacao”.

```

INSERT INTO ficheiro_audio_visual (id_ficheiro, ficheiro_audiovisual,
id_publicacao)
VALUES
(1, 'media/foto1.jpg', 1),
(2, 'media/video1.mp4', 1),
(3, 'media/foto2.jpg', 2),
(4, 'media/lisboa1.jpg', 3),
(5, 'media/trabalho1.jpg', 4),
(6, 'media/neve1.jpg', 5),
(7, 'media/neve2.mp4', 5),
(8, 'media/paris.jpg', 6),
(9, 'media/roma.jpg', 7),
(10, 'media/japao.mp4', 8);

```

A tabela “comentario” regista comentários efetuados por utilizadores em publicações existentes, dependendo de “utilizador” e “publicacao”.

```

INSERT INTO comentario (id_comentario, comentario, id_utilizador,
id_publicacao) VALUES
    (1, 'Que lugar incrível!', 2, 1),
    (2, 'Está na minha lista!', 3, 1),
    (3, 'Barcelona nunca falha.', 1, 2),
    (4, 'Roma é eterna.', 2, 3),
    (5, 'Boa sorte na viagem!', 4, 4),
    (6, 'Japão parece mágico.', 1, 5),
    (7, 'Comida francesa é única.', 3, 6),
    (8, 'Portugal é casa.', 5, 7);

```

Para testar a componente social da plataforma, foram criadas relações de seguimento entre utilizadores. A tabela “segue” modela uma relação recursiva N:M em “utilizador”, através de duas chaves estrangeiras.

```

INSERT INTO segue (id_utilizador, id_seguidor)
VALUES
    (1, 2),
    (1, 3),
    (2, 1),
    (3, 4),
    (4, 1),
    (5, 2);

```

Por fim, procedeu-se à criação de vales de desconto associados a utilizadores e a um tipo de vale (“tipo_vale”). O atributo “usado” permite simular vales disponíveis e já utilizados.

```

INSERT INTO vale_de_desconto (id_vale_de_desconto, usado, id_tipo_vale,
id_utilizador)
VALUES
    (1, FALSE, 1, 1),
    (2, TRUE, 1, 2),
    (3, FALSE, 2, 4);

```

O povoamento foi realizado respeitando as dependências entre tabelas e as restrições definidas no modelo físico, nomeadamente as chaves estrangeiras e a restrição de unicidade UNIQUE (“id_viagem”) na tabela “publicacao”. O povoamento revela-se, assim, uma etapa crucial para validar o funcionamento do Modelo Físico.

5.4. Cálculo do espaço da base de dados (inicial e taxa de crescimento anual)

O cálculo do espaço ocupado pela Base de Dados permite estimar antecipadamente os recursos necessários para armazenar todos os dados gerados pela plataforma, garantindo que o sistema é dimensionado de forma adequada, tanto para o uso atual, como para o crescimento futuro. Esta estimativa é realizada com base no tamanho expectável de cada atributo, no número de tuplos previstos para cada relação e no custo adicional introduzido pelos índices e pelas chaves primárias. Para efeitos de cálculo, foram considerados os tamanhos de armazenamento dos diferentes tipos de dados utilizados, de acordo com a documentação oficial do sistema de gestão de bases de dados MySQL da Oracle (Oracle, 2024).

Tipo de Dados	Tamanho (octetos)	Fonte
INT	4	Oracle. <i>MySQL 8.0 Reference Manual</i> . Oracle Corporation, 2024
DATE	3	Oracle. <i>MySQL 8.0 Reference Manual</i> . Oracle Corporation, 2024
VARCHAR (N), N ≤ 64	1 + N	Oracle. <i>MySQL 8.0 Reference Manual</i> . Oracle Corporation, 2024
VARCHAR (N), N >= 65	2 + N	Oracle. <i>MySQL 8.0 Reference Manual</i> . Oracle Corporation, 2024
TINYINT	1	Oracle. <i>MySQL 8.0 Reference Manual</i> . Oracle Corporation, 2024

Tabela 19 - Tipos de dados e o espaço consumido pelo InnoDB

Procedemos à análise individual das tabelas definidas:

“utilizador”

Atributo	Tipo de dados	Ocupação
id_utilizador (PK)	INT	4 B
nome_utilizador	VARCHAR(45)	≤ 46 B
palavra_passe	VARCHAR(64)	≤ 65 B
email	VARCHAR(45)	≤ 46 B
biografia	VARCHAR(400)	≤ 402 B
pontos	INT	4 B

Tabela 20 - Especificação dos domínios e ocupação dos atributos da entidade “utilizador”.

“viagem”

Atributo	Tipo de dados	Ocupação
id_viagem (PK)	INT	4 B
id_tipo_viagem (FK)	INT	4 B
data_chegada	DATE	3 B
data_partida	DATE	3 B
id_utilizador (FK)	INT	4 B

Tabela 21 - Especificação dos domínios e ocupação dos atributos da entidade “viagem”

“tipo_vigem”

Atributo	Tipo de dados	Ocupação
id_tipo_viagem (PK)	INT	4 B
nome_tipo	VARCHAR (45)	≤ 46 B

Tabela 22 - Especificação dos domínios e ocupação dos atributos da entidade “viagem”

“publicacao”

Atributo	Tipo de dados	Ocupação
id_publicacao (PK)	INT	4 B
data	DATE	3 B
descricao	VARCHAR(500)	≤ 502 B
id_viagem (FK)	INT	4 B
id_utilizador (FK)	INT	4 B

Tabela 23 - Especificação dos domínios e ocupação dos atributos da entidade “publicacao”

“vale_de_desconto”

Atributo	Tipo de dados	Ocupação
id_vale_de_desconto (PK)	INT	4 B
usado	TINYINT	1 B
id_tipo_vale (FK)	INT	4 B
id_utilizador (FK)	INT	4 B

Tabela 24 - Especificação dos domínios e ocupação dos atributos da entidade “vale_de_desconto”

“tipo_vale”

Atributo	Tipo de dados	Ocupação
id_vale_desconto (PK)	INT	4 B
nome_tipo_vale	VARCHAR(45)	≤ 46 B
descricao_tipo	VARCHAR(300)	≤ 302 B
valor_pontos_tipo	INT	4 B

Tabela 25 - Especificação dos domínios e ocupação dos atributos da entidade “tipo_vale”

“comentario”

Atributo	Tipo de dados	Ocupação
id_comentario (PK)	INT	4 B
comentario	VARCHAR(45)	≤ 46 B

id_publicacao (FK)	INT	4 B
id_utilizador (FK)	INT	4 B

Tabela 26 - Especificação dos domínios e ocupação dos atributos da entidade “comentario”

“ficheiros_audio_visual”

Atributo	Tipo de dados	Ocupação
id_ficheiro (PK)	INT	4 B
ficheiro_audiovisual	VARCHAR(400)	≤ 402 B
id_publicacao (FK)	INT	4 B

Tabela 27 - Especificação dos domínios e ocupação dos atributos da entidade “ficheiros_audio_visual”

“cidade”

Atributo	Tipo de dados	Ocupação
id_cidade (PK)	INT	4 B
nome_cidade	VARCHAR(45)	≤ 46 B
id_pais (FK)	INT	4 B

Tabela 28 - Especificação dos domínios e ocupação dos atributos da entidade “cidade”

“pais”

Atributo	Tipo de dados	Ocupação
id_pais (PK)	INT	4 B
nome_pais	VARCHAR(45)	≤ 46 B

Tabela 29 - Especificação dos domínios e ocupação dos atributos da entidade “pais”

“segue”

Atributo	Tipo de dados	Ocupação
id_seguidor (FK)	INT	4 B
id_utilizador (FK)	INT	4 B

Tabela 30 - Especificação dos domínios e ocupação dos atributos da relação “segue”

“viagem_cidade”

Atributo	Tipo de dados	Ocupação
id_viagem (FK)	INT	4 B
id_cidade (FK)	INT	4 B

Tabela 31 - Especificação dos domínios e ocupação dos atributos da relação “viagem_cidade”

Com base no tamanho estimado de cada registo e no número de registos por relação, é possível calcular uma aproximação ao tamanho da base de dados. Considerando os dados utilizados no povoamento da mesma, apresenta-se, de seguida, uma tabela com os resultados obtidos. Importa salientar que estes valores não correspondem ao tamanho real da BD, mas sim à dimensão que esta teria caso mantivesse o mesmo número de registos em cada relação.

Relação/Entidade	Tamanho por registo	Nº previsto de resgistos	Espaço Total
segue	8 B	500	4 000 B
viagem_cidade	8 B	200	1 600 B
tipo_viagem	50 B	10	500 B
utilizador	567 B	100	56 700 B
viagem	18 B	200	3 600 B
publicacao	517 B	400	206 800 B
vale_de_desconto	13 B	100	1 300 B
tipo_vale	356 B	10	3 560 B
comentario	58 B	500	29 000 B
ficheiro_audiovisual	410 B	200	82 000 B
cidade	54 B	50	2 700 B
pais	50 B	20	1 000 B
Total estimado	--	--	392 760 B

Tabela 32 - Tamanho ocupado por cada relação na BD NOMÁLIA (usando o tamanho teórico dos registos)

O espaço total de cada relação foi determinado pela multiplicação entre o tamanho por registo e o número previsto de registos dessa relação. Desta forma, obtém-se uma estimativa da ocupação em disco de cada tabela, permitindo avaliar o impacto global no armazenamento da base de dados e apoiar o dimensionamento dos requisitos de *hardware*.

Com base nos cálculos teóricos apresentados, a base de dados ocuparia cerca de 392 760 B, ou seja, aproximadamente 384 KB. Para garantir uma operação confortável da plataforma NOMÁLIA numa fase inicial, recomenda-se prever uma capacidade superior, de modo a acomodar o crescimento futuro do sistema de gestão da base de dados.

Podemos também analisar como este aumentará ao longo do tempo, considerando diferentes taxas de crescimento anuais possíveis (os valores foram arredondados às unidades), como se verifica na tabela abaixo:

Taxa de crescimento anual	Ano 0	Ano 1	Ano 2	Ano 3	Ano 4
5%	384 KB	403 KB	423 KB	444 KB	466 KB
10%	384 KB	422 KB	464 KB	510 KB	561 KB
15%	384 KB	442 KB	508 KB	584 KB	672 KB
20%	384 KB	461 KB	553 KB	664 KB	797 KB
30%	384 KB	499 KB	649 KB	844 KB	1 097 KB

Tabela 33 - Evolução do tamanho da BD NOMÁLIA (tamanho teórico dos registos)

5.5. Definição e caracterização de vistas de utilização em SQL

Na plataforma NOMÁLIA, as vistas em SQL podem ser utilizadas como um mecanismo complementar de controlo do acesso à informação, permitindo definir explicitamente quais os dados que podem ser consultados pela aplicação e pelos utilizadores, sem expor diretamente as tabelas base da base de dados.

A equipa decidiu não implementar vistas, uma vez que o controlo do acesso aos dados sensíveis é assegurado principalmente através da definição de *roles* e da atribuição de permissões ao nível das tabelas e dos atributos porém, foi considerada a utilização de uma vista destinada à disponibilização de informação pública sobre os utilizadores da plataforma, excluindo atributos confidenciais, como a “palavra_passe”.

A título ilustrativo, poderia ser definida a vista `vwUtilizadoresPublicos`, que expõe apenas os dados necessários para a apresentação dos perfis de utilizador:

```
CREATE VIEW vwUtilizadoresPublicos AS

SELECT

    id_utilizador,

    nome_utilizador,

    email,

    biografia,

    pontos

FROM utilizador;
```

Esta vista assegura que apenas os dados necessários para a apresentação dos perfis de utilizador são disponibilizados. No entanto, no contexto de NOMÁLIA, esta separação é igualmente garantida através do controlo de permissões ao nível dos atributos, não sendo estritamente necessária a utilização de vistas para esse fim.

Para além das vistas consideradas, a arquitetura da base de dados foi concebida de forma a permitir a introdução de novas vistas no futuro, caso surjam requisitos adicionais. Um exemplo seria a criação de uma vista que devolvesse o número total de viagens registadas na plataforma, caso se pretendesse disponibilizar esta informação de forma pública.

```
CREATE VIEW vwNumeroViagens AS

SELECT COUNT(id_viagem) AS total_viagens

FROM viagem;
```

Desta forma, as vistas assumem um papel complementar e opcional na arquitetura da base de dados, podendo ser utilizadas em cenários específicos, embora o controlo de acessos da plataforma seja assegurado principalmente através da definição de papéis, permissões e mecanismos de segurança implementados no SGBD.

5.6. Tradução das interrogações do utilizador para SQL

Após a validação em álgebra relacional, as interrogações definidas nos requisitos de manipulação foram traduzidas para SQL. Esta conversão é direta, dado que ambas as linguagens partilham a mesma semântica, diferenciando-se apenas pela sintaxe. Assim, cada consulta apresentada nesta secção mantém a mesma estrutura conceptual utilizada na fase anterior, preservando a correspondência entre projeções, seleções, junções e operações de modificação (inserção, atualização e eliminação).

As interrogações foram apresentadas seguindo a ordem definida no levantamento de requisitos, demonstrando de que forma cada expressão lógica se traduz em código SQL concreto.

Para facilitar a análise, cada interrogação é acompanhada das instruções SQL correspondentes, permitindo observar, de forma direta, a passagem da lógica conceptual para a implementação física no SGBD MySQL. Os identificadores representados por X ou [ID...] correspondem a parâmetros fornecidos pela aplicação no momento da execução, sendo utilizados apenas para fins ilustrativos na tradução das interrogações para SQL.

Tradução da interrogação 1 - Gestão de Seguidores/Seguidos

Esta interrogação permite identificar as relações de ligação entre utilizadores da plataforma, recorrendo a associações entre registos com a tabela “utilizador”. A tradução para SQL utiliza uma seleção condicionada pelo identificador do utilizador e projeta os nomes correspondentes, permitindo visualizar quem um utilizador segue e quem o segue, suportando assim a funcionalidade de rede social.

A instrução SQL realiza uma operação de junção (`INNER JOIN`) entre as tabelas “segue” e “utilizador”, designada como “u_seguido”, para obter os nomes dos utilizadores seguidos pelo utilizador com identificador X. A condição de junção associa o campo “id_utilizador” ao identificador do utilizador, enquanto a cláusula `WHERE` restringe os resultados ao seguidor em questão.

```
SELECT seguido.nome_utilizador AS NomeSeguido
FROM segue s
INNER JOIN utilizador seguido ON s.id_utilizador = seguido.id_utilizador
WHERE s.id_seguidor = X;
```

De forma complementar, esta junção é utilizada para identificar os seguidores do utilizador X, em que a tabela “utilizador” é designada como “u_seguidor” e a condição de junção associa o campo “id_seguidor” ao identificador do utilizador, sendo os resultados filtrados pela cláusula `WHERE` para o utilizador seguido.

```
SELECT u_seguidor.nome_utilizador AS NomeSeguidor
FROM segue s
INNER JOIN utilizador u_seguidor ON s.id_seguidor =
u_seguidor.id_utilizador
WHERE s.id_utilizador = X;
```

Tradução da interrogação 2 e 3 - Gestão de Publicações e Viagens

Para a gestão de viagens e respetivas memórias, o sistema utiliza uma abordagem otimizada através de **Procedimentos Armazenados (Stored Procedures)**. Em vez de executar múltiplas instruções `INSERT` isoladas, o que poderia comprometer a integridade dos

dados (ex: criar uma viagem sem a sua publicação), foi implementado o procedimento `RegistrarViagemPublicacao`.

Esta abordagem garante que a criação de uma experiência na plataforma NOMÁLIA seja tratada como uma operação única. O procedimento recebe como parâmetros o identificador do utilizador, o tipo de viagem, as datas de partida e chegada, e o conteúdo da publicação.

```
CALL RegistrarViagemPublicacao([ID_UTILIZADOR], [ID_TIPO],  
'[DATA_PARTIDA]', '[DATA_CHEGADA]', '[DESCRICAO]');
```

Tradução da Interrogação 4 - Consulta de Comentários

Na interrogação 4 utiliza-se uma junção entre as tabelas “comentario”, “utilizador” e “publicacao”, permitindo apresentar simultaneamente o autor, o conteúdo e a publicação comentada. Esta interrogação lista todos os comentários realizados em publicações, mostrando o nome do utilizador, a descrição da publicação e o respetivo comentário.

A operação combina três tabelas: “utilizador”, “comentario” e “publicacao”. A primeira junção (JOIN) associa o utilizador ao comentário através da chave de utilizador, enquanto a segunda relaciona o comentário com a publicação através da chave de publicação. O resultado apresenta o nome do utilizador, a descrição da publicação (designada como “DescricaoPublicacao”) e o texto do comentário (designado como `TextoComentario`).

```
SELECT    u.nome_utilizador,    p.descricao    AS    DescricaoPublicacao,  
c.comentario AS TextoComentario  
FROM utilizador u  
JOIN comentario c ON u.id_utilizador = c.id_utilizador  
JOIN publicacao p ON c.id_publicacao = p.id_publicacao;
```

Tradução da interrogação 5 - Gestão de Comentários

A gestão de interações na plataforma, especificamente no que diz respeito aos comentários, é efetuada de forma controlada através do procedimento armazenado `CriarComentario`. Esta implementação substitui a inserção direta na tabela, garantindo que as regras de negócio definidas (como a validação de conteúdo e a integridade da relação entre utilizador e publicação) sejam verificadas antes da persistência dos dados.

```
CALL CriarComentario([ID_UTILIZADOR], [ID_PUBLICACAO],  
'[TEXTO_DO_COMENTARIO]');
```

Tradução da Interrogação 6 - Gestão do Perfil do Utilizador

A gestão do perfil permite ao utilizador visualizar e atualizar as suas informações pessoais. No sistema NOMÁLIA, esta operação foi reforçada com a introdução do procedimento `AtualizarPerfil`, que centraliza as modificações de metadados do utilizador, garantindo que as alterações ao perfil seguem as normas de segurança da base de dados.

```
SELECT nome_utilizador, email, biografia, pontos  
FROM utilizador  
WHERE id_utilizador = 2;
```

```
CALL AtualizarPerfil(2, 'Explorador nato e apaixonado por  
fotografia!');
```

```
UPDATE utilizador
SET biografia = NULL
WHERE id_utilizador = 2;
```

Tradução da Interrogação 7 - Consulta de Vales

Esta interrogação lista todos os vales de desconto que já foram trocados por todos os utilizadores. A consulta combina três tabelas: “utilizador”, “vale_de_desconto” e “tipo_vale”. A primeira junção (JOIN) associa o utilizador aos vales que possui, através da chave “id_utilizador” presente na tabela “vale_de_desconto”. A segunda junção relaciona cada vale com o tipo correspondente, através da chave “id_tipo_vale”. O resultado da projeção final (SELECT) apresenta o “nome_utilizador” e o “valor_pontos_tipo” correspondente, o que permite a visualização do valor do vale que o utilizador trocará.

```
SELECT u.nome_utilizador, t.valor_pontos_tipo
FROM utilizador u
JOIN vale_de_desconto v ON u.id_utilizador = v.id_utilizador
JOIN tipo_vale t ON v.id_tipo_vale = t.id_tipo_vale;
```

Tradução da interrogação 8 - Administração de publicações/comentários

Esta interrogação confere ao administrador a capacidade de moderação a partir da habilidade de remoção de publicações e comentários dos vários utilizadores da plataforma. Esta é dividida em duas fases: a fase de identificação, que corresponde a uma operação SELECT e aparece também na interrogação em Álgebra Relacional; e a fase de execução (DELETE). A forma como a eliminação automática das publicações associadas às viagens e dos comentários associados às publicações é tratada é definida pela regra ON DELETE CASCADE, aplicada no Modelo Físico nas respectivas chaves estrangeiras.

Para a remoção de viagens na totalidade temos a instrução DELETE. Esta remoção implica também a remoção das publicações que estão a elas associadas, dos comentários escritos nas mesmas, dos ficheiros e do registo na tabela associativa.

```
DELETE FROM comentario
WHERE id_publicacao IN (
    SELECT id_publicacao
    FROM publicacao
    WHERE id_viagem = 1
);
```

```
DELETE FROM ficheiro_audio_visual
WHERE id_publicacao IN (
    SELECT id_publicacao
    FROM publicacao
    WHERE id_viagem = 1
);
```

```
DELETE FROM publicacao
WHERE id_viagem = 1;
```

```
DELETE FROM viagem_cidade
```



```
WHERE id_viagem = 1;
```

```
DELETE FROM viagem  
WHERE id_viagem = 1;
```

Todas as interrogações foram testadas e validadas, garantindo a correção dos resultados e a adequação aos requisitos funcionais. As interrogações parametrizadas (indicadas com []) devem ser implementadas de forma segura, de modo a prevenir ataques de injeção SQL e a assegurar a consistência dos dados. A organização das interrogações por funcionalidades contribui para uma manutenção mais simples e para uma melhor compreensão do sistema NOMÁLIA.

5.7. Indexação do Sistema de Dados

A indexação de uma base de dados relacional constitui um dos mecanismos fundamentais para otimizar o desempenho das operações de leitura, especialmente em sistemas onde as interrogações são frequentes e incidem sobre grandes volumes de registos. Um índice é uma estrutura auxiliar mantida pelo SGBD, que permite acelerar a procura de linhas numa tabela, evitando leituras completas e sequenciais (*full table scans*).

No caso da plataforma NOMÁLIA, a equipa de desenvolvimento adotou uma estratégia de indexação, baseada maioritariamente nos índices associados às restrições definidas no modelo físico, nomeadamente chaves primárias, chaves estrangeiras e restrições de unicidade.

O MySQL cria automaticamente índices para todas as chaves primárias, garantindo a unicidade dos registos e permitindo um acesso rápido através dos seus identificadores. De igual forma, as restrições de unicidade (UNIQUE), como as aplicadas ao nome de utilizador e ao endereço de correio eletrónico, originam índices que permitem pesquisas eficientes e asseguram a integridade dos dados.

Relativamente às chaves estrangeiras, o MySQL garante a existência de índices adequados para suportar a integridade referencial e otimizar as operações de junção entre tabelas relacionadas. Dado que o modelo físico da plataforma inclui várias relações entre entidades, nomeadamente entre “utilizador”, “viagem”, “publicacao” e “comentario”, a maioria das interrogações previstas beneficia diretamente destes índices.

Após a implementação do modelo físico e a análise dos requisitos de manipulação de dados, a equipa de desenvolvimento concluiu que não era necessária a criação de índices adicionais. As operações mais frequentes do sistema baseiam-se essencialmente em: identificadores primários; relações entre tabelas suportadas por chaves estrangeiras e atributos já protegidos por restrições de unicidade.

A criação de índices adicionais implicaria custos acrescidos ao nível do armazenamento e do desempenho das operações de escrita (INSERT, UPDATE e DELETE), uma vez que os índices necessitam de ser atualizados sempre que os dados são modificados. Considerando a dimensão prevista da plataforma e a complexidade moderada das interrogações, optou-se por uma solução simples e sustentável, confiando nos mecanismos de indexação fornecidos pelo SGBD.

Apesar de não terem sido criados índices adicionais, consideremos, a título ilustrativo, a criação de um índice sobre o atributo “data” da tabela “publicacao”, caso se verificasse um aumento significativo do volume de dados no SGBD ou uma utilização intensiva de ordenações cronológicas das publicações existentes:

```
CREATE INDEX idx_publicacao_data ON publicacao(data);
```

Com este índice, conseguimos otimizar interrogações que envolvam operações de ordenação ou filtragem por data. Porém, devido ao contexto atual da plataforma NOMÁLIA, a equipa de desenvolvimento considerou desnecessário este índice.

A estratégia de indexação adotada garante, assim, um equilíbrio adequado entre desempenho e facilidade de manutenção. Caso, no futuro, se verifique um aumento significativo do volume de dados ou a necessidade de otimizar interrogações específicas, poderão ser considerados índices adicionais.

5.8. Implementação de procedimentos, funções e gatilhos

Em MySQL é possível a execução e definição de procedimentos, funções e gatilhos (*triggers*), que permitem o controlo do fluxo de execução de várias consultas no Sistema de Gestão de Base de Dados, sem ser necessário recorrer a algum tipo de *software* aplicacional para gerir operações condicionais e cíclicas. Os procedimentos e funções são geralmente utilizados na automatização lógica repetitiva ou administrativa, enquanto os *triggers*, que são executados automaticamente devido a mudanças em tabelas, asseguram regras de consistência e atualização de atributos derivados.

Tal como referido na documentação (Oracle, 2024a), quando uma definição contém diversas instruções terminadas por ponto e vírgula, é necessário alterar o delimitador temporariamente com DELIMITER.

Na plataforma NOMÁLIA, estes mecanismos permitem:

- garantir a segurança das palavras-passes (requisitos 22, 24 e 25);
- automatizar o cálculo de pontos e níveis (requisito 21);
- assegurar a coerência na troca de vales (requisito 19);
- validar interações sociais (seguir, comentar);
- facilitar o povoamento massivo da BD para testes.

5.8.1. Procedimentos

- Procedimento TrocarValeDesconto

Para dar resposta ao **Requisito de Manipulação 19** e ao **Requisito de Controlo 21** que estabelece que os utilizadores devem poder trocar os pontos acumulados por vales de desconto e que a atribuição de pontos deve ser controlada de forma consistente, foi implementado o procedimento armazenado **TrocarValeDesconto**. Este procedimento tem como principal objetivo assegurar que a troca de pontos por vales ocorre de forma segura, coerente e consistente, evitando situações em que um utilizador possa obter um vale sem

dispor do número de pontos necessário ou em que o saldo de pontos fique inconsistente. Ao concentrar esta lógica no próprio SGBD, garante-se que as regras do sistema de recompensas são respeitadas independentemente da aplicação cliente utilizada. O procedimento recebe como parâmetros o identificador do utilizador e o identificador do tipo de vale pretendido. Durante a sua execução, começa por verificar se o tipo de vale existe na base de dados, obtendo o respetivo custo em pontos. De seguida, é calculado o saldo atual de pontos do utilizador através da função **PontosDisponiveis**, que devolve o valor armazenado no atributo “pontos” da tabela **utilizador**.

Caso o utilizador não disponha de pontos suficientes, a operação é interrompida e é devolvida uma mensagem de erro informativa. Se o saldo for suficiente, a troca é realizada no contexto de uma transação, garantindo a atomicidade da operação. O procedimento atualiza o saldo de pontos do utilizador e regista o novo vale na tabela “vale_de_desconto”, associando-o ao respetivo utilizador.

Desta forma, o procedimento **TrocarValeDesconto** assegura o cumprimento dos requisitos funcionais associados ao sistema de recompensas da plataforma NOMÁLIA, reforçando a integridade dos dados e reduzindo a complexidade da camada aplicacional.

```
-- DROP PROCEDURE IF EXISTS TrocarValeDesconto;
DELIMITER $$

CREATE PROCEDURE TrocarValeDesconto(
    IN p_id_utilizador INT,
    IN p_id_tipo_vale INT
)
BEGIN
    DECLARE custo INT;
    DECLARE saldo INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Erro na troca de vale.';
    END;

    SELECT valor_pontos_tipo INTO custo
    FROM tipo_vale
    WHERE id_tipo_vale = p_id_tipo_vale;

    IF custo IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Tipo de vale inexistente.';
    END IF;

    SET saldo = PontosDisponiveis(p_id_utilizador);

    IF saldo < custo THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Pontos insuficientes.';
    END IF;
```

```

START TRANSACTION;

UPDATE utilizador
SET pontos = pontos - custo
WHERE id_utilizador = p_id_utilizador;

INSERT INTO vale_de_desconto (usado, id_tipo_vale,
id_utilizador)
VALUES (FALSE, p_id_tipo_vale, p_id_utilizador);

COMMIT;
END $$

DELIMITER ;

```

- Procedimento RegistrarViagemPublicacao

Para dar resposta aos **Requisitos de Manipulação 14 e 15**, que definem que um utilizador deve poder registar viagens e criar publicações associadas às mesmas, foi desenvolvido o procedimento armazenado **RegistrarViagemPublicacao**. Este procedimento automatiza a criação de uma viagem e da respetiva publicação de forma integrada, concentrando num único ponto toda a lógica necessária para inserir informação relacionada com estas entidades. Esta abordagem reduz a complexidade da camada aplicacional e garante a consistência dos dados armazenados. O procedimento recebe como parâmetros o identificador do utilizador, o tipo de viagem, as datas de partida e de chegada e a descrição da publicação. A sua execução decorre no contexto de uma transação, assegurando que todas as operações são realizadas de forma atômica. Inicialmente, é criada a viagem associada ao utilizador e ao tipo de viagem selecionado. De seguida, é criada a publicação associada à viagem e ao utilizador. Caso ocorra algum erro durante a execução, a transação é revertida automaticamente, evitando a criação de registos parciais ou inconsistentes.

Desta forma, o procedimento **RegistrarViagemPublicacao** garante a integridade referencial entre viagens, publicações e utilizadores, contribuindo para uma gestão mais fiável e estruturada das experiências partilhadas na plataforma NOMÁLIA.

```

-- DROP PROCEDURE IF EXISTS RegistrarViagemPublicacao;
DELIMITER $$

```

```

CREATE PROCEDURE RegistrarViagemPublicacao(
    IN p_id_utilizador INT,
    IN p_id_tipo_viagem INT,
    IN p_data_partida DATE,
    IN p_data_chegada DATE,
    IN p_descricao VARCHAR(500)
)
BEGIN
    DECLARE v_id_viagem INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'

```

```

        SET MESSAGE_TEXT = 'Erro ao registrar viagem e publicação.';
    END;

    START TRANSACTION;

    INSERT INTO viagem (data_partida, data_chegada, id_tipo_viagem,
id_utilizador)
        VALUES (p_data_partida, p_data_chegada, p_id_tipo_viagem,
p_id_utilizador);

    SET v_id_viagem = LAST_INSERT_ID();

    INSERT INTO publicacao (data, descricao, id_viagem,
id_utilizador)
        VALUES (CURDATE(), p_descricao, v_id_viagem, p_id_utilizador);

    COMMIT;
END $$

DELIMITER ;

```

- Procedimento AtualizarPerfil

Para dar resposta ao **Requisito de Manipulação 18**, que estabelece que um utilizador deve poder inserir, editar e visualizar os dados do seu próprio perfil, foi desenvolvido o procedimento armazenado **AtualizarPerfil**. Este procedimento tem como objetivo permitir a atualização controlada dos dados de perfil de um utilizador, garantindo que apenas o respetivo registo é alterado. O procedimento recebe como parâmetros o identificador do utilizador autenticado e a nova biografia. A operação de atualização é restringida ao registo, cujo identificador corresponde ao utilizador em sessão, assegurando que não é possível alterar os dados de outros utilizadores.

Desta forma, o procedimento **AtualizarPerfil** permite cumprir o requisito funcional de edição do perfil e garante a integridade e a segurança dos dados pessoais armazenados na plataforma **NOMÁLIA**.

```

-- DROP PROCEDURE IF EXISTS AtualizarPerfil;
DELIMITER $$

CREATE PROCEDURE AtualizarPerfil(
    IN p_id_utilizador INT,
    IN p_biografia VARCHAR(400))
BEGIN
    UPDATE utilizador
    SET biografia = p_biografia
    WHERE id_utilizador = p_id_utilizador;
END $$

DELIMITER ;

```

- Procedimento CriarComentario

Este procedimento dá resposta ao Requisito de Manipulação 16, estabelecendo a possibilidade dos utilizadores poderem criar comentários nas publicações. A inserção do comentário é feita através da instrução INSERT sobre a tabela **comentario**, que associa o comentário ao utilizador autenticado e à publicação em causa, através dos identificadores. Assim, o processo de criação de comentários assegura o cumprimento de requisitos funcionais e a integridade de dados na NOMALIA.

```
-- DROP PROCEDURE IF EXISTS CriarComentario
DELIMITER $$

CREATE PROCEDURE CriarComentario(
    IN p_id_utilizador INT,
    IN p_id_publicacao INT,
    IN p_texto VARCHAR(500)
)
BEGIN
    INSERT INTO comentario (id_utilizador, id_publicacao, comentario)
    VALUES (p_id_utilizador, p_id_publicacao, p_texto);
END $$
DELIMITER;
```

5.8.2. Funções

- Função PontosDisponiveis

A função **PontosDisponiveis** está associada ao Requisito de Controlo 21, uma vez que suporta o sistema de atribuição e gestão de pontos, garantindo que o saldo de pontos de cada utilizador é consultado de forma consistente e centralizada.

Ao contrário de abordagens baseadas em cálculos dinâmicos a partir de múltiplas tabelas, o modelo físico atual armazena explicitamente o número de pontos no atributo “pontos” da tabela **utilizador**. A função recebe como parâmetro o identificador do utilizador e devolve diretamente o valor armazenado, garantindo simplicidade, eficiência e consistência.

Esta função permite centralizar o acesso ao saldo de pontos no próprio SGBD, facilitando a sua reutilização por outros componentes, nomeadamente procedimentos e funções relacionadas com o sistema de gamificação e de troca de vales de desconto.

```
-- DROP FUNCTION IF EXISTS PontosDisponiveis;
DELIMITER $$

CREATE FUNCTION PontosDisponiveis(p_id_utilizador INT)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE saldo INT;

    SELECT pontos INTO saldo
    FROM utilizador
```

```

WHERE id_utilizador = p_id_utilizador;

RETURN IFNULL(saldo, 0);
END $$

DELIMITER ;

```

5.8.3. Gatilhos (*Triggers*)

- Gatilho ValidaPasse

De forma a cumprir os **Requisitos de Controlo 24 e 25** relativos à segurança das palavras-passe, foi implementado o gatilho **ValidaPasse**, executado antes da inserção de um novo registo na tabela **utilizador**.

Este gatilho tem como objetivo garantir que todas as palavras-passe armazenadas na base de dados cumprem os critérios mínimos de segurança definidos e que nunca são guardadas em formato legível. Este gatilho tem como objetivo garantir que todas as palavras-passe armazenadas na base de dados cumprem os critérios mínimos de segurança definidos, nomeadamente um comprimento mínimo de caracteres. Quando a palavra-passe fornecida não cumpre os requisitos, a operação de inserção é interrompida através da instrução SIGNAL, sendo devolvida uma mensagem de erro apropriada.

Quando a palavra-passe é considerada válida, o gatilho procede à sua cifragem utilizando a função SHA2, garantindo que a palavra-passe nunca seja armazenada em formato legível. Desta forma, a política de segurança é aplicada de forma centralizada no próprio SGBD, assegurando confidencialidade e conformidade com os requisitos definidos.

Desta forma, o gatilho **ValidaPasse** assegura que a política de segurança de palavras-passe é aplicada de forma centralizada no próprio SGBD, independentemente da aplicação cliente utilizada, garantindo confidencialidade, consistência e conformidade com os requisitos de segurança definidos para a plataforma NOMÁLIA.

```

-- DROP TRIGGER IF EXISTS ValidaPasse;
DELIMITER $$

CREATE TRIGGER ValidaPasse
BEFORE INSERT ON utilizador
FOR EACH ROW
BEGIN
    IF CHAR_LENGTH(NEW.palavra_passe) < 15 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'A palavra-passe deve ter pelo menos 15
caracteres.';
    END IF;

    SET NEW.palavra_passe = SHA2(NEW.palavra_passe, 256);

```

```
END $$
```

```
DELIMITER ;
```

- Gatilho ImpedeAutoFollow

Para cumprir o **Requisito de Manipulação 13** e garantir a coerência lógica das relações sociais da plataforma NOMÁLIA, foi implementado o gatilho **ImpedeAutoFollow**, executado antes da inserção de novos registos na tabela **segue**. Durante a sua execução, o gatilho verifica se o identificador do utilizador que segue coincide com o identificador do utilizador seguido. Caso esta condição se verifique, a operação é interrompida através da instrução SIGNAL, impedindo a criação de registos inválidos.

Assim, o gatilho **ImpedeAutoFollow** assegura a integridade lógica das interações sociais da plataforma, reforçando o cumprimento dos requisitos funcionais definidos e reduzindo a necessidade de validações adicionais na camada aplicacional.

```
-- DROP TRIGGER IF EXISTS ImpedeAutoFollow;  
DELIMITER $$
```

```
CREATE TRIGGER ImpedeAutoFollow  
BEFORE INSERT ON segue  
FOR EACH ROW  
BEGIN  
    IF NEW.id_utilizador = NEW.id_seguidor THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Um utilizador não pode seguir-se a si  
próprio.';  
    END IF;  
END $$  
DELIMITER ;
```

- Gatilho ValidaComentario

Com o objetivo de cumprir os **Requisitos de Manipulação 16 e 17** e garantir a qualidade e a coerência do conteúdo gerado pelos utilizadores na plataforma NOMÁLIA, foi implementado o gatilho **ValidaComentario**, executado antes da inserção de novos registos na tabela **comentario**. Este gatilho assegura que apenas comentários válidos são armazenados na base de dados, evitando que o texto do comentário seja nulo ou se, após a remoção de espaços em branco, se encontre vazio. Caso alguma destas situações se verifique, a inserção é interrompida através da instrução SIGNAL, evitando o armazenamento de comentários sem conteúdo relevante.

Desta forma, o gatilho **ValidaComentario** reforça a integridade lógica da funcionalidade de comentários da plataforma, garantindo o cumprimento dos requisitos funcionais e reduzindo a necessidade de validações adicionais na camada aplicacional.

```
-- DROP TRIGGER IF EXISTS ValidaComentario;  
DELIMITER $$
```



```

CREATE TRIGGER ValidaComentario
BEFORE INSERT ON comentario
FOR EACH ROW
BEGIN
    IF NEW.comentario IS NULL OR TRIM(NEW.comentario) = '' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'O comentário não pode ser vazio.';
    END IF;
END $$
DELIMITER ;

```

- Gatilho AtribuiPontosPublicacao

O gatilho AtribuiPontosPublicacao é utilizado na atribuição de pontuação (10 pontos por publicação) após a inserção de uma publicação. Este gatilho atualiza os pontos do utilizador que adicionou uma publicação, de forma a garantir que o sistema de pontos é efetuado de forma consistente. Esta ação de atribuição não foi incluída no procedimento RegistrarViagemPublicacao, porque em situações de INSERT publicacao(...), não seriam atribuídos os pontos, dado que apenas seriam atribuídos caso o procedimento fosse utilizado.

```

-- DROP TRIGGER IF EXISTS AtribuiPontosPublicacao
DELIMITER $$

```

```

CREATE TRIGGER AtribuiPontosPublicacao
AFTER INSERT ON publicacao
FOR EACH ROW
BEGIN
    UPDATE utilizador
    SET pontos = pontos + 10
    WHERE id_utilizador = NEW.id_utilizador;
END $$
DELIMITER ;

```

- Gatilho ValidaAutorComentario

O gatilho ValidaAutorComentario encontra-se inteiramente associado ao Requisito de Manipulação 17, dado que garante que um utilizador não pode alterar o comentário de um outro utilizador. Com este gatilho é possível garantir a segurança dos utilizadores, dado que com o ValidaAutorComentario, o identificador do utilizador é comparado com o identificador do autor do comentário que foi alterado anteriormente, antes da sua atualização da tabela **comentario**. Caso os identificadores não coincidam, a operação é interrompida.

```

-- DROP TRIGGER IF EXISTS ValidaAutorComentario;
DELIMITER $$
CREATE TRIGGER ValidaAutorComentario
BEFORE UPDATE ON comentario
FOR EACH ROW
BEGIN
    IF OLD.id_utilizador <> NEW.id_utilizador THEN
        SIGNAL SQLSTATE '45000'

```

```

        SET MESSAGE_TEXT = 'Não é permitido alterar comentários de
outros utilizadores.';
    END IF;
END $$
DELIMITER ;

```

5.9. Backup da Base de Dados

De forma a garantir a segurança e a integridade dos dados armazenados na base de dados da plataforma NOMÁLIA, foi desenvolvido um *shell script* que permite a realização automática de cópias de segurança (*backups*) da base de dados.

O script cria um ficheiro de *backup* contendo a data atual no nome, armazenando-o numa subpasta designada *backups*, localizada no mesmo diretório onde o script é executado. Caso a pasta não exista, esta é criada automaticamente. O processo de backup é efetuado recorrendo à ferramenta *mysqldump*, utilizada em sistemas *MySQL* para exportação de bases de dados. No final da execução, o script informa o utilizador se o processo foi concluído com sucesso ou se ocorreu algum erro durante a criação da cópia de segurança.

Para efeitos de simplicidade e demonstração académica, as credenciais de acesso à base de dados encontram-se definidas diretamente no script. Em ambientes de produção, esta informação deverá ser protegida através de mecanismos mais seguros, como variáveis de ambiente ou ficheiros de configuração com permissões restritas.

```

#!/bin/bash

USER="root"

PASSWORD="root"

DATABASE="nomalia"

BACKUP_DIR="$(dirname "$(realpath "$0")")/backups"

mkdir -p "$BACKUP_DIR"

DATA=$(date +"%Y-%m-%d")

BACKUP_FILE="$BACKUP_DIR/backup_${DATABASE}_${DATA}.sql"

mysqldump -u "$USER" -p"$PASSWORD" "$DATABASE" > "$BACKUP_FILE"

if [ $? -eq 0 ]; then

    echo "Backup realizado com sucesso: $BACKUP_FILE"

else

    echo "Erro ao realizar o backup da base de dados."

```

fi

6. Conclusões e Trabalho Futuro

A implementação da base de dados da plataforma NOMÁLIA permitiu consolidar todas as fases do ciclo de desenvolvimento de um sistema de informação relacional, desde a análise inicial dos requisitos até ao desenvolvimento do modelo físico. O trabalho realizado demonstra a importância de uma abordagem estruturada para garantir a integridade, consistência e eficiência do sistema, especialmente numa aplicação que envolve relacionamento entre utilizadores, criação de conteúdos multimédia, interação social, entre outros.

Ao longo deste projeto, foi possível identificar e formalizar os Requisitos de Manipulação, Controlo e Descrição, que orientaram a criação das entidades do modelo de dados e a implementação das regras de acesso e segurança. A normalização das entidades, garantindo conformidade até à 3FN, permitiu eliminar redundâncias e assegurar a integridade lógica dos dados. O modelo físico, definido em SQL, foi posteriormente traduzido num *script* rigoroso que cria todas as tabelas, restrições e relações necessárias para o correto funcionamento da plataforma.

A definição de *roles* de permissões garantiram que cada tipo de utilizador possui exatamente o nível de acesso adequado às suas responsabilidades, respeitando os requisitos de segurança e privacidade associados a uma plataforma social.

Adicionalmente, o processo de povoamento demonstrou a adequação do modelo lógico às necessidades reais da aplicação, permitindo validar relações, chaves estrangeiras e interações típicas no uso da plataforma.

A análise do espaço de armazenamento confirmou que a base de dados se encontra otimizada, garantindo uma utilização eficiente dos recursos. A secção relativa às vistas de utilização realizou uma reflexão crítica sobre a sua utilidade e justificou a opção de não as implementar nesta fase, dado que o controlo de acessos pôde ser inteiramente gerido ao nível dos *roles*.

Por fim, a tradução das interrogações do utilizador para SQL evidenciou que o modelo desenvolvido suporta corretamente todas as operações exigidas, validando a coerência entre a definição teórica do sistema e a sua implementação concreta.

Um ponto relevante do desenvolvimento desta solução foi o respeito pela planificação estabelecida, cumprindo-se o cronograma de trabalho representado no diagrama de Gantt. Apesar de não terem ocorrido atrasos significativos, foi necessário um ajustamento, a meio do desenvolvimento, do número de entidades inicialmente previsto, devido à identificação de requisitos adicionais e à complexidade do relacionamento entre utilizadores, conteúdos e interações. Este contratempo implicou um acréscimo de esforço em cumprir as datas estabelecidas.

Embora a base de dados, atualmente implementada, cumpra todos os requisitos identificados na fase de análise, existem diversas oportunidades de evolução e otimização que poderão ser consideradas em futuras versões da plataforma. Estas melhorias visam reforçar a segurança, o desempenho e a eficiência do sistema, acompanhando o crescimento natural da plataforma e o aumento do volume de dados.

Uma dessas melhorias consiste na implementação de vistas para controlo de acesso avançado. Embora não tenham sido utilizadas nesta fase, as vistas poderão ser úteis, permitindo expor subconjuntos específicos de dados, como viagens pertencentes apenas ao

próprio utilizador, ocultando simultaneamente atributos sensíveis quando necessário. Além disso, podem servir como mecanismos de simplificação de consultas complexas.

Com o crescimento da plataforma, será igualmente necessária uma otimização avançada de performance, recorrendo à criação de índices compostos baseados em padrões reais de consulta. Esta medida permitirá reduzir os tempos de resposta.

No que respeita à segurança, prevê-se a implementação de mecanismos de recuperação de conta e de autenticação mais robustos.

Por fim, a interação social poderá ser expandida através de funcionalidades adicionais, como mensagens privadas entre utilizadores, gostos e reações às publicações, bem como recomendações automáticas de perfis a seguir.

Concluindo, o desenvolvimento da base de dados da plataforma NOMÁLIA, não só permitiu consolidamos os ensinamentos da Unidade Curricular de Base de Dados, como também demonstrou que um sistema social e interativo pode ser corretamente suportado por um modelo relacional robusto, desde que exista uma análise criteriosa dos requisitos e uma implementação fiel e rigorosa. A arquitetura atual garante estabilidade, segurança e consistência, proporcionando uma fundação sólida para o crescimento da plataforma. As propostas de trabalho futuro abrem portas para novas funcionalidades e melhorias que poderão expandir a experiência dos utilizadores e otimizar o desempenho global do sistema.

7. Bibliografia

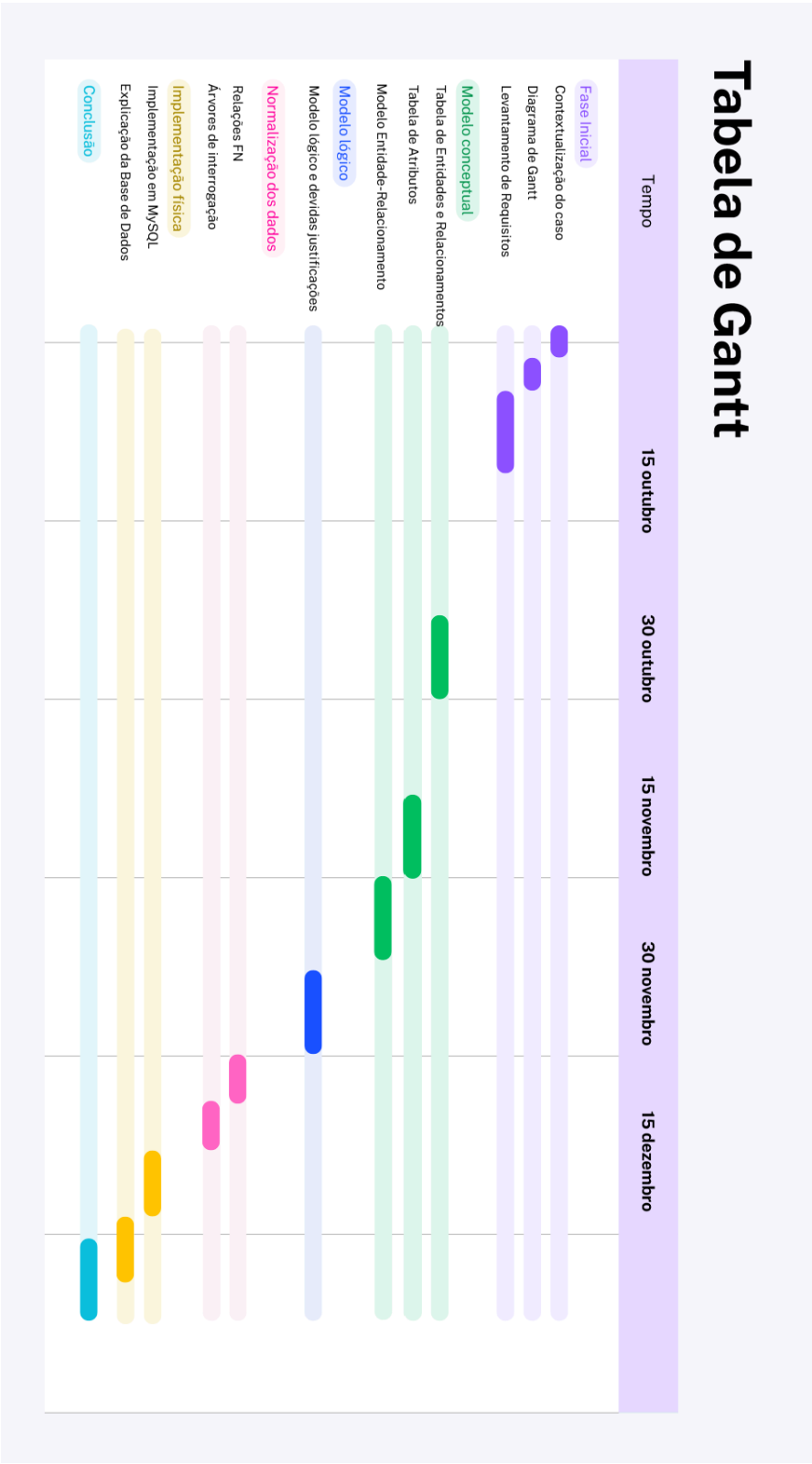
- Cândido, C. (2020). *brModelo*. SIS4.com.
<https://www.sis4.com/brModelo/>
- Oracle Corporation. (2025). *MySQL Workbench – Downloads*.
<https://dev.mysql.com/downloads/workbench/>
- Oracle Corporation. (2025). *MySQL Community Server – Downloads*.
<https://dev.mysql.com/downloads/mysql/>
- Kessler, J., Tschuggnall, M. & Specht, G. (2025). *RelaX – Relational Algebra Calculator*.
<https://dbis-uibk.github.io/relax/landing>
- International Hellenic University. (2025). *RelaX – Relational Algebra Calculator*.
<https://nireas.iee.ihu.gr/relax/calc.htm>
- mdaines. (2025). *Viz.js – Graphviz in your browser*.
<https://viz-js.com/>
- Elmasri, R. & Navathe, S. (2016). *Fundamentals of Database Systems*. 7th ed. Pearson.
<https://people.vts.su.ac.rs/~peti/Baze%20podataka/Literatura/Elmasri-Navathe-Fundamentals%20of%20DB%20Systems-EbookDB.pdf>
- Silberschatz, A., Korth, H. F. & Sudarshan, S. (2011). *Database System Concepts*. 6th ed. New York: McGraw-Hill.
<https://people.vts.su.ac.rs/~peti/Baze%20podataka/Literatura/Silberschatz-Database%20System%20Concepts%206th%20ed.pdf>
- Codd, E., Further Normalization of the Data Base Relational Model, R. Rustin (ed.), *Data Base Systems* (Courant Computer Science Symposia 6), Prentice-Hall, 1972.
<https://forum.thethirdmanifesto.com/wp-content/uploads/asgarosforum/987737/00-efc-further-normalization.pdf>
- Fagin, R., Normal Forms and Relational Database Operators, ACM SIGMOD International Conference on Management of Data , Boston, Mass, May 31-June 1, 1979.
https://www.researchgate.net/profile/Ronald-Fagin/publication/215697523_Normal_forms_and_relational_database_operators/links/5536610f0cf218056e949795/Normal-forms-and-relational-database-operators.pdf
- Connolly, T., Begg, C., *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th edition, Pearson, January, 2014.
- Chen, P., The entity-relationship model - Toward a unified view of data. *ACM Trans. Database Syst.* 1, 1, 9-36, March 1976.

Lista de Siglas e Acrónimos

BD	Base de Dados
SBD	Sistema de Base de Dados
SBDR	Sistema de Base de Dados Relacional
SGBDR	Sistema de Gestão de Base de Dados Relacional
CEO	<i>Chief Executive Officer</i>
MySQL; SQL	<i>Structured Query Language</i> (tecnologia; linguagem)
id	Identificador Único
ER	Entidade-Relacionamento
PNG	<i>Portable Network Graphic</i>
1FN	Primeira Forma Normal
2FN	Segunda Forma Normal
3FN	Terceira Forma Normal
PK	<i>Primary Key</i>
FK	<i>Foreign Key</i>

Anexos

I. Diagrama de Gantt





NOMÁLIA
VIAJA, REGISTA, VIVE

ATA DA REUNIÃO
DATA: 20 DE OUTUBRO DE 2025
HORA: 15H

II. Ata de Reunião

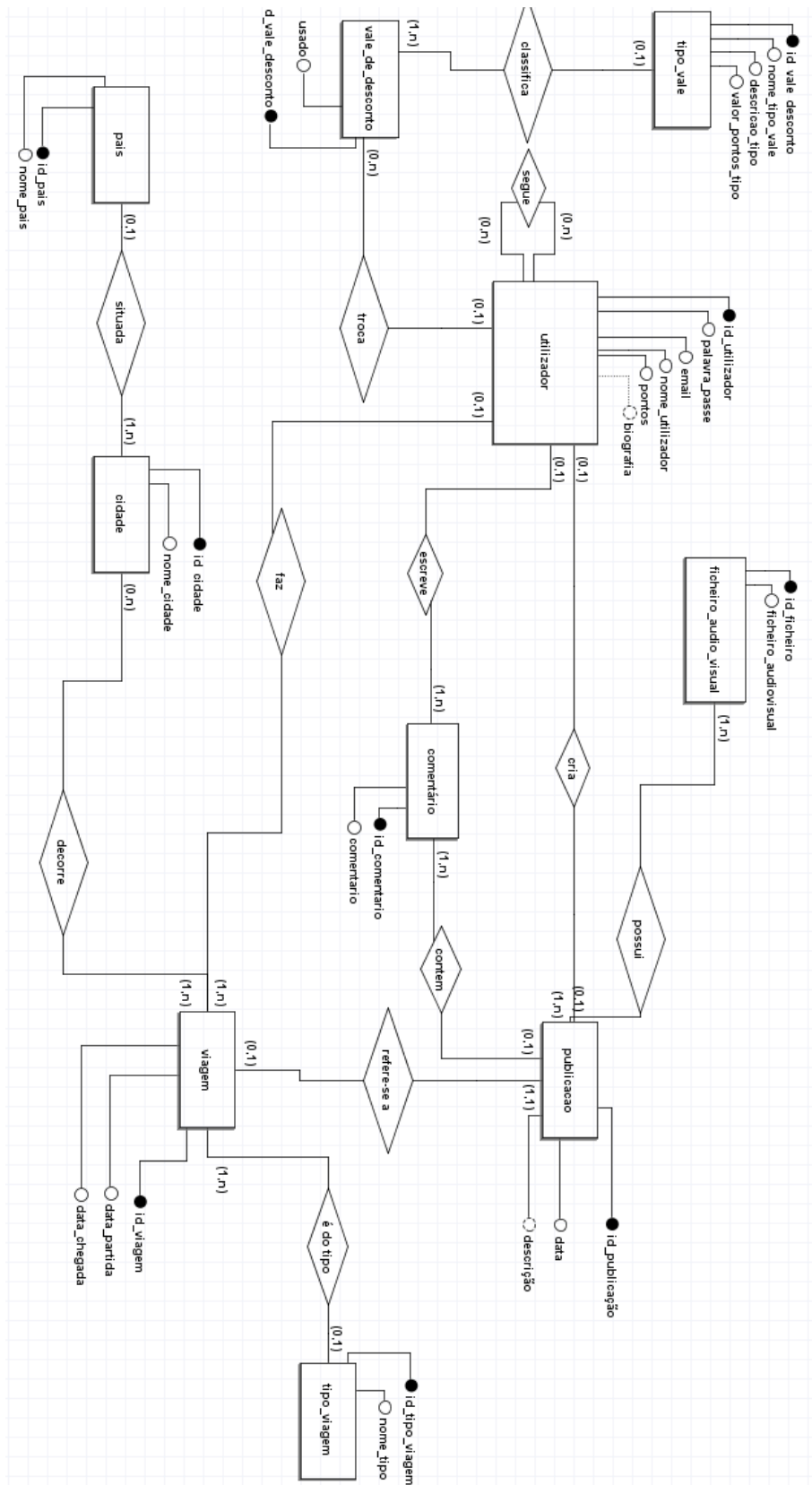
1. Objetivo da Reunião	
Discutir os objetivos principais da plataforma e as funções essenciais da Base de Dados. Explicar a sua visão para NOMÁLIA.	
2. Participantes presentes	
Nome	Posição
Doutora Filipa Martins	Fundadora e Diretora de Nomália
Beatriz Silva	Coordenadora de Comunicação
Ricardo Matos	Gestor técnico de NOMÁLIA

III. Requisitos levantados

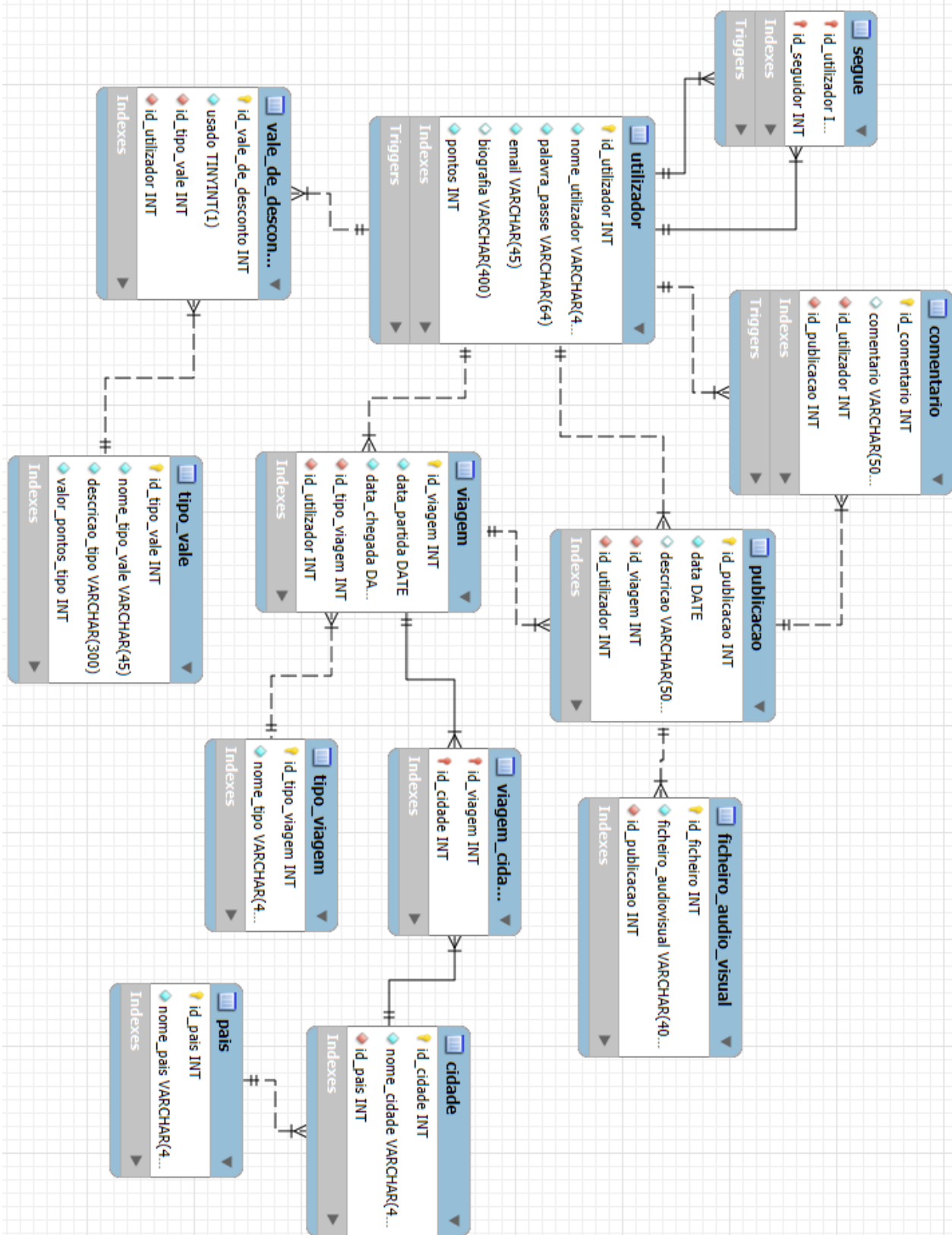
Requisitos	Data/Hora	Descrição	Area	Fonte	Analista
1	18/10 15h30	Cada perfil de utilizador deve incluir o identificador único, nome do utilizador, <i>email</i> , biografia e o número de pontos conquistados.	Utilizador	Filipa Martins	Sara Marques
2	19/10 10h30	A viagem é caracterizada pelo seu tipo, identificador único, destino, data de partida e chegada.	Viagem	Beatriz Silva	Sara Marques
3	19/10 10h40	Uma viagem pode incluir várias cidades pertencentes ao mesmo país.	Viagem	Filipa Martins	Marta Araújo
4	19/10 11h00	A publicação é caracterizada por um identificador único, a respetiva descrição e data de publicação.	Publicação	Beatriz Silva	Teresa Teixeira
5	21/10 19h30	Cada publicação pode conter comentários, e cada comentário deve incluir o respetivo texto e identificador único.	Comentário	Ricardo Matos	Teresa Teixeira
6	21/10 19h50	Cada publicação é associada a apenas uma viagem.	Publicação	Ricardo Matos	Ana Patrícia Machado
7	23/10 15h00	Existe a possibilidade de trocar os pontos acumulados por vales de desconto.	Vale de desconto	Filipa Martins	Teresa Teixeira
8	23/10 19h00	Existem vários tipos de vales possíveis para resgatar: vale de desconto em companhias aéreas, em hotéis e em atividades lúdicas.	Tipo Vale	Ricardo Matos	Marta Araújo
9	23/10 19h00	Existe a possibilidade de anexar a cada publicação, ficheiros áudio-visuais para enriquecer o seu conteúdo.	Ficheiro Áudio-visual	Beatriz Silva	Marta Araújo
10	23/10 19h20	É necessário identificar a cidade em cada viagem publicada, sendo esta caracterizada pelo seu identificador único e nome.	Cidade	Ricardo Matos	Sara Marques
11	23/10 19h40	É necessário associar à cidade que foi conectada a uma viagem, o país correspondente, que é caracterizado pelo seu identificador único e nome.	País	Filipa Martins	Sara Marques
12	25/10 16h00	Existem vários tipos de viagem possíveis: viagem profissional, de lazer, cultural e gastronómica.	Tipo Viagem	Beatriz Silva	Ana Patrícia Machado
13	20/10 18h10	O utilizador pode seguir outros perfis, interagir com as respetivas publicações, deixar de seguir outros utilizadores e visualizar a lista dos seus seguidores.	Utilizador	Beatriz Silva	Ana Patrícia Machado
14	20/10 18h30	O utilizador deve poder criar, editar e visualizar publicações associadas às suas viagens.	Publicação	Filipa Martins	Marta Araújo
15	20/10 19h10	O utilizador deve poder criar, visualizar e editar as suas viagens na plataforma.	Viagem	Ricardo Matos	Ana Patrícia Machado
16	21/10 19h00	Cada publicação tem uma secção de comentários, onde os utilizadores podem partilhar opiniões sobre a viagem em questão.	Comentário	Ricardo Matos	Marta Araújo
17	21/10 19h20	O utilizador deve poder criar, editar e visualizar os seus comentários.	Comentário	Beatriz Silva	Ana Patrícia Machado
18	22/10 10h40	O utilizador deve poder inserir, visualizar ou editar dados do seu perfil.	Utilizador	Filipa Martins	Marta Araújo
19	23/10 15h00	A plataforma deve permitir que os pontos sejam trocados por vales de desconto.	Vale de desconto	Beatriz Silva	Ana Patrícia Machado
20	23/10 15h30	A plataforma deve permitir ao administrador visualizar e remover viagens, as publicações a elas associadas e os comentários dos vários utilizadores que forem escritos nas mesmas.	Administração	Ricardo Matos	Teresa Teixeira
21	23/10 15h00	A plataforma deve atribuir pontos ao utilizador de acordo com a sua atividade no sistema.	Vale de desconto	Filipa Martins	Marta Araújo
22	24/10 11h00	O acesso ao perfil do utilizador deve ser protegido através de um sistema de autenticação com palavra-passe.	Utilizador	Ricardo Matos	Sara Marques

23	24/10 11h20	O e-mail e nome de utilizador devem ser únicos na plataforma.	Utilizador	Beatriz Silva	Ana Patrícia Machado
24	24/10 11h40	A palavra-passe deve cumprir requisitos mínimos de segurança (ex.: número mínimo de 15 caracteres).	Segurança	Ricardo Matos	Marta Araújo
25	24/10 12h00	A palavra-passe não deve ser armazenada em formato legível.	Segurança	Filipa Martins	Sara Marques
26	24/10 12h20	O administrador deve ter permissões específicas que lhe permitam gerir conteúdos e contas, sem acesso aos dados privados dos utilizadores.	Administração	Ricardo Matos	Teresa Teixeira

IV. Diagrama ER



V. Modelo Lógico



VI. Expressões de Álgebra Relacional

$\pi_{seguido.nome_utilizador \rightarrow NomeSeguido}(\sigma_{id_seguidor = X} (segue) \bowtie_{segue.id_utilizador = seguido.id_utilizador} \rho_{seguido} (utilizador))$

$\pi_{u_seguidor.nome_utilizador \rightarrow NomeSeguidor}(\sigma_{id_utilizador = X} (segue) \bowtie_{segue.id_seguidor = u_seguidor.id_utilizador} \rho_{u_seguidor} (utilizador))$

$\pi_{v.id_viagem, u.nome_utilizador, t.nome_tipo, c.nome_cidade, v.data_partida, v.data_chegada} (((\rho_v (viagem) \bowtie_{v.id_utilizador = u.id_utilizador} \rho_u (utilizador)) \bowtie_{v.id_tipo_viagem = t.id_tipo_viagem} \rho_t (tipo_viagem)) \bowtie_{v.id_viagem = vc.id_viagem} \rho_{vc} (viagem_cidade)) \bowtie_{vc.id_cidade = c.id_cidade} \rho_c (cidade))$

$\bowtie_{viagem.id_viagem = vc.id_viagem} \rho_{vc} (viagem_cidade)) \bowtie_{vc.id_cidade = cidade.id_cidade} cidade) \bowtie_{viagem.id_viagem = publicacao.id_viagem} publicacao)$

$\pi_{id_viagem, nome_tipo, nome_cidade, nome_pais, data_partida, data_chegada} (((((\sigma_{id_utilizador = X} (viagem) \bowtie \rho_{tp} (tipo_viagem)) \bowtie \rho_{vc} (viagem_cidade)) \bowtie_{vc.id_cidade = cidade.id_cidade} cidade) \bowtie_{cidade.id_pais = pais.id_pais} pais))$

$\pi_{utilizador.nome_utilizador, publicacao.descricao \rightarrow DescricaoPublicacao, comentario \rightarrow TextoComentario} ((utilizador \bowtie_{utilizador.id_utilizador = comentario.id_utilizador} comentario) \bowtie_{comentario.id_publicacao = publicacao.id_publicacao} publicacao)$

$\pi_{id_comentario, comentario, id_publicacao} (\sigma_{id_utilizador = X} (comentario))$

$\pi_{nome_utilizador, email, biografia, pontos} (\sigma_{id_utilizador = X} (utilizador))$

$\pi_{u.nome_utilizador, t.valor_pontos_tipo} ((\rho_u (utilizador) \bowtie_{u.id_utilizador = v.id_utilizador} \rho_v (vale_de_desconto)) \bowtie_{v.id_tipo_vale = t.id_tipo_vale} \rho_t (tipo_vale))$

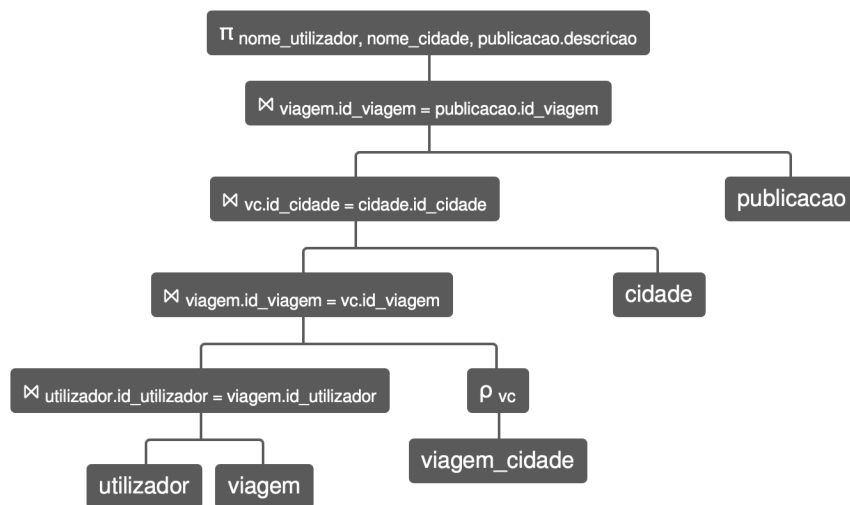
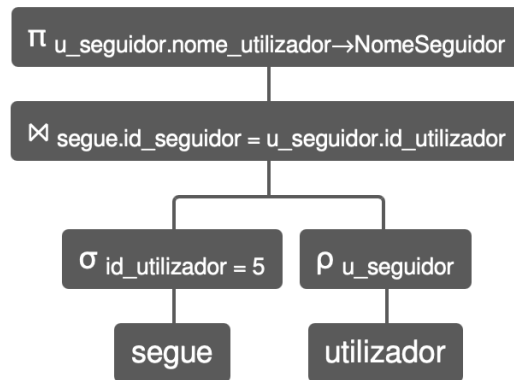
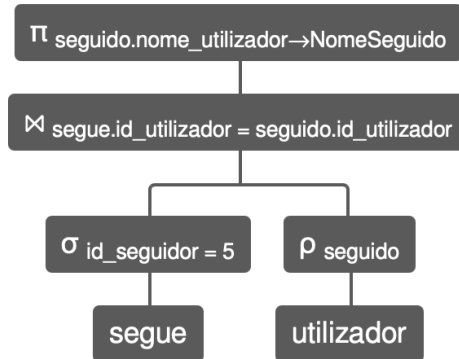
$\pi_{v.id_viagem, u.nome_utilizador, t.nome_tipo, c.nome_cidade, v.data_partida, v.data_chegada} (((\rho_v (viagem) \bowtie_{v.id_utilizador = u.id_utilizador} \rho_u (utilizador)) \bowtie_{v.id_tipo_viagem = t.id_tipo_viagem} \rho_t (tipo_viagem)) \bowtie_{v.id_viagem = vc.id_viagem} \rho_{vc} (viagem_cidade)) \bowtie_{vc.id_cidade = c.id_cidade} \rho_c (cidade))$

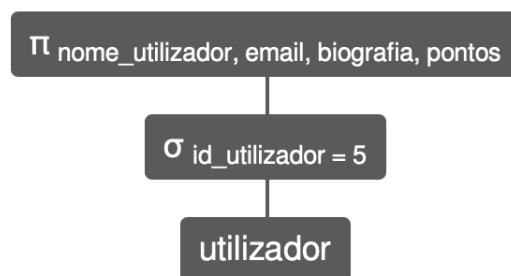
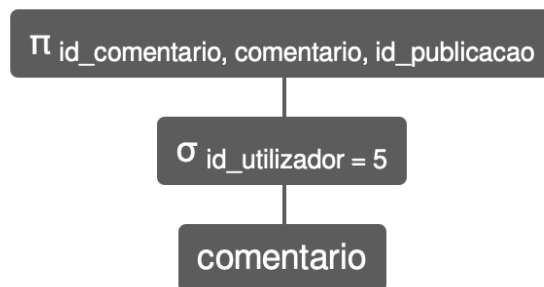
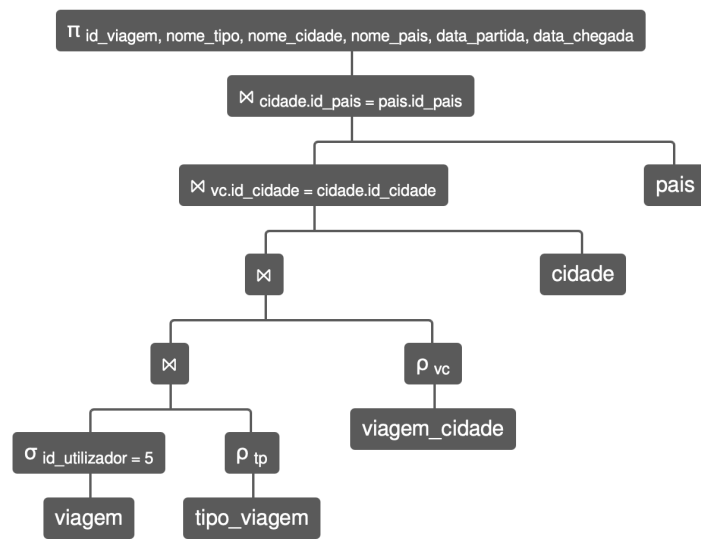
$\pi_{f.id_ficheiro, f.ficheiro_audiovisual} (\rho_f (ficheiro_audio_visual) \bowtie_{f.id_publicacao = p.id_publicacao} \rho_p (publicacao))$

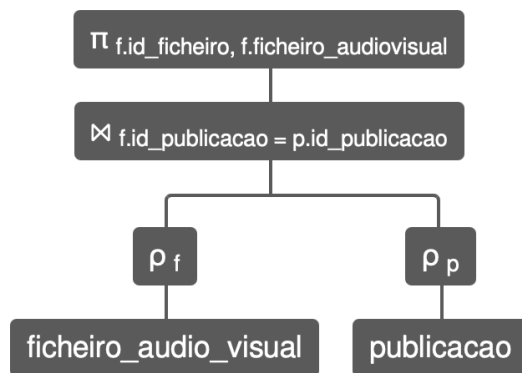
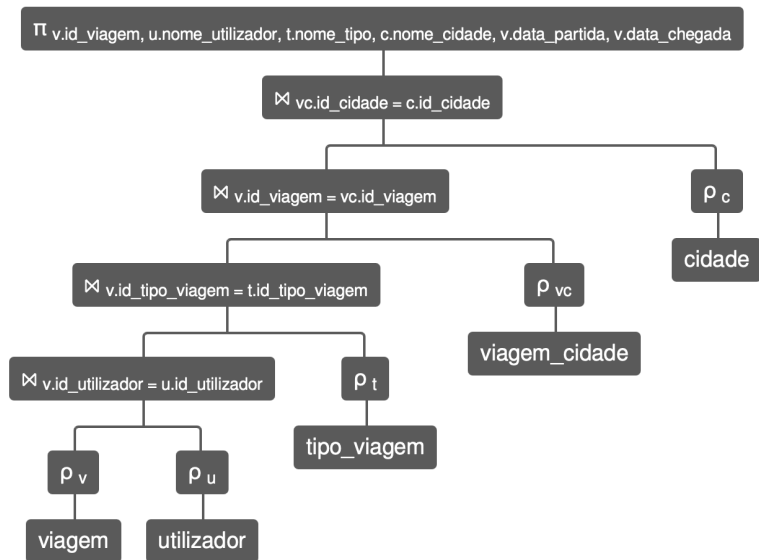
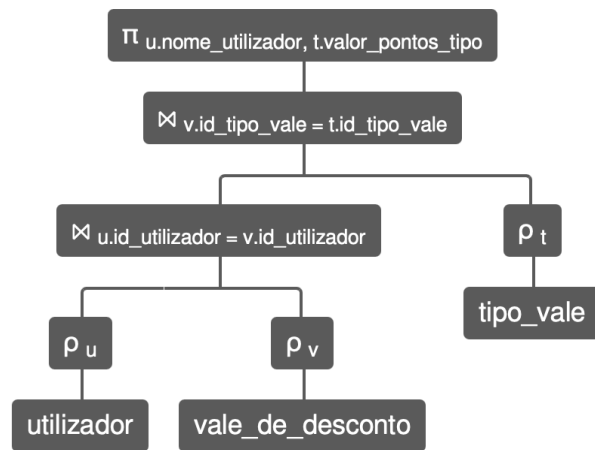
$\pi_{p.id_publicacao, u.nome_utilizador, p.descricao} ((\rho_p (publicacao) \bowtie_{p.id_viagem = v.id_viagem} \rho_v (viagem)) \bowtie_{v.id_utilizador = u.id_utilizador} \rho_u (utilizador))$

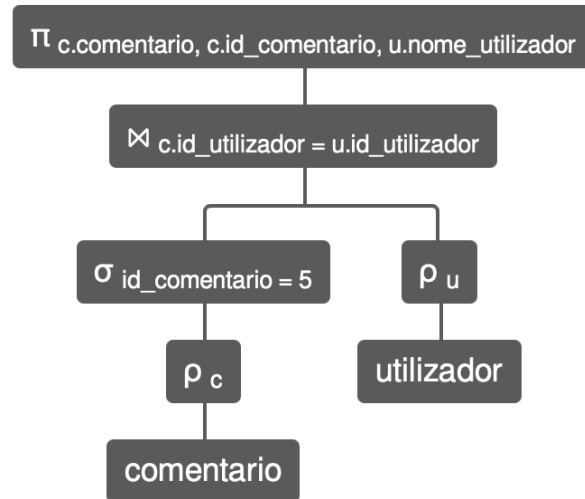
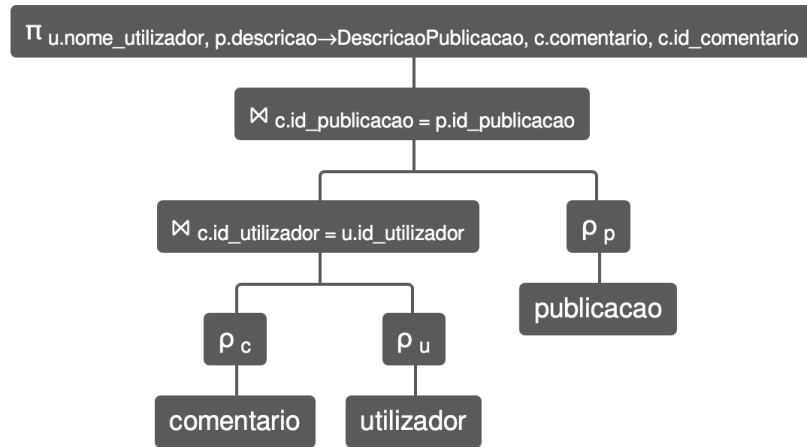
$\pi_{c.comentario, c.id_comentario, u.nome_utilizador} (\sigma_{id_comentario = X} \rho_c (comentario) \bowtie_{c.id_utilizador = u.id_utilizador} \rho_u (utilizador))$

VII. Árvores de Interrogação

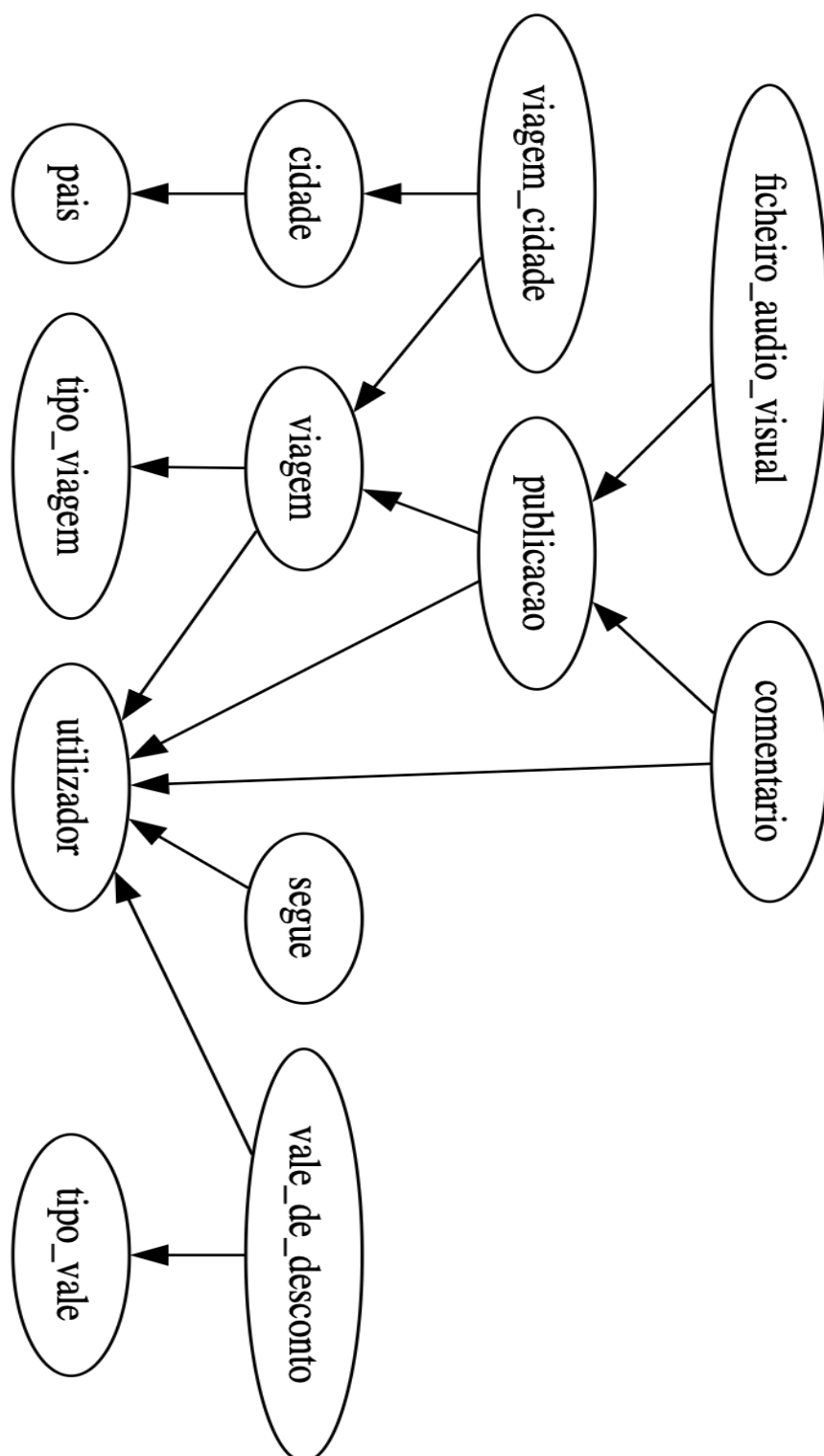








VIII. Grafo do diagrama de dependências



IX. Modelo Físico

```
-- -----  
-- DATABASE nomalia  
-- -----  
  
-- DROP DATABASE IF EXISTS nomalia;  
CREATE DATABASE IF NOT EXISTS nomalia  
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE nomalia ;  
  
-- -----  
-- Table tipo_viagem  
-- -----  
CREATE TABLE IF NOT EXISTS tipo_viagem (  
    id_tipo_viagem INT AUTO_INCREMENT,  
    nome_tipo VARCHAR(45) NOT NULL,  
    PRIMARY KEY (id_tipo_viagem),  
    UNIQUE (nome_tipo)  
) ENGINE=InnoDB;  
  
-- -----  
-- Table utilizador  
-- -----  
CREATE TABLE IF NOT EXISTS utilizador (  
    id_utilizador INT AUTO_INCREMENT,  
    nome_utilizador VARCHAR(45) NOT NULL,  
    palavra_passe VARCHAR(64) NOT NULL,  
    email VARCHAR(45) NOT NULL,  
    biografia VARCHAR(400),  
    pontos INT NOT NULL,  
    PRIMARY KEY (id_utilizador),  
    UNIQUE (nome_utilizador),  
    UNIQUE (email)  
) ENGINE=InnoDB;  
  
-- -----  
-- Table pais  
-- -----  
CREATE TABLE IF NOT EXISTS pais (  
    id_pais INT AUTO_INCREMENT,  
    nome_pais VARCHAR(45) NOT NULL,  
    PRIMARY KEY (id_pais),  
    UNIQUE (nome_pais)  
) ENGINE=InnoDB;  
  
-- -----  
-- Table cidade  
-- -----
```

```

CREATE TABLE IF NOT EXISTS cidade (
    id_cidade INT AUTO_INCREMENT,
    nome_cidade VARCHAR(45) NOT NULL,
    id_pais INT NOT NULL,
    PRIMARY KEY (id_cidade),
    FOREIGN KEY (id_pais)
        REFERENCES pais(id_pais)
) ENGINE=InnoDB;

-- -----
-- Table viagem
-- -----

CREATE TABLE IF NOT EXISTS viagem (
    id_viagem INT AUTO_INCREMENT,
    data_partida DATE NOT NULL,
    data_chegada DATE NOT NULL,
    id_tipo_viagem INT NOT NULL,
    id_utilizador INT NOT NULL,
    PRIMARY KEY (id_viagem),
    FOREIGN KEY (id_tipo_viagem)
        REFERENCES tipo_viagem(id_tipo_viagem),
    FOREIGN KEY (id_utilizador)
        REFERENCES utilizador(id_utilizador)
) ENGINE=InnoDB;

-- -----
-- Table viagem_cidade
-- -----

CREATE TABLE IF NOT EXISTS viagem_cidade (
    id_viagem INT NOT NULL,
    id_cidade INT NOT NULL,
    PRIMARY KEY (id_viagem, id_cidade),
    FOREIGN KEY (id_viagem) REFERENCES viagem(id_viagem),
    FOREIGN KEY (id_cidade) REFERENCES cidade(id_cidade)
);

-- -----
-- Table publicacao
-- -----

CREATE TABLE IF NOT EXISTS publicacao (
    id_publicacao INT AUTO_INCREMENT,
    data DATE NOT NULL,
    descricao VARCHAR(500),
    id_viagem INT NOT NULL,
    id_utilizador INT NOT NULL,
    PRIMARY KEY (id_publicacao),
    UNIQUE (id_viagem),
    FOREIGN KEY (id_viagem) REFERENCES viagem(id_viagem),
    FOREIGN KEY (id_utilizador) REFERENCES utilizador(id_utilizador)
);

-- -----

```

```

-- Table ficheiros_audio_visual
-----
CREATE TABLE IF NOT EXISTS ficheiro_audio_visual (
    id_ficheiro INT AUTO_INCREMENT,
    ficheiro_audiovisual VARCHAR(400) NOT NULL,
    id_publicacao INT NOT NULL,
    PRIMARY KEY (id_ficheiro),
    FOREIGN KEY (id_publicacao)
        REFERENCES publicacao(id_publicacao)
) ENGINE=InnoDB;

-----

-- Table comentario
-----
CREATE TABLE IF NOT EXISTS comentario (
    id_comentario INT AUTO_INCREMENT,
    comentario VARCHAR(500),
    id_utilizador INT NOT NULL,
    id_publicacao INT NOT NULL,
    PRIMARY KEY (id_comentario),
    FOREIGN KEY (id_utilizador)
        REFERENCES utilizador(id_utilizador),
    FOREIGN KEY (id_publicacao)
        REFERENCES publicacao(id_publicacao)
) ENGINE=InnoDB;

-----

-- Table segue
-----
CREATE TABLE IF NOT EXISTS segue (
    id_utilizador INT NOT NULL,
    id_seguidor INT NOT NULL,
    PRIMARY KEY (id_utilizador, id_seguidor),
    FOREIGN KEY (id_utilizador)
        REFERENCES utilizador(id_utilizador),
    FOREIGN KEY (id_seguidor)
        REFERENCES utilizador(id_utilizador)
) ENGINE=InnoDB;

-----

-- Table tipo_vale
-----
CREATE TABLE tipo_vale (
    id_tipo_vale INT AUTO_INCREMENT,
    nome_tipo_vale VARCHAR(45) NOT NULL CHECK (nome_tipo_vale IN
('hotel', 'atividade', 'companhia aerea')),
    descricao_tipo VARCHAR(300) NOT NULL,
    valor_pontos_tipo INT NOT NULL,
    PRIMARY KEY (id_tipo_vale),
    UNIQUE (nome_tipo_vale)
) ENGINE=InnoDB;

```

```

-----
-- Table vale_de_desconto
-----
CREATE TABLE IF NOT EXISTS vale_de_desconto (
    id_vale_de_desconto INT AUTO_INCREMENT,
    usado BOOLEAN NOT NULL,
    id_tipo_vale INT NOT NULL,
    id_utilizador INT NOT NULL,
    PRIMARY KEY (id_vale_de_desconto),
    FOREIGN KEY (id_tipo_vale)
        REFERENCES tipo_vale(id_tipo_vale),
    FOREIGN KEY (id_utilizador)
        REFERENCES utilizador(id_utilizador)
) ENGINE=InnoDB;

```


X. Script de criação de utilizadores

```
USE nomalia;

-- Roles
DROP ROLE IF EXISTS 'administrador', 'utilizador';
CREATE ROLE 'administrador', 'utilizador';

-- Criação de contas de utilizador
DROP USER IF EXISTS 'admin1'@'localhost', 'user1'@'localhost';
CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'Admin123#';
CREATE USER 'user1'@'localhost' IDENTIFIED BY 'User123#';

-- Associação de roles
GRANT 'administrador' TO 'admin1'@'localhost';
SET DEFAULT ROLE 'administrador' TO 'admin1'@'localhost';

GRANT 'utilizador' TO 'user1'@'localhost';
SET DEFAULT ROLE 'utilizador' TO 'user1'@'localhost';

-- RM 13 (Seguir utilizadores,deixar de seguir e visualizar seguidores)
GRANT INSERT, SELECT, DELETE ON segue TO 'utilizador';

-- RM 14 e 15 (Criar, editar e visualizar publicações e viagens)
GRANT SELECT, UPDATE ON viagem TO 'utilizador';
GRANT SELECT, UPDATE ON publicacao TO 'utilizador';
GRANT EXECUTE ON PROCEDURE RegistrarViagemPublicacao TO 'utilizador';

-- RM 16 e 17 (Inserção, edição e visualização de comentários)
GRANT SELECT, UPDATE ON comentario TO 'utilizador';
GRANT EXECUTE ON PROCEDURE CriarComentario TO 'utilizador';

-- RM 18 (Inserir, editar e visualizar dados do perfil)
GRANT SELECT (id_utilizador, nome_utilizador, email, biografia, pontos)
ON utilizador
TO 'utilizador';
GRANT EXECUTE ON PROCEDURE AtualizarPerfil TO 'utilizador';

-- RM 19 (Trocar pontos por vales de desconto)
GRANT SELECT ON tipo_vale TO 'utilizador';
GRANT SELECT ON vale_de_desconto TO 'utilizador';
GRANT EXECUTE ON PROCEDURE TrocarValeDesconto TO 'utilizador';
GRANT EXECUTE ON FUNCTION PontosDisponiveis TO 'utilizador';

-- RM 20 e RC 26 (O administrador visualizar e remover viagens,publicações e os comentários)
GRANT SELECT, DELETE ON viagem TO 'administrador';
GRANT SELECT, DELETE ON publicacao TO 'administrador';
GRANT SELECT, DELETE ON comentario TO 'administrador';
GRANT SELECT, DELETE ON viagem_cidade TO 'administrador';
```

```
GRANT SELECT, DELETE ON ficheiro_audio_visual TO 'administrador';
GRANT SELECT (id_utilizador, nome_utilizador, email, biografia,pontos)
ON nomalia.utilizador
TO administrador;
```

XI. Povoamento da BD

```
-- Povoamento
```

```
-- TIPOS DE VIAGEM
```

```
INSERT INTO tipo_viagem (id_tipo_viagem, nome_tipo) VALUES
(1, 'Lazer'),
(2, 'Profissional'),
(3, 'Cultural'),
(4, 'Gastronómica');
```

```
-- PAÍSES
```

```
INSERT INTO pais (id_pais, nome_pais) VALUES
(1, 'Portugal'),
(2, 'Espanha'),
(3, 'Islândia'),
(4, 'Italy'),
(5, 'França'),
(6, 'Estados Unidos da América'),
(7, 'Japão'),
(8, 'Brasil');
```

```
-- CIDADES
```

```
INSERT INTO cidade (id_cidade, nome_cidade, id_pais) VALUES
```

```
    -- Portugal
```

```
(1, 'Lisboa', 1),
    (3, 'Porto', 1),
(5, 'Braga', 1),
```

```
    -- Espanha
```

```
(6, 'Barcelona', 2),
(7, 'Madrid', 2),
(8, 'Sevilha', 2),
```

```
    -- Islândia
```

```
(10, 'Reiquiavique', 3),
(11, 'Vik', 3),
```

```
    -- Itália
```

```
(13, 'Roma', 4),
(15, 'Florença', 4),
(16, 'Veneza', 4),
```

```

-- França
(17, 'Paris', 5),
(18, 'Lyon', 5),

-- EUA
(19, 'Nova Iorque', 6),
(20, 'Los Angeles', 6),
(21, 'São Francisco', 6),

-- Japão
(22, 'Tóquio', 7),
(23, 'Quioto', 7),
(24, 'Osaca', 7),

-- Brasil
(25, 'Rio de Janeiro', 8),
(26, 'São Paulo', 8),
(27, 'Salvador', 8);

-- TIPOS DE VALE
INSERT INTO tipo_vale (id_tipo_vale, nome_tipo_vale, descricao_tipo,
valor_pontos_tipo) VALUES
(1, 'hotel', 'Desconto de 10% em parceiros.', 100),
(2, 'atividade', 'Desconto de 15% em parceiros.', 150);

-- UTILIZADORES
-- (as palavras-passe serão cifradas pelo trigger)
INSERT INTO utilizador (id_utilizador, nome_utilizador, palavra_passe,
email, biografia, pontos) VALUES
(1, 'soraia_faria', 'PasswordMuitoSegura#123', 'sfaria@gmail.com',
'Apaixonada por viagens.', 120),
(2, 'matheus_azevedo', 'PasswordMuitoSegura#456', 'matheus@gmail.com',
'Explorador urbano.', 80),
(3, 'miguel_santos', 'PasswordMuitoSegura#789', 'miguel@gmail.com',
'Viajar é viver.', 60),
(4, 'joana_mendes', 'PasswordUltraSegura#ABC', 'joana@gmail.com',
'Cultura e gastronomia.', 95),
(5, 'carolina_ribeiro', 'PasswordMuitoSegura#477',
'carolina.ribeiro@gmail.com', 'Amante de viagens culturais e
gastronómicas', 70),
(6, 'admin_nomalia', 'AdminPasswordSuperSafe#1', 'admin@nomalia.com',
'Administrador.', 0);

-- VIAGENS
INSERT INTO viagem (id_viagem, data_partida, data_chegada,
id_tipo_viagem, id_utilizador) VALUES
(1, '2024-02-10', '2024-02-18', 3, 1),

```

```
(2, '2023-09-10', '2023-09-15', 1, 2),
(3, '2024-03-01', '2024-03-10', 3, 4),
(4, '2024-04-05', '2024-04-20', 2, 1),
(5, '2024-05-01', '2024-05-12', 1, 3),
(6, '2024-06-10', '2024-06-15', 4, 5),
(7, '2024-07-01', '2024-07-07', 1, 2),
(8, '2024-08-10', '2024-08-25', 1, 3);
```

```
-- ASSOCIAÇÃO VIAGEM CIDADE
```

```
INSERT INTO viagem_cidade (id_viagem, id_cidade) VALUES
```

```
(1, 10),
(1, 11),
```

```
(2, 6),
(2, 8),
```

```
(3, 12),
(3, 15),
(3, 16),
```

```
(4, 19),
(4, 20),
(4, 21),
```

```
(5, 22),
(5, 23),
(5, 24),
(6, 17),
(6, 18),
```

```
(7, 1),
(7, 5),
```

```
(8, 25),
(8, 27);
```

```
-- PUBLICAÇÕES
```

```
INSERT INTO publicacao (id_publicacao, data, descricao, id_viagem,
id_utilizador) VALUES
```

```
(1, '2024-02-15', 'As paisagens da Islândia são irreais.', 1, 1),
(2, '2023-09-14', 'Barcelona é cheia de energia.', 2, 2),
(3, '2024-03-06', 'Roma é história viva.', 3, 4),
(4, '2024-04-12', 'Viagem profissional intensa.', 4, 1),
(5, '2024-05-07', 'Japão superou expectativas.', 5, 3),
(6, '2024-06-13', 'Gastronomia francesa incrível.', 6, 5),
(7, '2024-07-04', 'Portugal é sempre especial.', 7, 2),
(8, '2024-08-18', 'Brasil é pura alegria.', 8, 3);
```

```
-- FICHEIROS ÁUDIO-VISUAIS
```

```
INSERT INTO ficheiro_audio_visual (id_ficheiro, ficheiro_audiovisual,
id_publicacao)
VALUES
```

```
(1, 'media/foto1.jpg', 1),
(2, 'media/video1.mp4', 1),
(3, 'media/foto2.jpg', 2),
(4, 'media/lisboa1.jpg', 3),
(5, 'media/trabalho1.jpg', 4),
(6, 'media/neve1.jpg', 5),
(7, 'media/neve2.mp4', 5),
(8, 'media/paris.jpg', 6),
(9, 'media/roma.jpg', 7),
(10, 'media/japao.mp4', 8);
```

```
-- COMENTÁRIOS
```

```
INSERT INTO comentario (id_comentario, comentario, id_utilizador,
id_publicacao) VALUES
```

```
(1, 'Que lugar incrível!', 2, 1),
(2, 'Está na minha lista!', 3, 1),
(3, 'Barcelona nunca falha.', 1, 2),
(4, 'Roma é eterna.', 2, 3),
(5, 'Boa sorte na viagem!', 4, 4),
(6, 'Japão parece mágico.', 1, 5),
(7, 'Comida francesa é única.', 3, 6),
(8, 'Portugal é casa.', 5, 7);
```

```
-- SEGUIDORES
```

```
INSERT INTO segue (id_utilizador, id_seguidor) VALUES
```

```
(1, 2),
(1, 3),
(2, 1),
(3, 4),
(4, 1),
(5, 2);
```

```
-- VALES DE DESCONTO
```

```
INSERT INTO vale_de_desconto (id_vale_de_desconto, usado, id_tipo_vale,
id_utilizador) VALUES
```

```
(1, FALSE, 1, 1),
(2, TRUE, 1, 2),
(3, FALSE, 2, 4);
```

XII. Tamanho ocupado por cada relação na BD

Relação/Entidade	Tamanho por registo	Nº previsto de resgistos	Espaço Total
segue	8 B	500	4 000 B
viagem_cidade	8 B	200	1 600 B
tipo_viagem	50 B	10	500 B
utilizador	567 B	100	56 700 B
viagem	18 B	200	3 600 B
publicacao	517 B	400	206 800 B
vale_de_desconto	13 B	100	1 300 B
tipo_vale	356 B	10	3 560 B
comentario	58 B	500	29 000 B
ficheiro_audiovisual	410 B	200	82 000 B
cidade	54 B	50	2 700 B
pais	50 B	20	1 000 B
Total estimado	--	--	392 760 B

XIII. Tradução das Interrogações

```
USE nomalia;

-- I1 Gestão de Seguidores/Seguidos
SELECT seguido.nome_utilizador AS NomeSeguido
FROM segue s
INNER JOIN utilizador seguido ON s.id_utilizador =
seguido.id_utilizador
WHERE s.id_seguidor = 1;

SELECT u_seguidor.nome_utilizador AS NomeSeguidor
FROM segue s
INNER JOIN utilizador u_seguidor ON s.id_seguidor =
u_seguidor.id_utilizador
WHERE s.id_utilizador = 1;

-- I2 e I3 Gestão de Publicações e Viagens
CALL RegistrarViagemPublicacao(3, 1, '2025-11-16', '2025-11-23', 'A
cidade é realmente um paraíso!');

-- I4 Consulta de Comentários
SELECT u.nome_utilizador, p.descricao AS DescricaoPublicacao,
c.comentario AS TextoComentario
FROM utilizador u
JOIN comentario c ON u.id_utilizador = c.id_utilizador
JOIN publicacao p ON c.id_publicacao = p.id_publicacao;

-- I5 Gestão de Comentários
CALL CriarComentario(1, 1, 'Que lindo! Adorei a foto.');
```

```
-- I6 Gestão do Perfil do Utilizador
SELECT nome_utilizador, email, biografia, pontos
FROM utilizador
WHERE id_utilizador = 2;

CALL AtualizarPerfil(2, 'Explorador nato e apaixonado por
fotografia!');
```

```
UPDATE utilizador
SET biografia = NULL
WHERE id_utilizador = 2;

-- I7 Consulta de Vales
SELECT u.nome_utilizador, t.valor_pontos_tipo
FROM utilizador u
JOIN vale_de_desconto v ON u.id_utilizador = v.id_utilizador
JOIN tipo_vale t ON v.id_tipo_vale = t.id_tipo_vale;

-- I8 Administração de publicações/comentários
```

```
DELETE FROM comentario
WHERE id_publicacao IN (
    SELECT id_publicacao
    FROM publicacao
    WHERE id_viagem = 1
);

DELETE FROM ficheiro_audio_visual
WHERE id_publicacao IN (
    SELECT id_publicacao
    FROM publicacao
    WHERE id_viagem = 1
);

DELETE FROM publicacao
WHERE id_viagem = 1;

DELETE FROM viagem_cidade
WHERE id_viagem = 1;

DELETE FROM viagem
WHERE id_viagem = 1;
```


XIV. Procedimentos, Funções e Gatilhos

```
USE nomalia;

-- Procedimentos

-- DROP PROCEDURE IF EXISTS TrocarValeDesconto;
DELIMITER $$

CREATE PROCEDURE TrocarValeDesconto(
    IN p_id_utilizador INT,
    IN p_id_tipo_vale INT
)
BEGIN
    DECLARE custo INT;
    DECLARE saldo INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Erro na troca de vale.';
    END;

    SELECT valor_pontos_tipo INTO custo
    FROM tipo_vale
    WHERE id_tipo_vale = p_id_tipo_vale;

    IF custo IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Tipo de vale inexistente.';
    END IF;

    SET saldo = PontosDisponiveis(p_id_utilizador);

    IF saldo < custo THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Pontos insuficientes.';
    END IF;

    START TRANSACTION;

    UPDATE utilizador
    SET pontos = pontos - custo
    WHERE id_utilizador = p_id_utilizador;
```

```

            INSERT INTO vale_de_desconto (usado, id_tipo_vale,
id_utilizador)
            VALUES (FALSE, p_id_tipo_vale, p_id_utilizador);

        COMMIT;
    END $$

DELIMITER ;

-- DROP PROCEDURE IF EXISTS RegistrarViagemPublicacao;
DELIMITER $$

CREATE PROCEDURE RegistrarViagemPublicacao(
    IN p_id_utilizador INT,
    IN p_id_tipo_viagem INT,
    IN p_data_partida DATE,
    IN p_data_chegada DATE,
    IN p_descricao VARCHAR(500)
)
BEGIN
    DECLARE v_id_viagem INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Erro ao registrar viagem e publicação.';
    END;

    START TRANSACTION;

    INSERT INTO viagem (data_partida, data_chegada, id_tipo_viagem,
id_utilizador)
        VALUES (p_data_partida, p_data_chegada, p_id_tipo_viagem,
p_id_utilizador);

    SET v_id_viagem = LAST_INSERT_ID();

    INSERT INTO publicacao (data, descricao, id_viagem,
id_utilizador)
        VALUES (CURDATE(), p_descricao, v_id_viagem, p_id_utilizador);

    COMMIT;
END $$

DELIMITER ;

-- DROP PROCEDURE IF EXISTS AtualizarPerfil;
DELIMITER $$

CREATE PROCEDURE AtualizarPerfil(
    IN p_id_utilizador INT,
    IN p_biografia VARCHAR(400))

```

```

BEGIN
    UPDATE utilizador
    SET biografia = p_biografia
    WHERE id_utilizador = p_id_utilizador ;
END $$

DELIMITER ;

-- Procedimento CriarComentario
-- DROP PROCEDURE IF EXISTS CriarComentario
DELIMITER $$

CREATE PROCEDURE CriarComentario(
    IN p_id_utilizador INT,
    IN p_id_publicacao INT,
    IN p_texto VARCHAR(500)
)
BEGIN
    INSERT INTO comentario (id_utilizador, id_publicacao, comentario)
    VALUES (p_id_utilizador, p_id_publicacao, p_texto);
END $$

DELIMITER ;

-- Funções
-- DROP FUNCTION IF EXISTS PontosDisponiveis;
DELIMITER $$

CREATE FUNCTION PontosDisponiveis(p_id_utilizador INT)
RETURNS INT
READS SQL DATA
BEGIN
    DECLARE saldo INT;

    SELECT pontos INTO saldo
    FROM utilizador
    WHERE id_utilizador = p_id_utilizador;

    RETURN IFNULL(saldo, 0);
END $$

DELIMITER ;

-- Triggers

-- DROP TRIGGER IF EXISTS ValidaPasse;
DELIMITER $$

CREATE TRIGGER ValidaPasse
BEFORE INSERT ON utilizador
FOR EACH ROW
BEGIN

```

```

        IF CHAR_LENGTH(NEW.palavra_passe) < 15 THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'A palavra-passe deve ter pelo menos 15
caracteres.';
        END IF;

        SET NEW.palavra_passe = SHA2(NEW.palavra_passe, 256);
    END $$

DELIMITER ;

-- DROP TRIGGER IF EXISTS ImpedeAutoFollow;
DELIMITER $$

CREATE TRIGGER ImpedeAutoFollow
BEFORE INSERT ON segue
FOR EACH ROW
BEGIN
    IF NEW.id_utilizador = NEW.id_seguidor THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Um utilizador não pode seguir-se a si
próprio.';
    END IF;
END $$

DELIMITER ;

-- DROP TRIGGER IF EXISTS ValidaComentario;
DELIMITER $$

CREATE TRIGGER ValidaComentario
BEFORE INSERT ON comentario
FOR EACH ROW
BEGIN
    IF NEW.comentario IS NULL OR TRIM(NEW.comentario) = '' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'O comentário não pode ser vazio.';
    END IF;
END $$

DELIMITER ;

-- DROP TRIGGER IF EXISTS AtribuiPontosPublicacao
DELIMITER $$

CREATE TRIGGER AtribuiPontosPublicacao
AFTER INSERT ON publicacao
FOR EACH ROW
BEGIN
    UPDATE utilizador
    SET pontos = pontos + 10
    WHERE id_utilizador = NEW.id_utilizador;
END $$

DELIMITER ;

```

```

-- DROP TRIGGER IF EXISTS ValidaAutorComentario;
DELIMITER $$
CREATE TRIGGER ValidaAutorComentario
BEFORE UPDATE ON comentario
FOR EACH ROW
BEGIN
    IF OLD.id_utilizador <> NEW.id_utilizador THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Não é permitido alterar comentários de
outros utilizadores.';
    END IF;
END $$
DELIMITER ;

```